
도커와 쿠버네티스를 활용한
컨테이너 관리
(Docker/Kubernetes)

Contents



01 Docker와 컨테이너

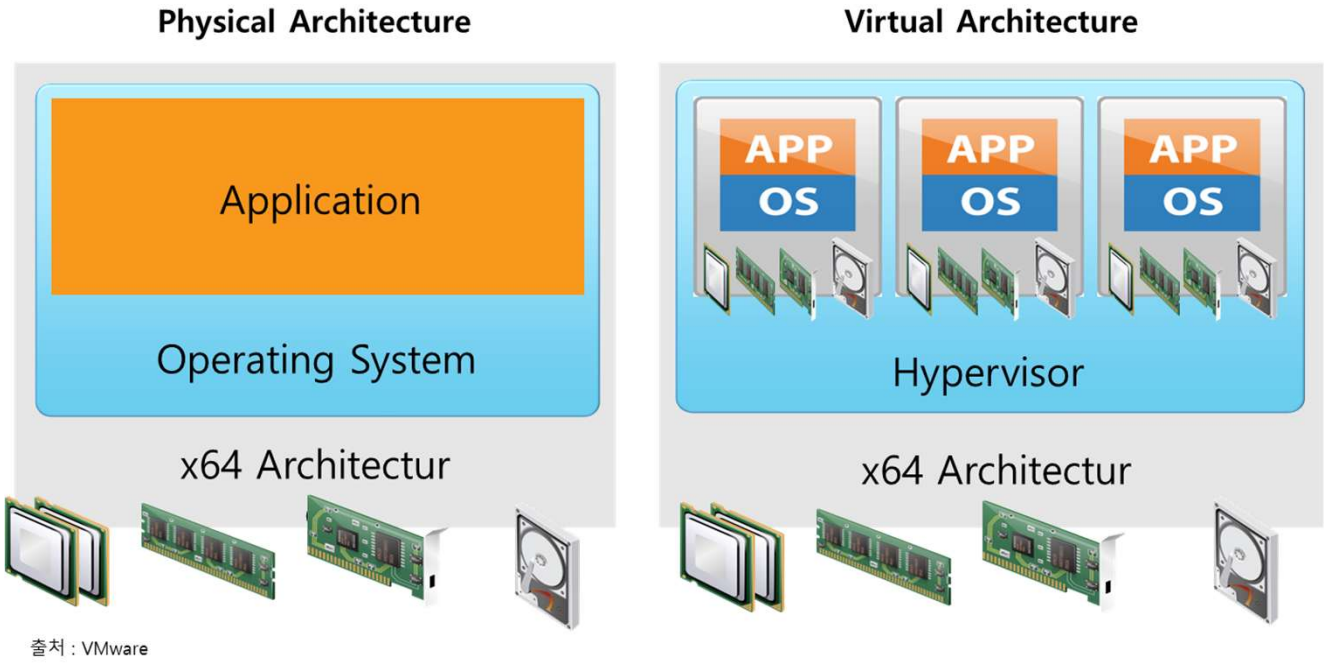
02 Docker Compose

Chapter 1

Docker와 컨테이너

1. Container란

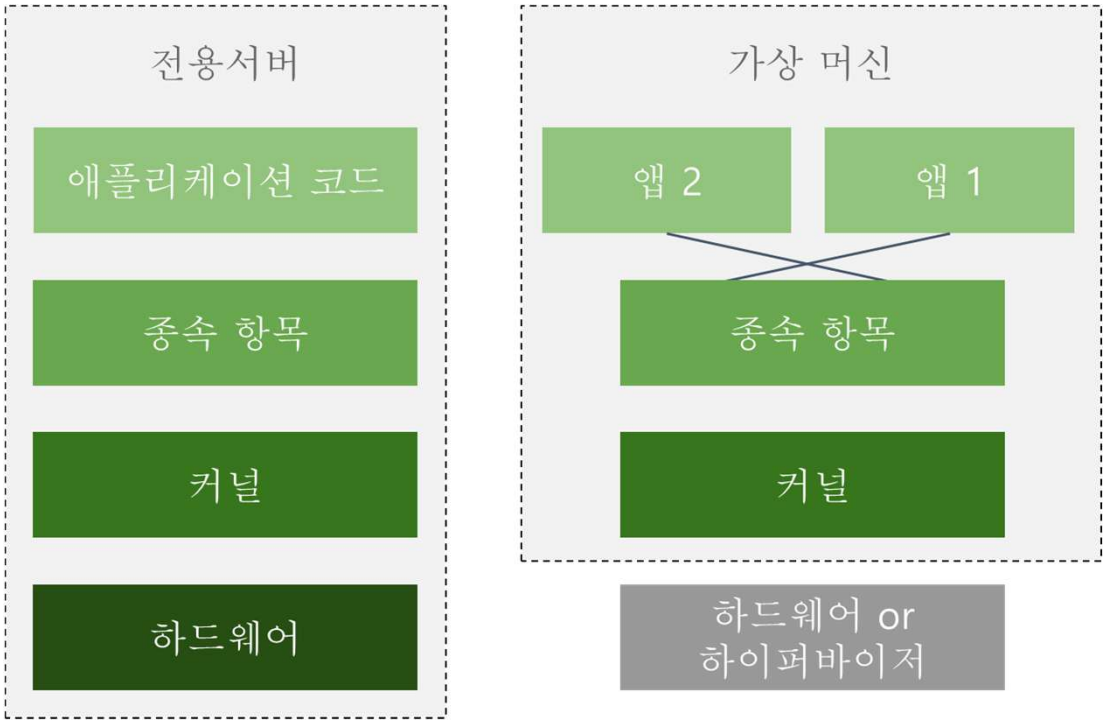
1 물리 서버 vs 가상 머신



출처 : VMware

1. Container란

1 물리 서버 vs 가상 머신



1. Container란

1 물리 서버 vs 가상 머신



1. Container란

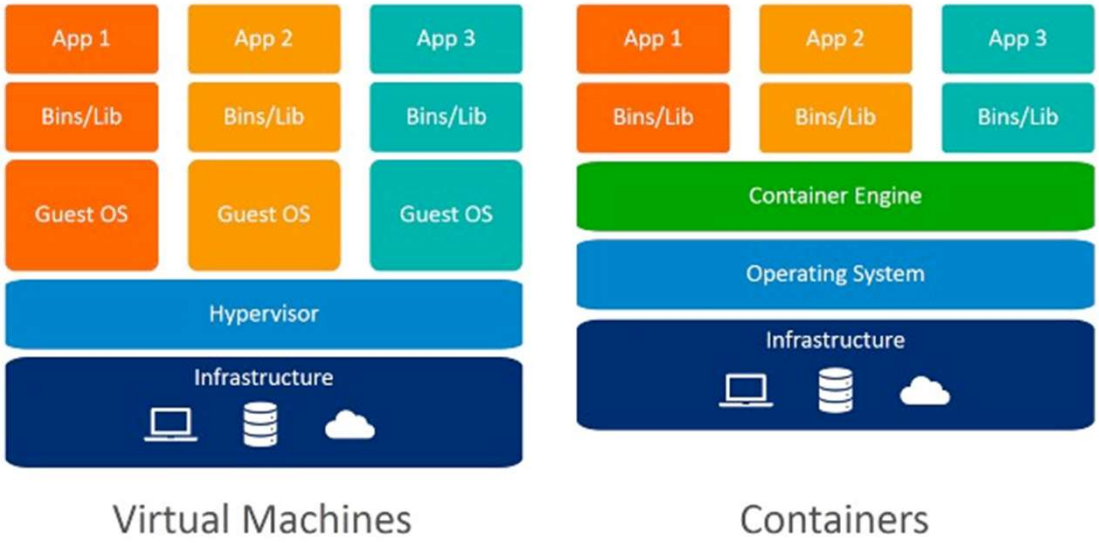
1 물리 서버 vs 가상 머신



1. Container란

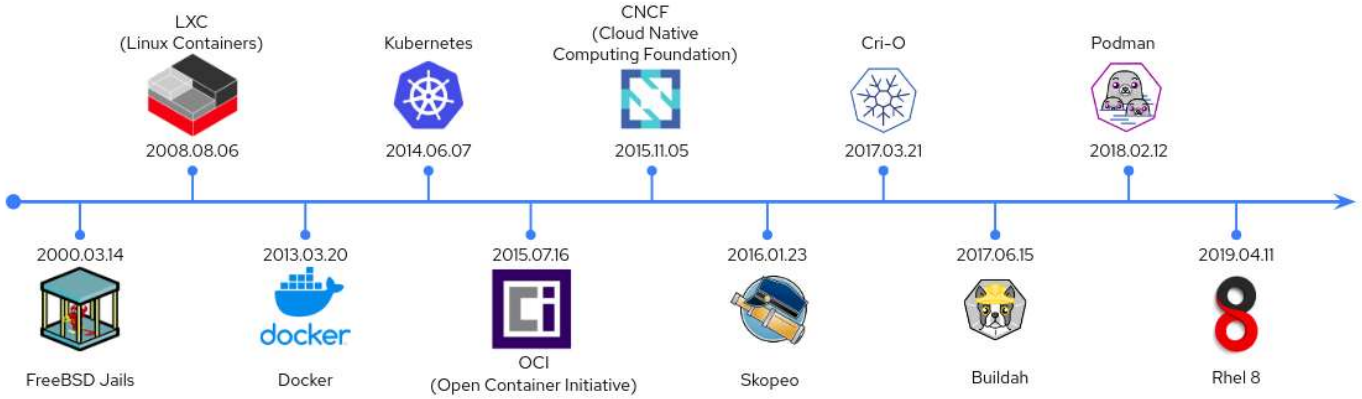
2 Container의 정의

소프트웨어 서비스를 실행하는 데 필요한 특정 버전의 프로그래밍 언어 런타임 및 라이브러리와 같은 종속 항목과 애플리케이션 코드를 함께 포함하는 경량 패키지



1. Container란

3 기술 발전



- ✓ Docker는 2013년 3월 산타클라라에서 열린 Pycon Conference에서 dotCloud 창업자인 Solomon Hykes가 The future of Linux Containers 세션 발표
- ✓ 2013년 10월 Docker로 프로젝트 시작
- ✓ 2014년 6월에 Docker 1.0 발표
- ✓ 2014년 Kubernetes
- ✓ 2015년 OCI구성, CNCF 구성

2. Docker

1 정의

애플리케이션을 빌드하고 컨테이너화하기 위한 오픈 소스 컨테이너화 기술

다양한 프로그램, 실행환경을 컨테이너로 추상화

동일한 인터페이스를 제공하여 프로그램의 배포 및 관리를 단순

백엔드 프로그램, 데이터베이스 서버, 메시지 큐등 어떤 프로그램도 컨테이너로 추상화

어떤 하드웨어 뿐 아니라 AWS, Azure, GCP와 같은 클라우드를 비롯한 어디서든 실행 가능



2. Docker

3 역사

☑ 1. 기술적 배경

- 리눅스 컨테이너(Linux Container, LXC)
  - 리눅스 커널의 cgroups(2007), namespaces(2002) 기능을 이용해 프로세스를 격리할 수 있게 됨.
  - 이 기능들을 조합해 가벼운 운영체제 수준 가상화(OS-level virtualization)가 가능해짐.
  - 2010년대 초까지는 LXC 기반의 컨테이너 관리가 복잡하고 개발자 친화적이지 않음



2. Docker

2 역사

☑ 2. 도커의 탄생 (2013)

- 2013년 3월: Solomon Hykes가 창업한 DotCloud에서 처음 도커 오픈소스 공개
  - 내부적으로 사용하던 “dotCloud” PaaS를 위해 개발한 컨테이너 도구.
  - 당시 이름은 docker.io, 처음에는 LXC를 백엔드로 사용.
- 주요 특징:
  - 이미지 기반 컨테이너 모델
  - Dockerfile로 애플리케이션 정의
  - 명령형이 아닌 선언형 기반 이미지 생성
  - 버전 관리 가능한 컨테이너 이미지



2. Docker

2 역사

☑ 3. 급격한 성장과 독립 (2013~2015)

- 2013년 10월: 도커의 인기가 높아지면서 DotCloud는 회사명을 **Docker Inc.**로 변경.
- 2014년: LXC 대신 자체 엔진인 libcontainer 도입 → 이후 runc로 발전.
- 2015년: Docker Hub, Docker Compose, Docker Swarm 등 생태계 구성 요소 추가.
    - Docker Compose: 멀티컨테이너 앱 정의
    - Docker Swarm: 클러스터링 오케스트레이션 도구
    - Docker Machine: VM에 도커 설치 및 관리



2. Docker

2 역사

☑ 4. 생태계의 확대와 컨테이너 표준화 (2015~2017)

- 2015년 6월: Docker Inc.는 컨테이너 생태계의 표준화를 위해 Open Container Initiative (OCI) 설립
  - Linux Foundation과 함께 컨테이너 이미지 및 실행 런타임 표준 정의
- 2016년: 쿠버네티스(Kubernetes) 등장과 성장 → 도커의 컨테이너 런타임이 표준 런타임으로 채택됨
- 도커의 역할은 플랫폼이 아닌 '컨테이너 런타임' 중심으로 재편됨



2. Docker

2 역사

☑ 5. 컨테이너 오케스트레이션 경쟁 → 쿠버네티스의 승리 (2017~2019)

- Docker Swarm vs Kubernetes 경쟁
  - Kubernetes의 생태계 및 기능 우위로 Kubernetes가 업계 표준으로 자리잡음.
- Docker는 Kubernetes 통합 지원 발표(2017) → 내부적으로도 dockerd를 통한 Kubernetes 지원 제공



2. Docker

2 역사

☑ 5. 컨테이너 오케스트레이션 경쟁 → 쿠버네티스의 승리 (2017~2019)

- Docker Swarm vs Kubernetes 경쟁
  - Kubernetes의 생태계 및 기능 우위로 Kubernetes가 업계 표준으로 자리잡음.
- Docker는 Kubernetes 통합 지원 발표(2017) → 내부적으로도 dockerd를 통한 Kubernetes 지원 제공



2. Docker

2 역사

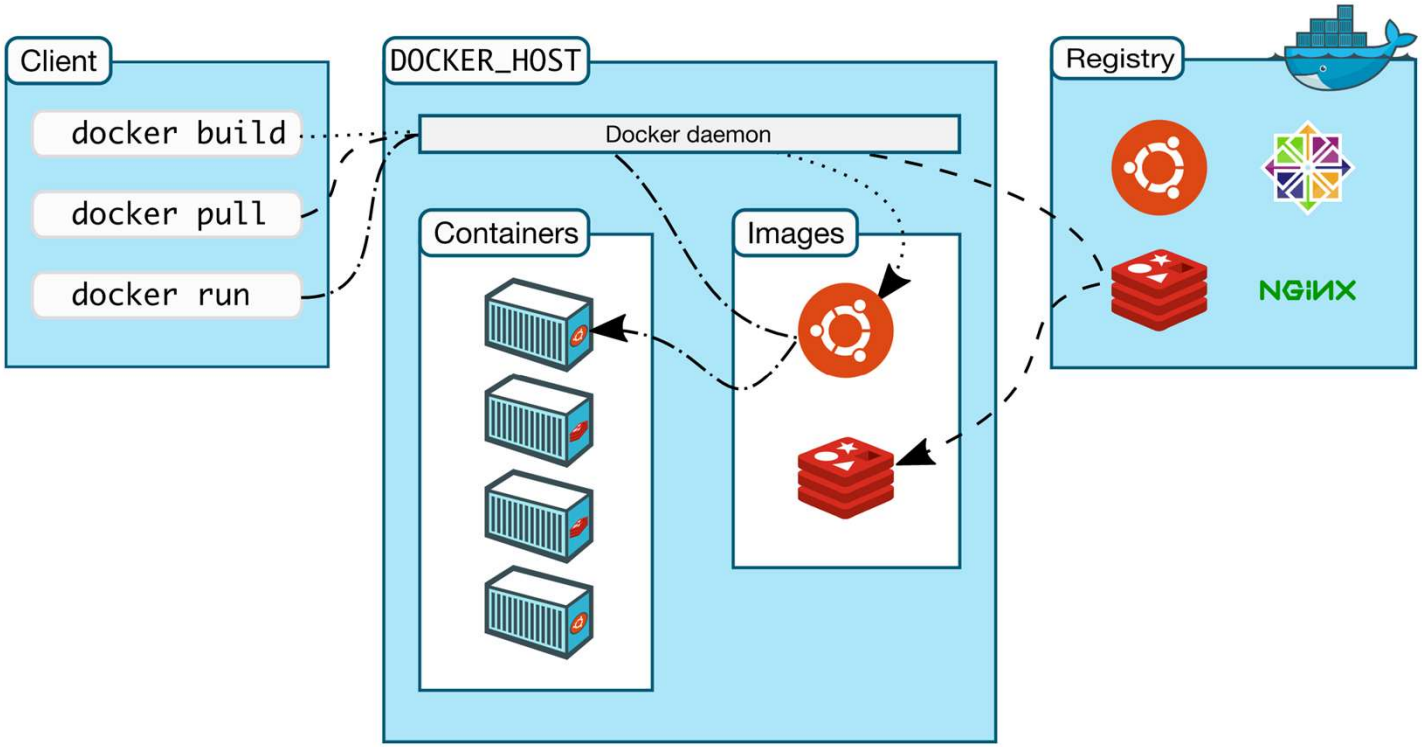
☑ 6. 도커의 분할 및 기업 전략 변화 (2020 이후)

- 2019년: Docker Inc.는 엔터프라이즈 사업부(Docker Enterprise)를 **Mirantis**에 매각
  - Docker는 다시 개발자 중심 도구 개발(Docker CLI, Docker Desktop, Docker Hub 등)에 집중.
- 2020년~현재: Docker Desktop (Mac/Windows 개발 환경) 강화, Docker Hub 프리미엄 기능 추가, BuildKit, Docker Compose v2 등 개선 지속
- 2021년: Docker Desktop 라이선스 변경 (상업적 사용 시 유료화) → 일부 커뮤니티 비판



2. Docker

구성요소



2. Docker

4 이미지 & 컨테이너

✓ 이 미 지

이미지는 컨테이너를 생성할 때 필요한 요소로 컨테이너의 목적에 맞는 바이너리와 의존성이 설치되어 있음 여러 개의 계층으로 된 바이너리 파일로 존재

✓ 컨 테 이 너

호스트와 다른 컨테이너로부터 격리된 시스템 자원과 네트워크를 사용하는 프로세스 이미지는 읽기 전용으로 사용하여 변경사항은 컨테이너 계층에 저장“ 프 로 세 스 를 격 리 하 는 방 식 ”

2. Docker

5 이미지 tag

예)

hifrodo/nginx:1.0

ⓐ ⓑ ⓒ

ⓐ Docker hub 계정 또는 Container Registry

ⓑ Container Repository

ⓑ Image Tag로 보통 버전을 명시 합니다.

2. Docker

6 이미지 저장소(Registry)

이미지를 관리하고 공유하기 위한 서버 application으로 다음과 같은 종류가 있음

Docker hub

AWS ACR

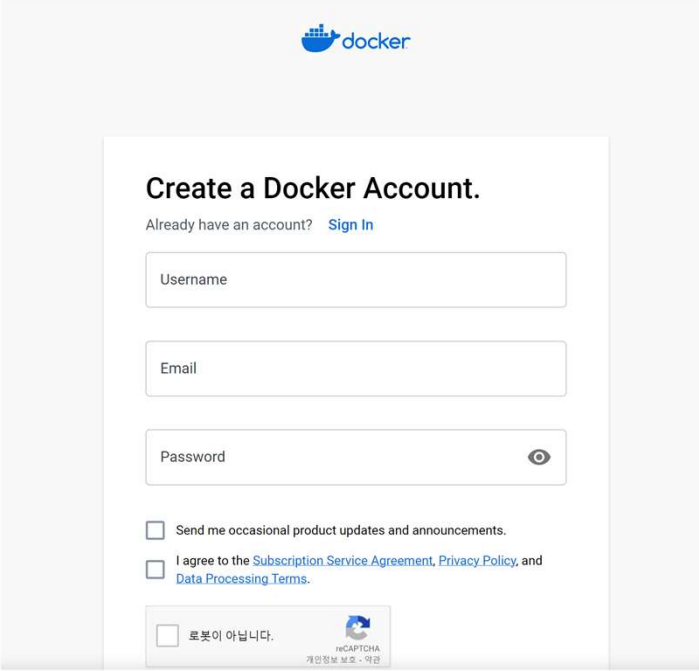
Google GCR

MS Azure Container Registry

2. Docker

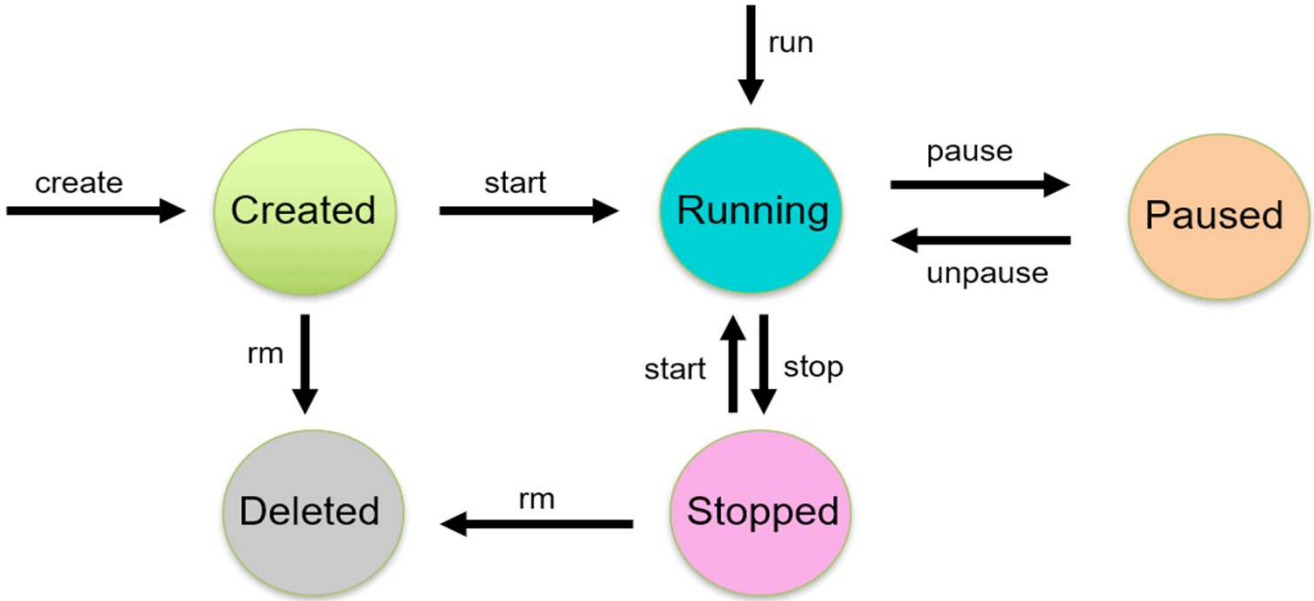
Docker hub 가입하기

https://hub.docker.com/에 접속 상단 Register 버튼을 클릭하여 가입



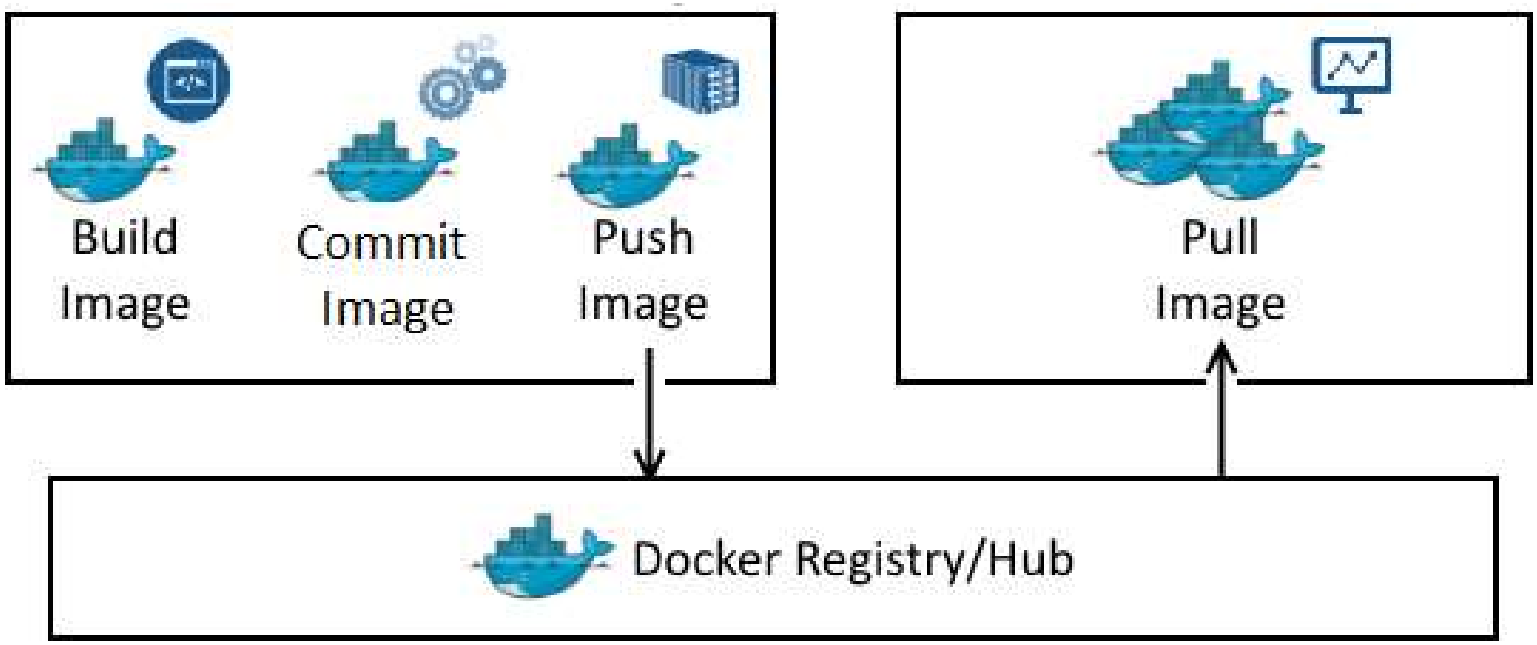
3. Docker 운영

1 Container life Cycle



3. Docker 운영

➋ image pull 및 push



4. Docker 설치

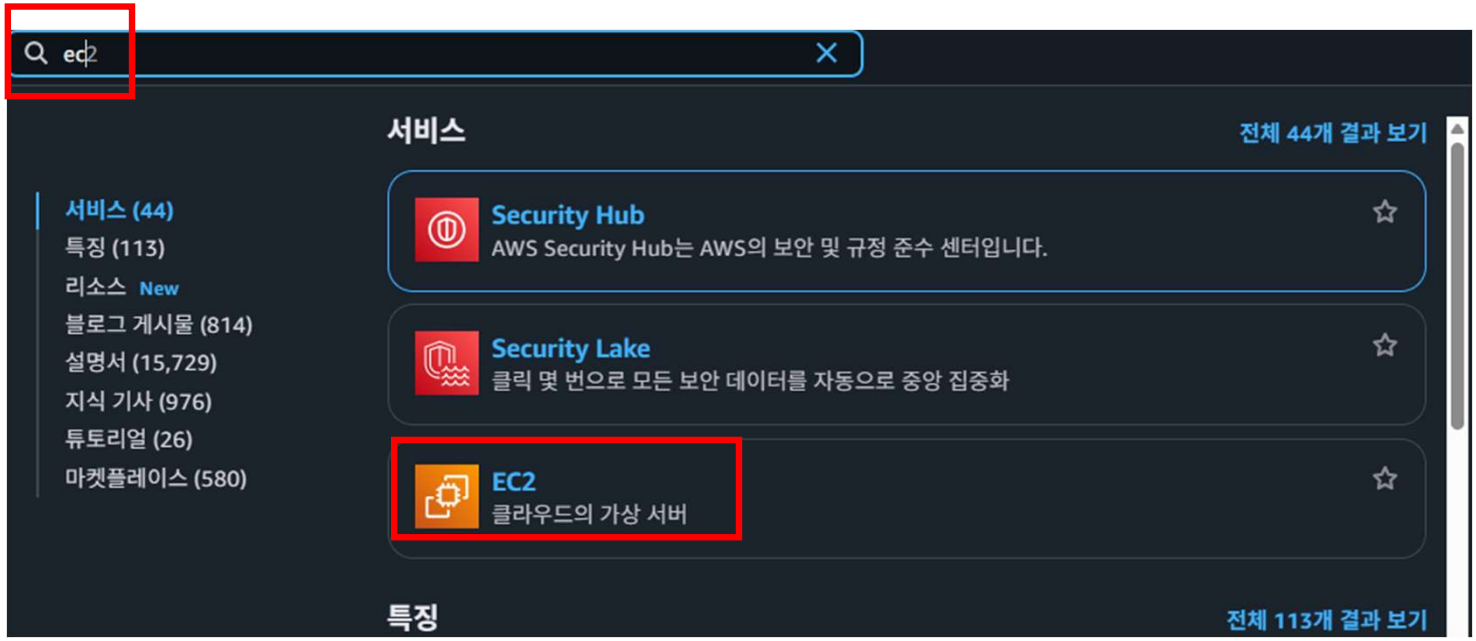
1 AWS VM생성하기

https://aws.amazon.com/ko/console/

4. Docker 설치

1 AWS EC2(VM)생성하기

서비스에서 EC2를 검색합니다. 그리고 EC2를 선택합니다.

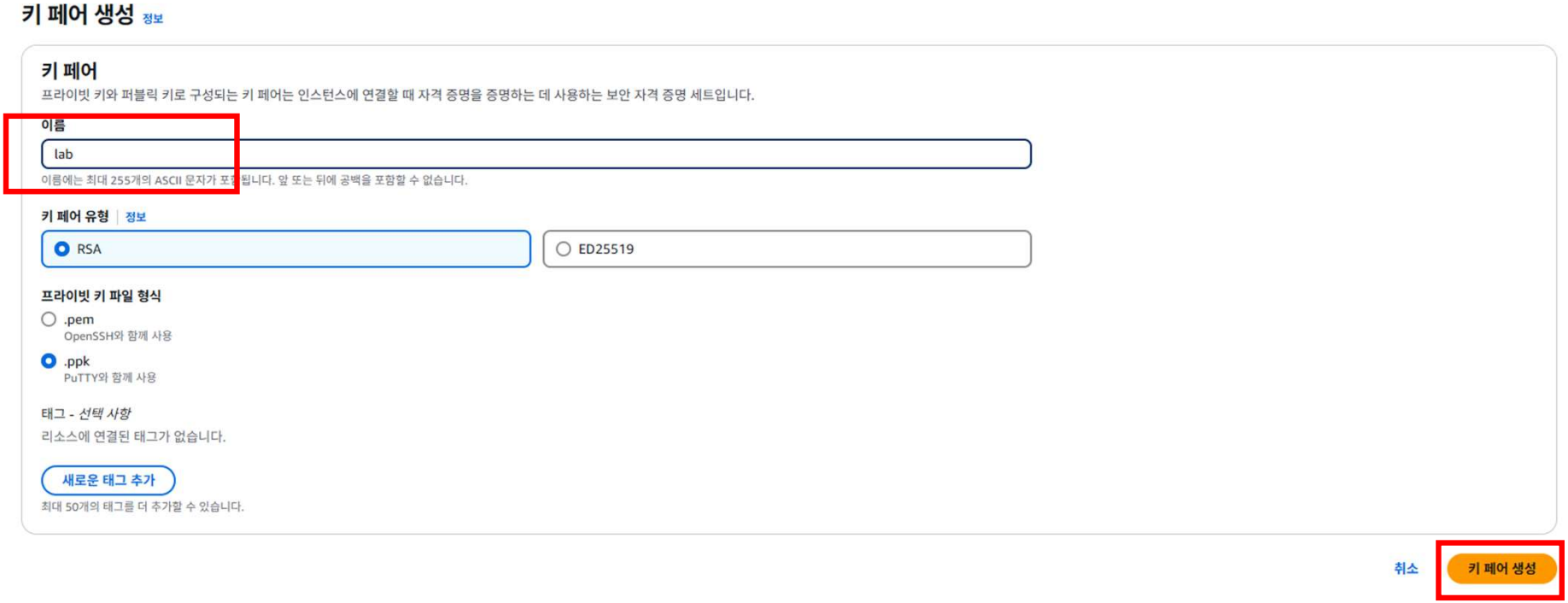


4. Docker 설치

1 AWS EC2(VM)생성하기

키페어 이름을 입력하고 키페이 생성을 클릭 합니다.

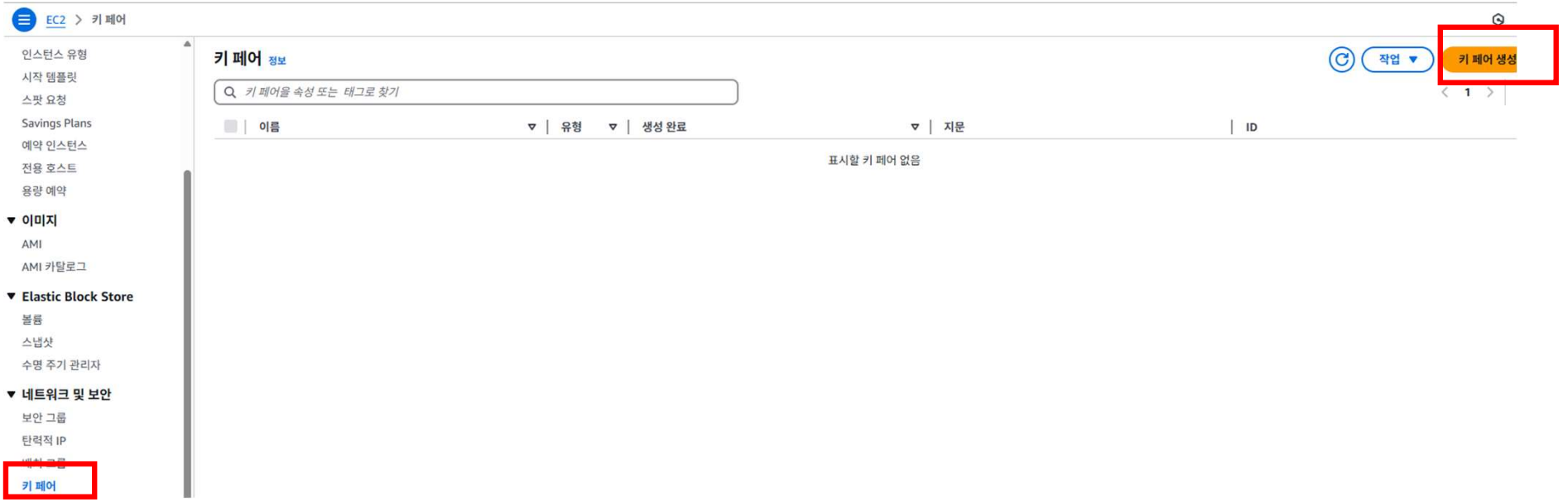
키페어 생성이 되면서 private key가 다운로드 됩니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

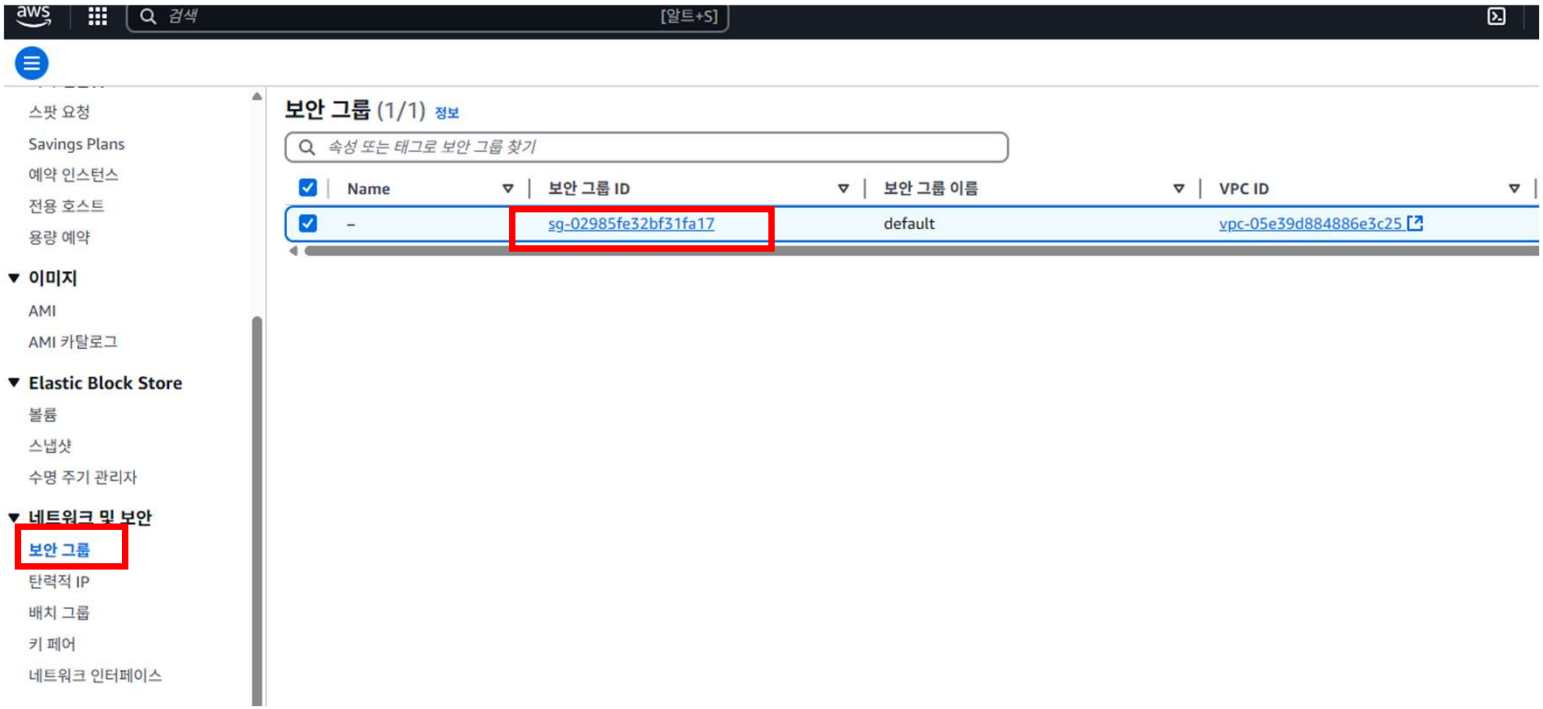
왼쪽 메뉴에서 키 페어를 선택하고 오른 쪽 키 페이 생성을 클릭합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

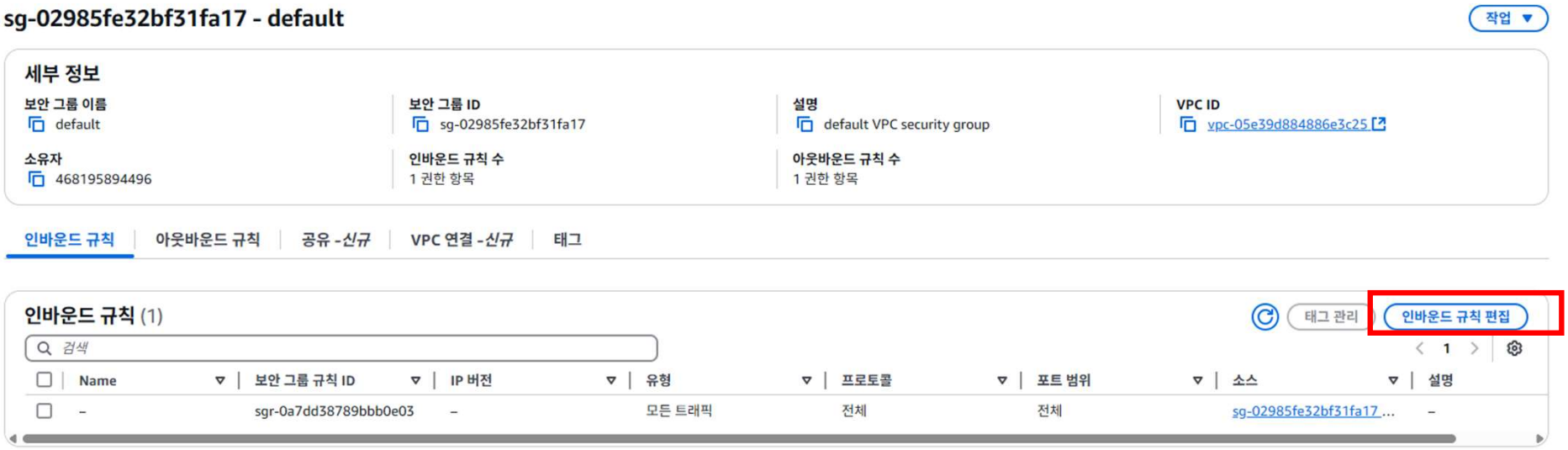
왼쪽 메뉴에서 보안 그룹를 선택하고 보안 그룹 ID를 클릭 합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

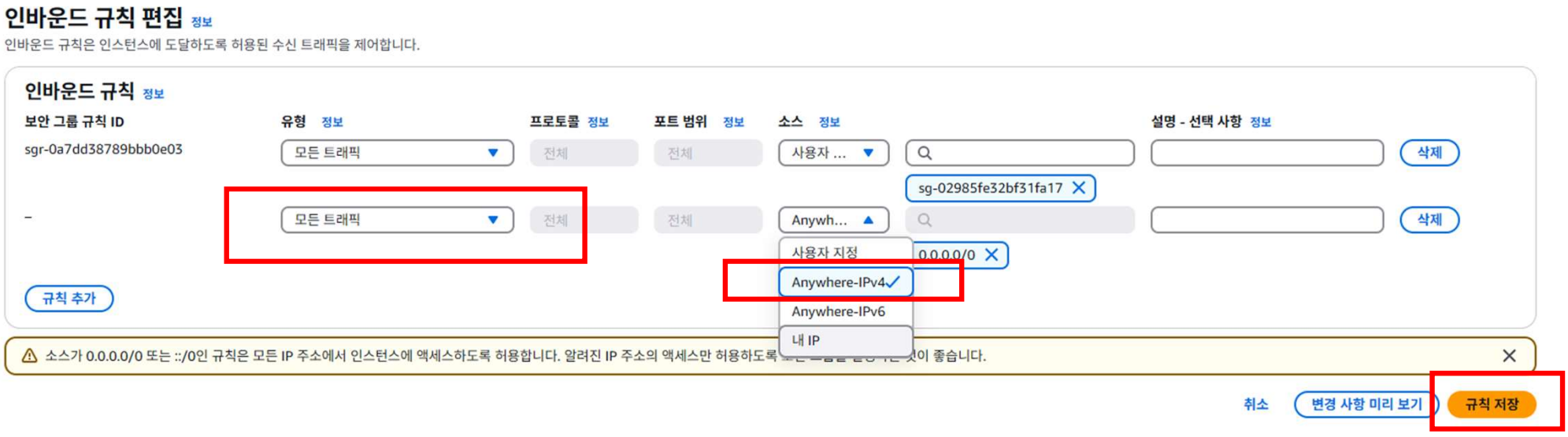
인바운드 규칙 편집을 클릭합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

규칙추가를 클릭하고 유형은 모든 트래픽 소스는 Anywhere IPv4를 선택하고 규칙저장을 클릭합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

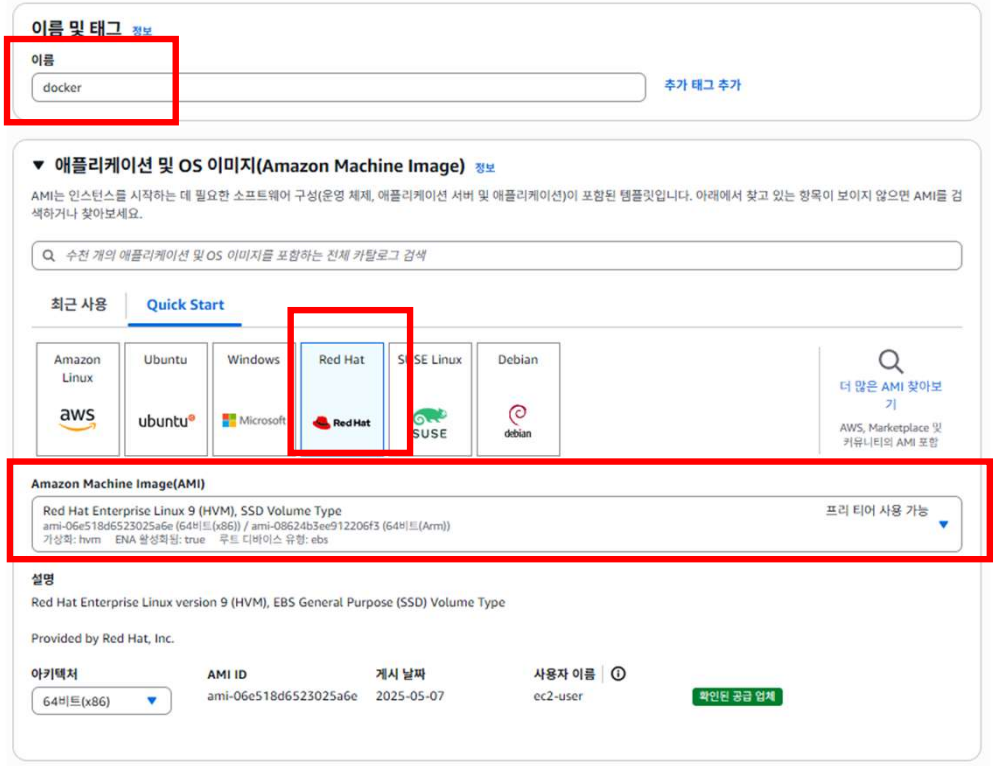
왼쪽 메뉴에서 EC2를 선택하고 인스턴스 시작을 클릭합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

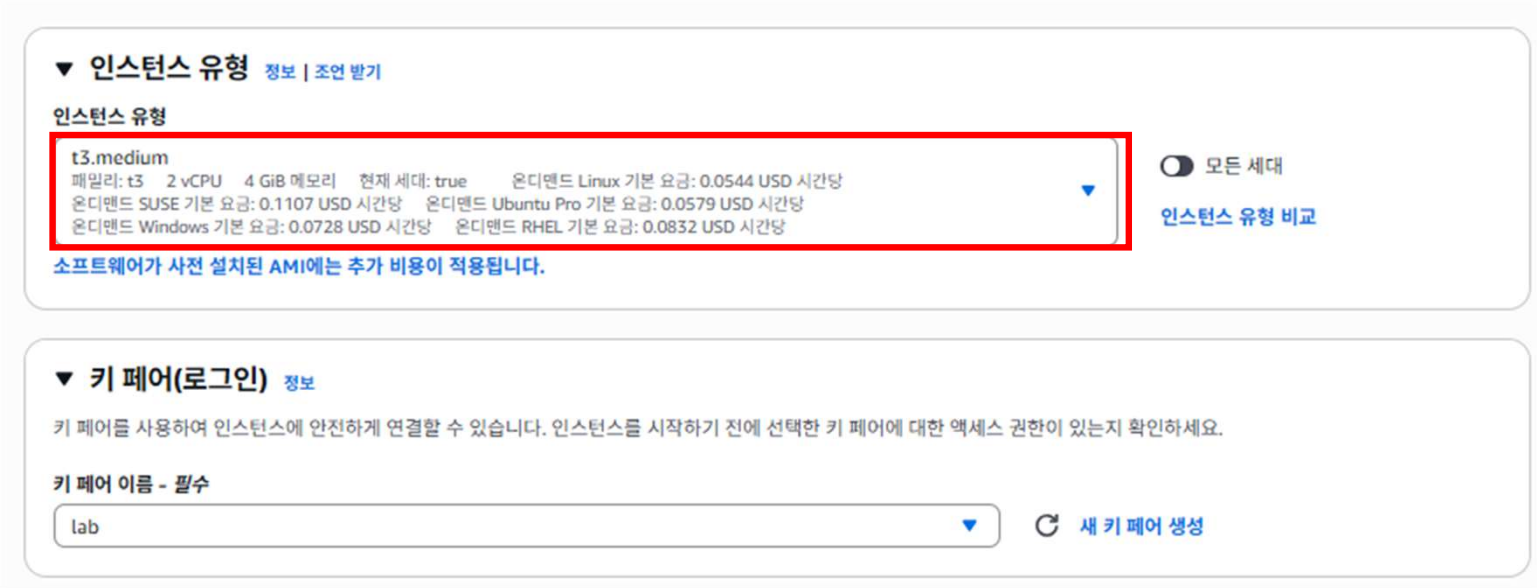
인스턴스 이름은 docker로 입력하고 AMI는 RedHat을 선택 합니다. 그리고 Red Hat Enterprise 9을 선택 합니다. (기본은 10입니다.)



4. Docker 설치

① AWS EC2(VM)생성하기

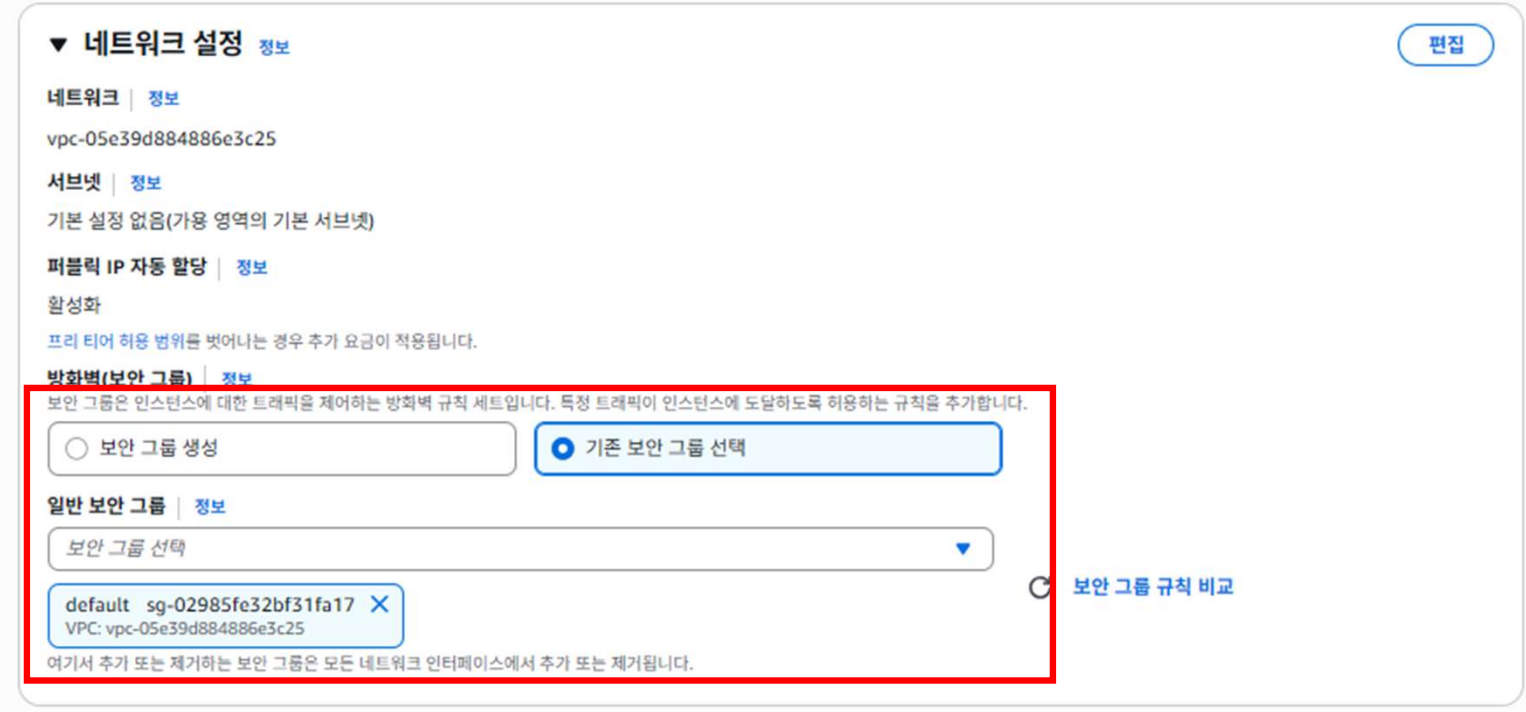
인스턴스 유형은 t3.medium을 선택 합니다. 키 페어는 앞서 만든 키페어를 선택합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

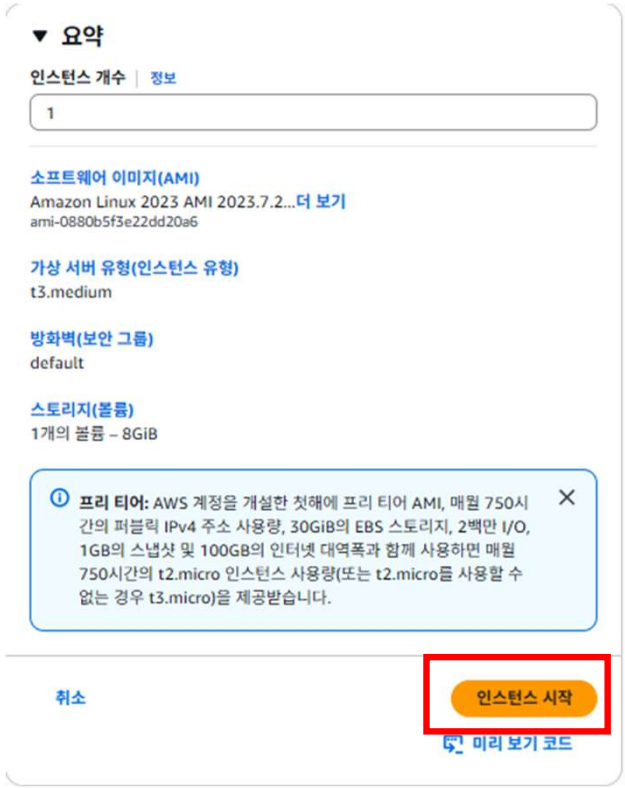
보안 그룹은 기존 보안 그룹을 선택하고 default 보안 그룹을 선택 합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

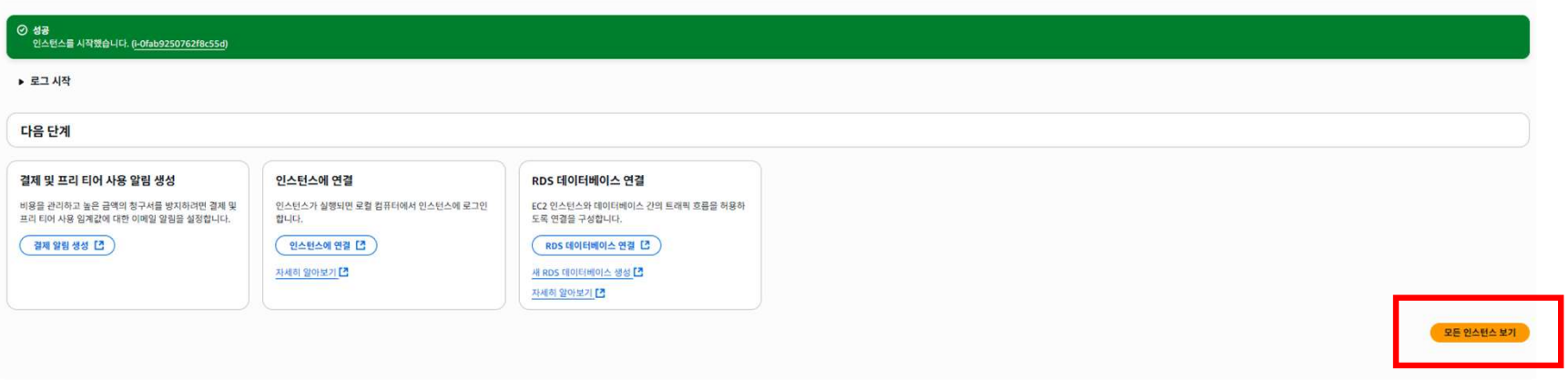
인스턴스 시작을 클릭 합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

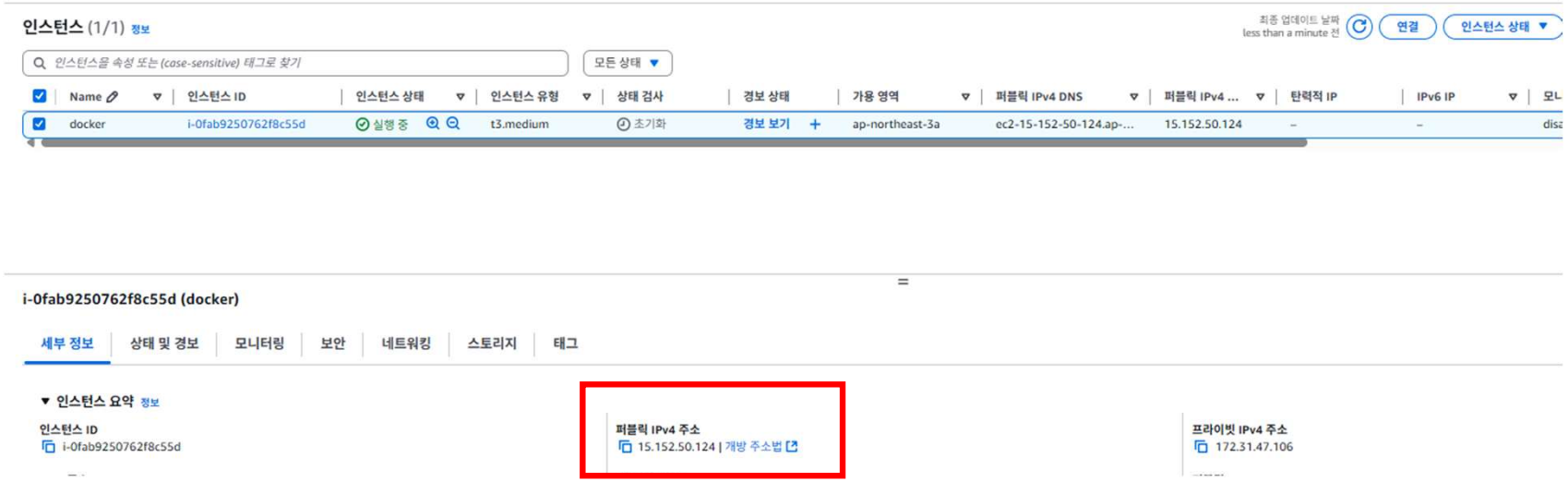
모든 인스턴스 보기를 클릭 합니다.



4. Docker 설치

1 AWS EC2(VM)생성하기

인스턴스 세부 정보에서 퍼블릭 IP를 확인 합니다.

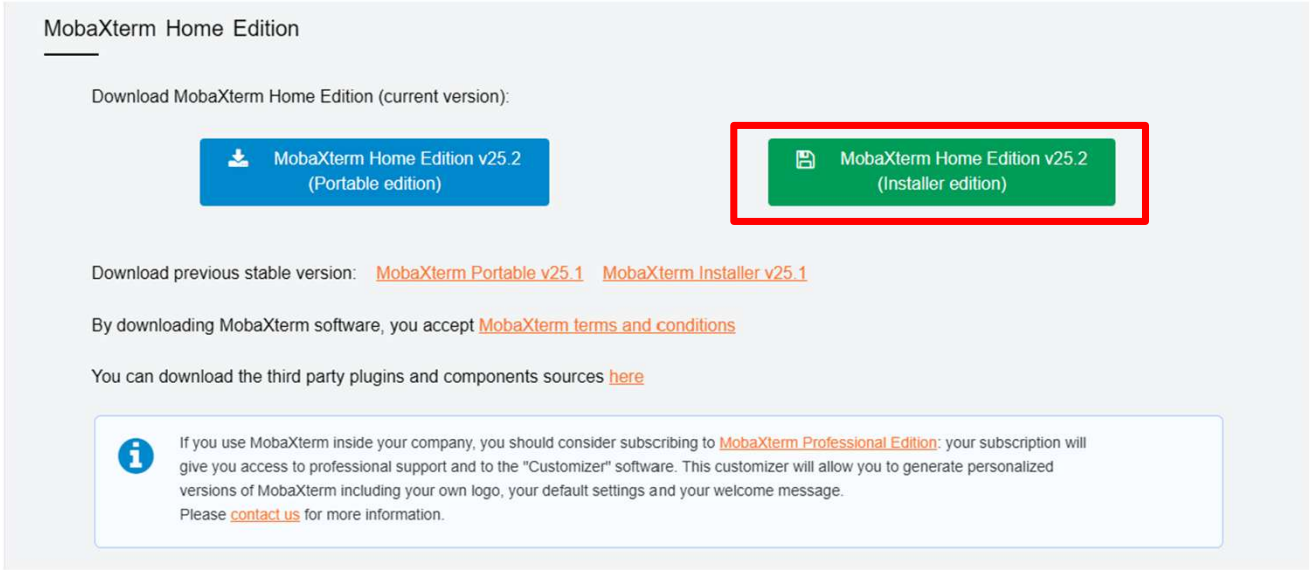


4. Docker 설치

2 SSH 접속하기

SSH 접속을 위해서 아래 사이트에서 Mobaxterm을 다운로드 받아 설치 합니다.

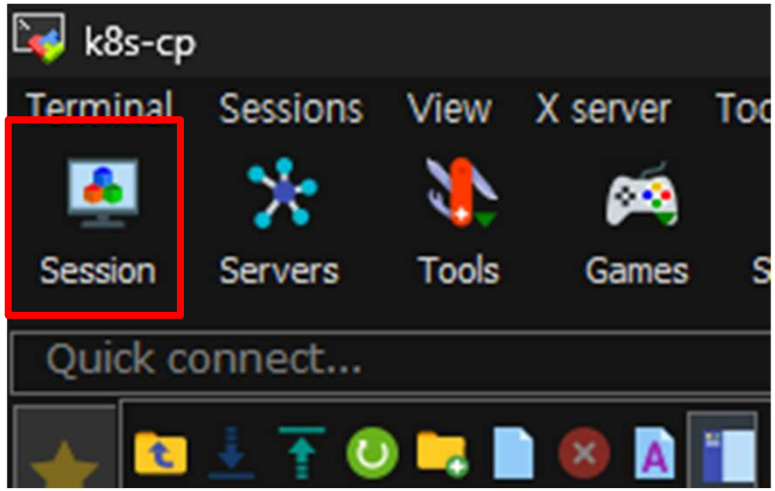
https://mobaxterm.mobatek.net/download-home-edition.html



4. Docker 설치

2 SSH 접속하기

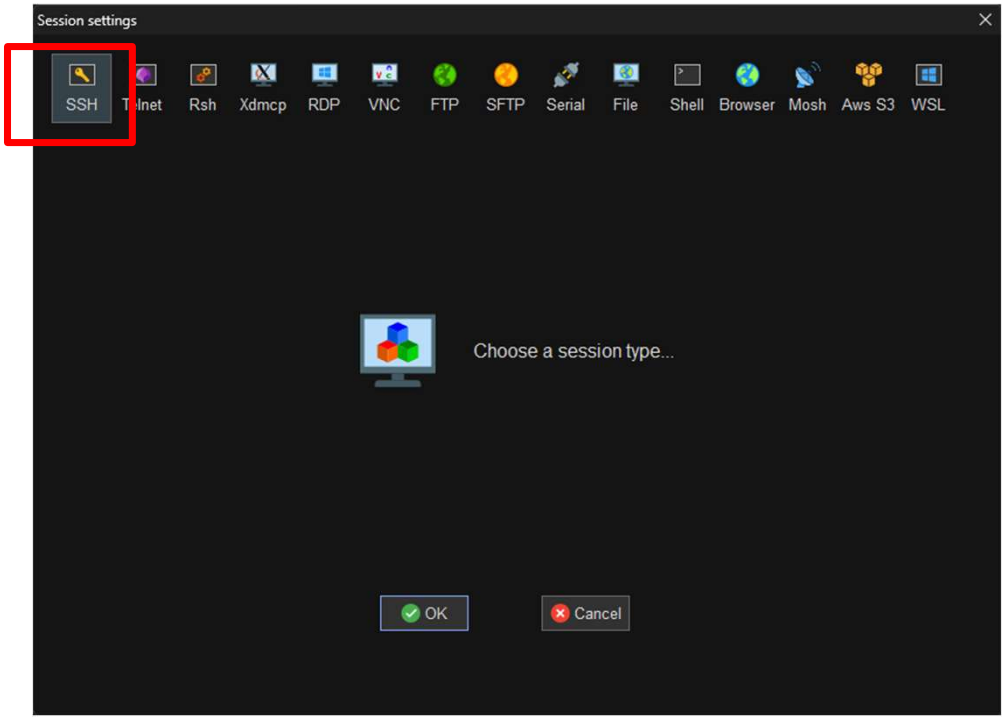
Mobaxterm을 실행하고 우측 상단에 Session을 클릭 합니다.



4. Docker 설치

2 SSH 접속하기

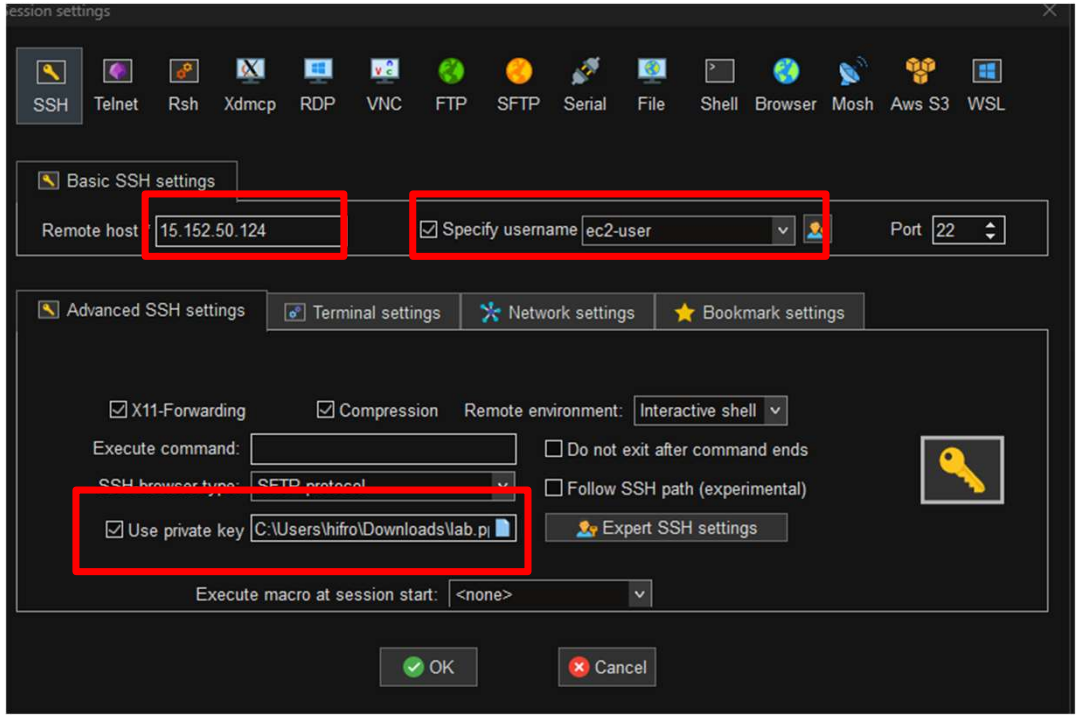
SSH를 클릭 합니다.



4. Docker 설치

2 SSH 접속하기

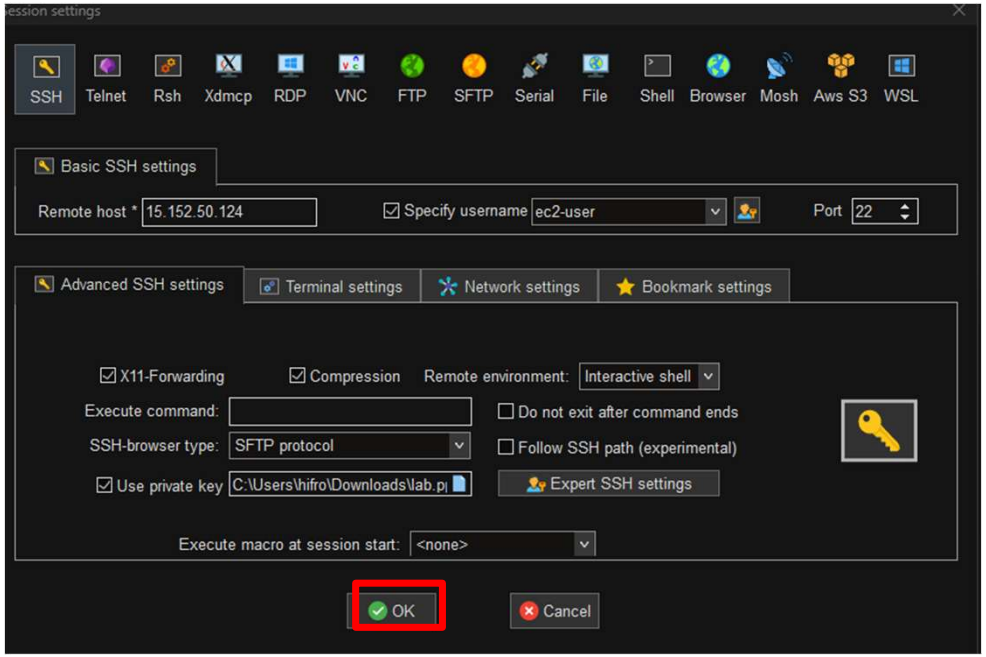
Remote Host에는 생성한 EC2의 공인 IP 를 입력하고 specify username을 선택하고 **ec2-user**를 입력합니다. User private key를 선택하고 앞에서 다운로드 받은 private key를 선택합니다.

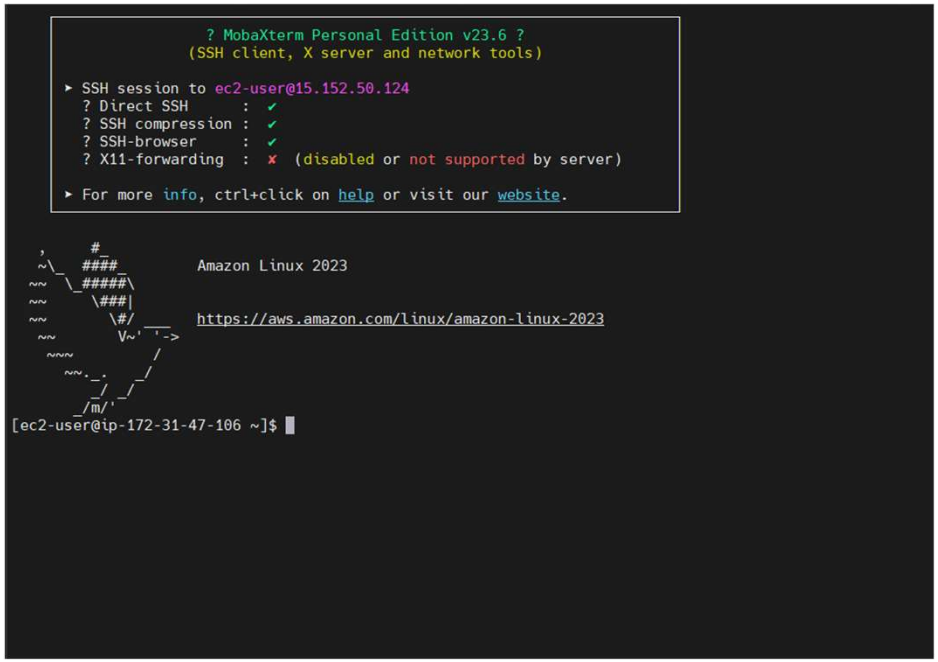


4. Docker 설치

2 SSH 접속하기

입력이 끝나고 OK를 클릭하면 좌측과 같이 접속이 됩니다.

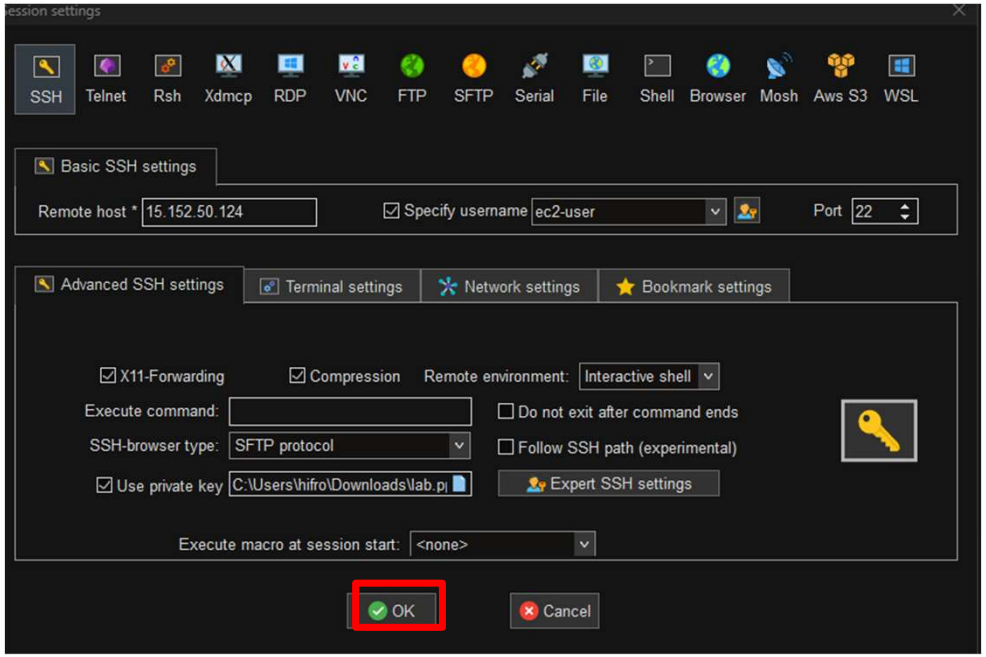


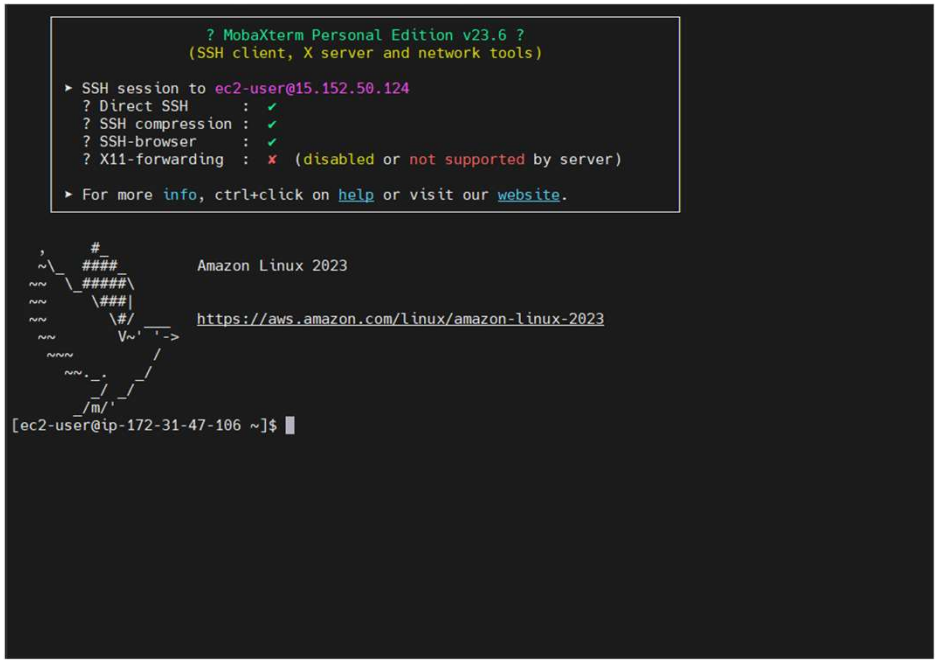


4. Docker 설치

2 SSH 접속하기

입력이 끝나고 OK를 클릭하면 좌측과 같이 접속이 됩니다.





4. Docker 설치

3 Docker 설치 하기

https://docs.docker.com/engine/install/rhel/ 사이트를 참고 해서 Docker를 설치 합니다.

```
# 1. Docker Repository 설정
sudo dnf -y install dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo

# 2. Docker 설치
sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

4. Docker 설치

3 Docker 설치 하기

모든 설정이 완료 되면 SSH 세션을 닫고 다시 접속 합니다.

```
# 3. Docker 활성화
sudo systemctl enable --now docker

#4. 사용자 그룹 추가
sudo usermod -aG docker ec2-user
```

4. Docker 설치

3 Docker 설치 하기

docker version으로 확인 합니다.

```
[ec2-user@ip-172-31-47-106 ~]$ docker version
Client:
 Version:           25.0.8
 API version:       1.44
 Go version:        go1.23.8
 Git commit:        0bab007
 Built:             Fri Apr 11 00:00:00 2025
 OS/Arch:           linux/amd64
 Context:           default

Server:
 Engine:
  Version:          25.0.8
  API version:      1.44 (minimum version 1.24)
  Go version:       go1.23.8
  Git commit:       71907ca
  Built:            Fri Apr 11 00:00:00 2025
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.7.27
  GitCommit:        05044ec0a9a75232cad458027ca83437aae3f4da
 runc:
  Version:          1.2.4
  GitCommit:        6c52b3fc541fb26fe8c374d5f58112a0a5dbda66
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
[ec2-user@ip-172-31-47-106 ~]$
```

5. Container 생성하기

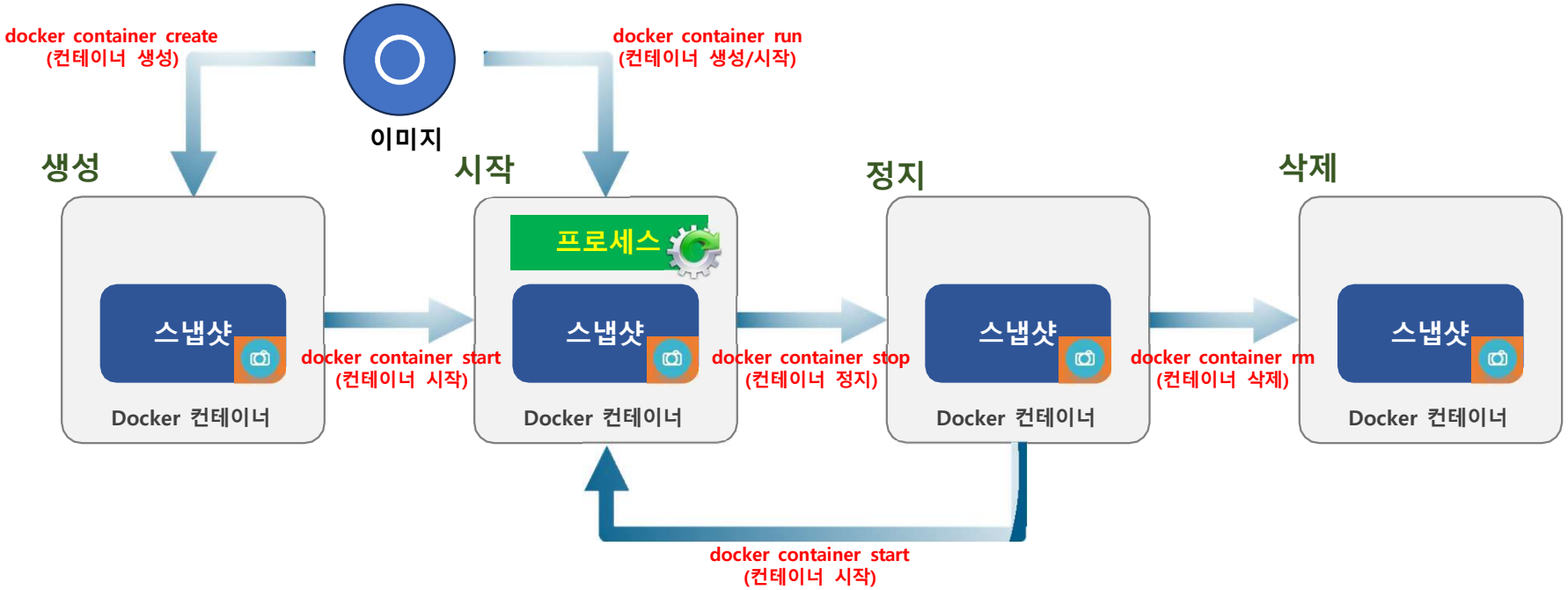
1 Local image 확인하기

docker image ls 로컬 시스템에 존재하는 Docker 이미지 목록을 출력합니다.

```
[ec2-user@ip-172-31-47-106 docker]$ docker image ls
REPOSITORY   TAG       IMAGE ID   CREATED   SIZE
[ec2-user@ip-172-31-47-106 docker]$
```

5. Container 생성하기

1 Container 상태



5. Container 생성하기

② Nginx image pull 하기

docker image pull nginx 명령어로 nginx image를 pull합니다.

```
[ec2-user@ip-172-31-47-106 docker]$ docker image pull nginx
Using default tag: latest
latest: Pulling from library/nginx
254e724d7786: Pull complete
913115292750: Pull complete
3e544d53ce49: Pull complete
4f21ed9ac0c0: Pull complete
d38f2ef2d6f2: Pull complete
40a6e9f4e456: Pull complete
d3dc5ec71e9d: Pull complete
Digest: sha256:c15da6c91de8d2f436196f3a768483ad32c258ed4e1beb3d367a27ed67253e66
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
[ec2-user@ip-172-31-47-106 docker]$ docker image ls
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    a830707172e8   4 weeks ago   192MB
[ec2-user@ip-172-31-47-106 docker]$
```

5. Container 생성하기

3 Nginx Container 생성 하기

docker container ls -a 명령어로 현재 생성되어 있는 Container가 있는지 확인 합니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[ec2-user@ip-172-31-33-52 ~]$
```

5. Container 생성하기

3 Nginx Container 생성 하기

docker container create nginx 명령어로 download 받은 이미지를 이용해서 nginx container를 생성 합니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker container create nginx
7e4a228d5d3833bd4e770ec06abfaa26ea78980b72b2d7dbd291d5469d4985d0
[ec2-user@ip-172-31-33-52 ~]$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS    PORTS     NAMES
7e4a228d5d38   nginx     "/docker-entrypoint.…"   4 seconds ago   Created             funny_babbage
[ec2-user@ip-172-31-33-52 ~]$
```

5. Container 생성하기

4 NGINX Container 실행하기

docker container ls –a 명령어를 통해 container가 생성 된 것과 container ID를 확인 한 후 docker container start *id* 로 container를 실행 합니다. 그리고 docker container ls 명령어로 실행 중인 container를 확인 합니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS    PORTS     NAMES
7e4a228d5d38   nginx     "/docker-entrypoint.…"   3 minutes ago   Created             funny_babbage
[ec2-user@ip-172-31-33-52 ~]$ docker container start 7e4a228d5d38
7e4a228d5d38
[ec2-user@ip-172-31-33-52 ~]$ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS     NAMES
7e4a228d5d38   nginx     "/docker-entrypoint.…"   4 minutes ago   Up 4 seconds   80/tcp    funny_babbage
[ec2-user@ip-172-31-33-52 ~]$
```

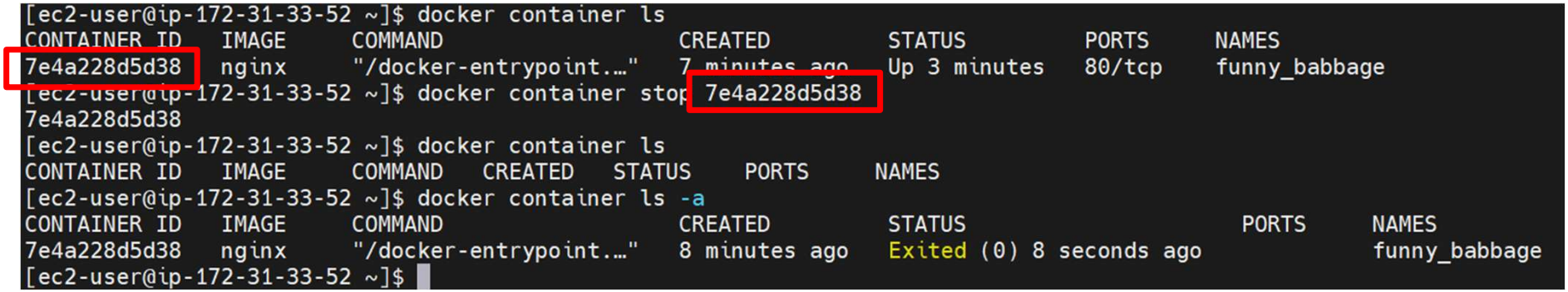
6. Container 삭제하기

1 Container stop 하기

아래와 같이 docker container stop container id를 입력해서 Container를 중지 합니다.

Docker container ls 와 docker container ls -a 명령어로 확인 해 봅니다.

.



6. Container 삭제하기

② Container 삭제 하기

아래와 같이 docker container rm 명령어를 사용하여 container를 삭제 합니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS                      PORTS     NAMES
f5ac9b941769   nginx     "/docker-entrypoint.…"   50 seconds ago   Exited (0) 11 seconds ago             eager_mendeleev
[ec2-user@ip-172-31-33-52 ~]$ docker container rm f5ac9b941769
f5ac9b941769
[ec2-user@ip-172-31-33-52 ~]$
```

6. Container 삭제하기

2 Container 삭제 하기

삭제 후 docker container ls 명령어와 docker container ls -a 명령어로 확인 합니다.

아래와 같이 container가 없는 것이 확인 됩니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker container ls
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[ec2-user@ip-172-31-33-52 ~]$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

6. Container 삭제하기

3 Image 삭제 하기

아래와 같이 docker image rm 이미지_tag 또는 docker image rm id 명령어를 사용하여 image를 삭제 합니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker image ls
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
nginx        latest    a830707172e8   4 weeks ago    192MB
[ec2-user@ip-172-31-33-52 ~]$ docker image rm nginx:latest
Untagged: nginx:latest
Untagged: nginx@sha256:c15da6c91de8d2f436196f3a768483ad32c258ed4e1beb3d367a27ed67253e66
Deleted: sha256:a830707172e8069c09cf6c67a04e23e5a1a332d70a90a54999b76273a928b9ce
Deleted: sha256:f33f068b16dcd524bfab3d9c0fe569c586b498f882b3646575f8ca9dad28183b
Deleted: sha256:2ca2fe99d37993d2cd67b9459690ca6204d4d52cb4ed785e5db36f7a0dd962bd
Deleted: sha256:9219473716627829567af49238831efb419c04a3b0fb551e8b2403b01acc32c7
Deleted: sha256:f5fb61f1d156dfaab34b082893de038561147bda56dce0665a09aa45b8f0ebd0
Deleted: sha256:da1ec5e247ce7e2343b89c04cc7378a48a108ce052887856c8f768f94d41fcbe
Deleted: sha256:04179f3ea760289be096f97bd5c92241bf7322df00e17161655e83eca3b53859
Deleted: sha256:6c4c763d22d0c5f9b2c5901dfa667fbbc4713cee6869336b8fd5022185071f1c
```

이미지 id를 사용할 수도 있음

7. Container 재 생성하기

1 docker container run

새로운 컨테이너를 생성하고 실행하는 Docker의 가장 핵심적인 명령어입니다.

docker container run [OPTIONS] IMAGE [COMMAND] [ARG...]

7. Container 재 생성하기

1 docker container run

옵션	설명
-d	Detached 모드: 백그라운드 실행
-it	Interactive + TTY: 터미널에서 직접 명령어 입력 가능
--name	컨테이너 이름 지정
-p HOST:CONTAINER	포트 매핑 (예: -p 8080:80)
-v HOST:CONTAINER	볼륨 마운트 (예: -v /host:/data)
--rm	컨테이너 종료 시 자동 삭제
--env, -e	환경 변수 설정
--network	컨테이너가 연결될 네트워크 지정

7. Container 재 생성하기

1. docker run

```
docker container run -d -p 80:80 --name web nginx
```

7. Container 재 생성하기

② Container 생성하기

docker container run -d -p 80:80 --name web nginx

```
[ec2-user@ip-172-31-33-52 ~]$ docker container run -d -p 80:80 --name web nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
254e724d7786: Pull complete
913115292750: Pull complete
3e544d53ce49: Pull complete
4f21ed9ac0c0: Pull complete
d38f2ef2d6f2: Pull complete
40a6e9f4e456: Pull complete
d3dc5ec71e9d: Pull complete
Digest: sha256:c15da6c91de8d2f436196f3a768483ad32c258ed4e1beb3d367a27ed67253e66
Status: Downloaded newer image for nginx:latest
7effff09201b44fe28d7705eafd194c553d15415b6b47eebd9279ff7ab2ee239
[ec2-user@ip-172-31-33-52 ~]$
```

7. Container 재 생성하기

3 상태 확인 하기

아래와 같이 image와 container의 상태를 확인 합니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker image ls
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    a830707172e8   4 weeks ago   192MB
[ec2-user@ip-172-31-33-52 ~]$ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED
7efffff09201b   nginx     "/docker-entrypoint.…"   About a minute ago
[ec2-user@ip-172-31-33-52 ~]$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND                  CREATED
 STATUS              PORTS                                     NAMES
7efffff09201b   nginx     "/docker-entrypoint.…"   About a minute ago
 Up About a minute   0.0.0.0:80->80/tcp, [::]:80->80/tcp   web
[ec2-user@ip-172-31-33-52 ~]$
```

7. Container 재 생성하기

④ 외부 접속 확인 하기

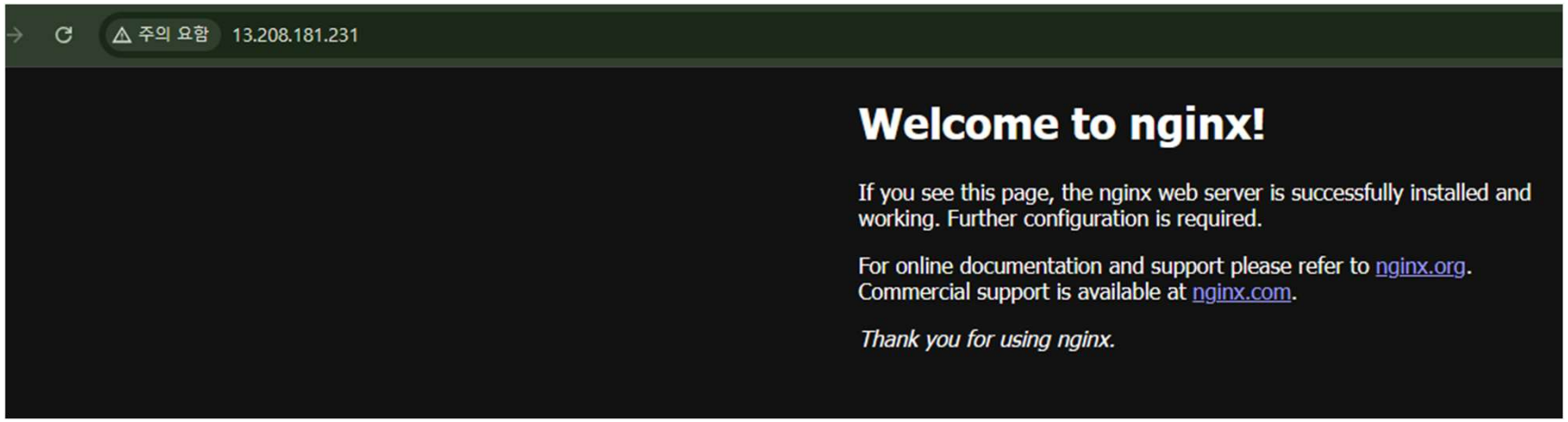
AWS Console에서 EC2 메뉴에서 실행 중인 EC2의 Public IP Address를 확인 한 후 브라우저에서 접속 해 봅니다.



7. Container 재 생성하기

4 외부 접속 확인 하기

AWS Console에서 EC2 메뉴에서 실행 중인 EC2의 Public IP Address를 확인 한 후 브라우저에서 접속 해 봅니다.



7. Container 재 생성하기

5 Container 상태 변경하기

아래와 같이 **docker container exec -it web bash** 명령어로 container에 shell을 실행 합니다.

Shell이 실행 되면 echo “my nginx V1” > /user/share/nginx/index.html 을 입력해서 web 페이지를 변경 한 후 다시 접속 해 봅니다.

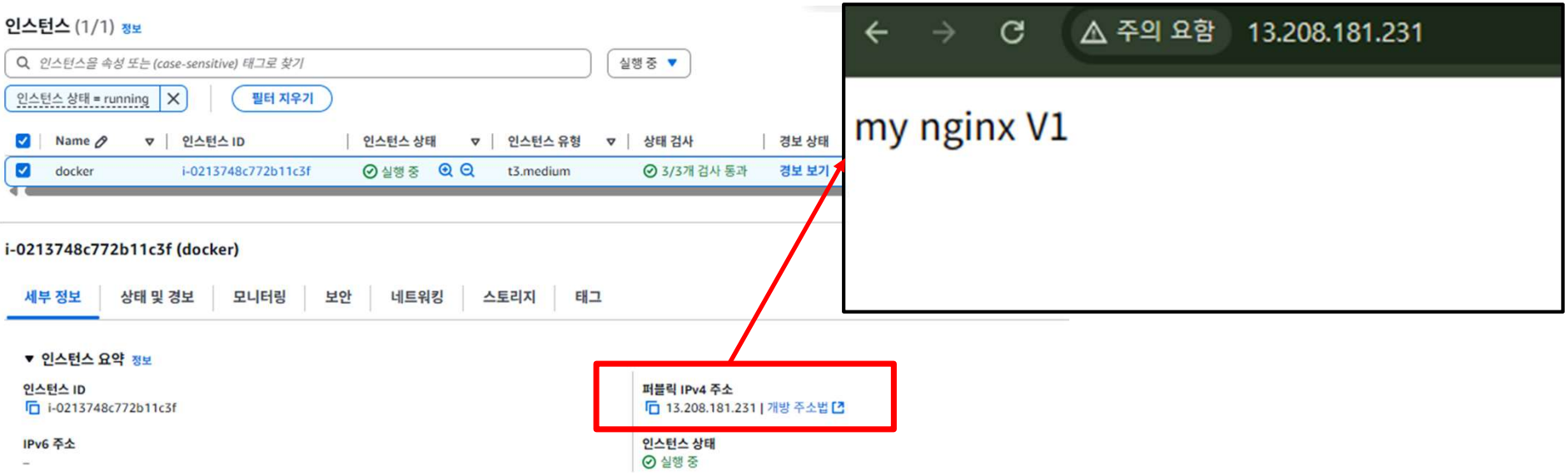
실행 후 exit로 컨테이너 shell상태에서 빠져 나옵니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker container exec -it web bash
root@7efff09201b:/# echo "my nginx V1" > /usr/share/nginx/html/index.html
root@7efff09201b:/# exit
exit
[ec2-user@ip-172-31-33-52 ~]$
```

7. Container 재 생성하기

5 Container 상태 변경하기

EC2 공인 IP를 브라우저에서 다시 접속 합니다.



7. Container 재 생성하기

6 변경된 container로 새로운 이미지 만들기

docker container commit web mynginx 명령어를 이용해서 새로운 이미지를 만듭니다.

그러면 새로운 이미지가 만들어 진 것을 확인 할 수 있습니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    a830707172e8   4 weeks ago   192MB
[ec2-user@ip-172-31-33-52 ~]$ docker container commit web mynginx
sha256:9068225f58844f75000e0bce5c780df1337c0c0c8eba7344c94b743f30368ac7
[ec2-user@ip-172-31-33-52 ~]$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
mynginx      latest    9068225f5884   2 seconds ago    192MB
nginx        latest    a830707172e8   4 weeks ago      192MB
[ec2-user@ip-172-31-33-52 ~]$
```

7. Container 재 생성하기

새로운 이미지로 container 생성하기

docker container rm -f $(docker container ls -aq) 로 생성 된 컨테이너와 실행중인 모든 컨테이너를 삭제 합니다.

```
[ec2-user@ip-172-31-33-52 ~]$ docker container rm -f $(docker container ls -aq)
7efffff09201b
[ec2-user@ip-172-31-33-52 ~]$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[ec2-user@ip-172-31-33-52 ~]$
```

7. Container 재 생성하기

새로운 이미지로 container 생성하기

아래와 같이 docker container run 명령어를 실행합니다.

docker container run -d -p 80:80 --name myweb mynginx

```
[ec2-user@ip-172-31-33-52 ~]$ docker container run -d -p 80:80 --name myweb mynginx
66b03e7ebe788a823e46532bfd21c8be4291e4e3c92f690cd40df540f44b21bb
[ec2-user@ip-172-31-33-52 ~]$ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS
                  NAMES
66b03e7ebe78   mynginx   "/docker-entrypoint.…"   3 seconds ago   Up 3 seconds   0.0.0.0:80->80/tc
p, [::]:80->80/tcp   myweb
[ec2-user@ip-172-31-33-52 ~]$
```

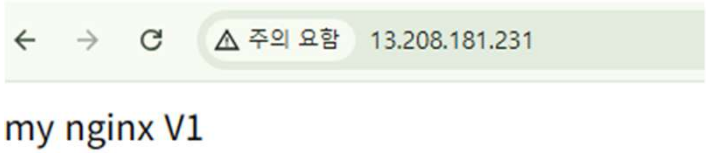
7. Container 재 생성하기

8 접속확인 하기

아래와 같이 VM의 외부 IP를 확인 한 후 웹 브라우저에서 접속을 합니다. 변경된 이미지로 만들어진 것을 확인 할 수 있습니다.

```
[ec2-user@ip-172-31-33-52 ~]$ curl ipconfig.io
13.208.181.231
```

Public IP는
터미널에서 curl ipconfig.io 명령어나
AWS 콘솔에서 확인 해도 됩니다.



7. Container 재 생성하기

9 모든 자원 삭제하기

아래 명령어로 컨테이너와 이미지를 모두 삭제 합니다.

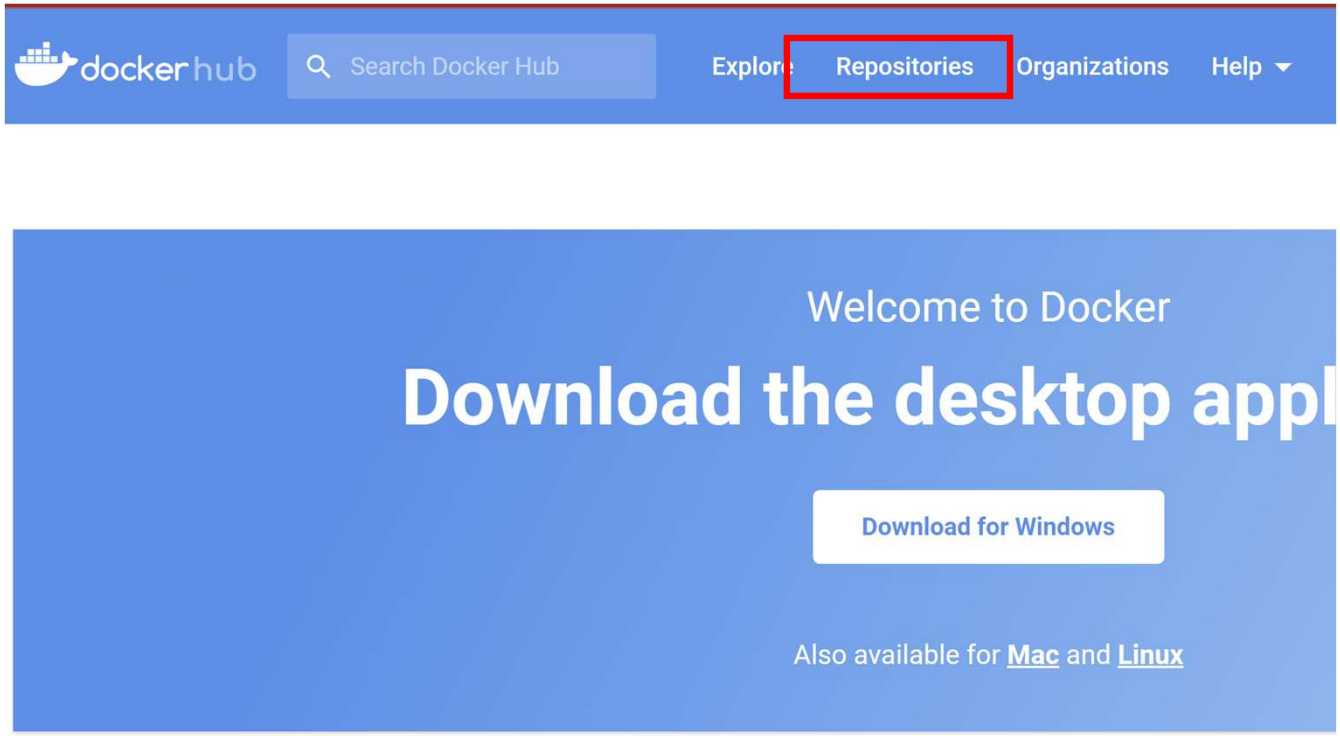
docker container rm -f $(docker container ls -aq)

docker image rm $(docker image ls -q)

8. Dockerfile 이해하기

1 Docker hub에 Repository 생성하기

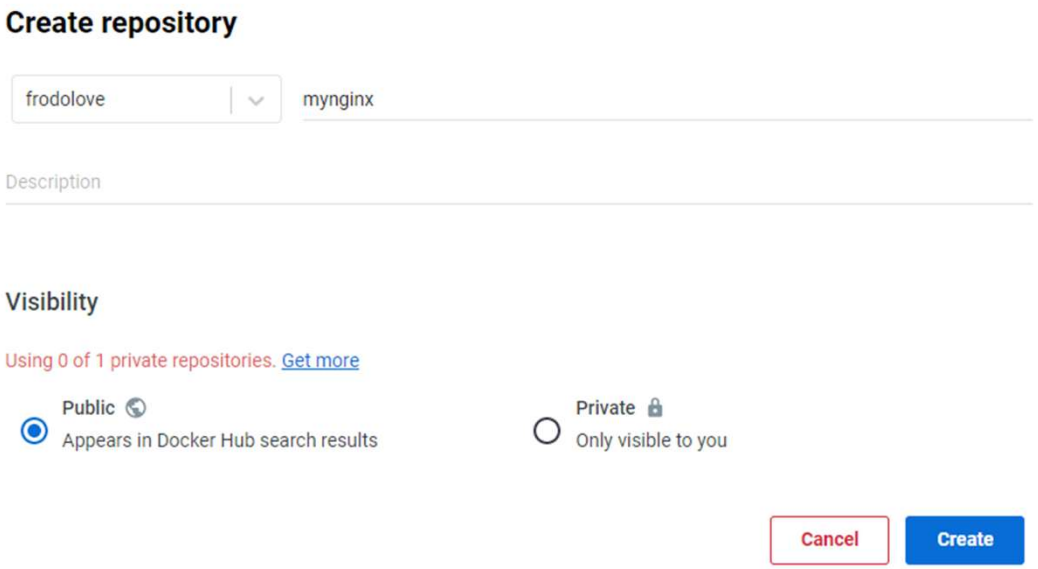
Docker hub 접속 후 메뉴에서 Repositories를 선택 합니다.



8. Dockerfile 이해하기

1 Docker hub에 Repository 생성하기

가입 한 docker hub에 mynginx라는 이름을 가진 public repository를 생성합니다.

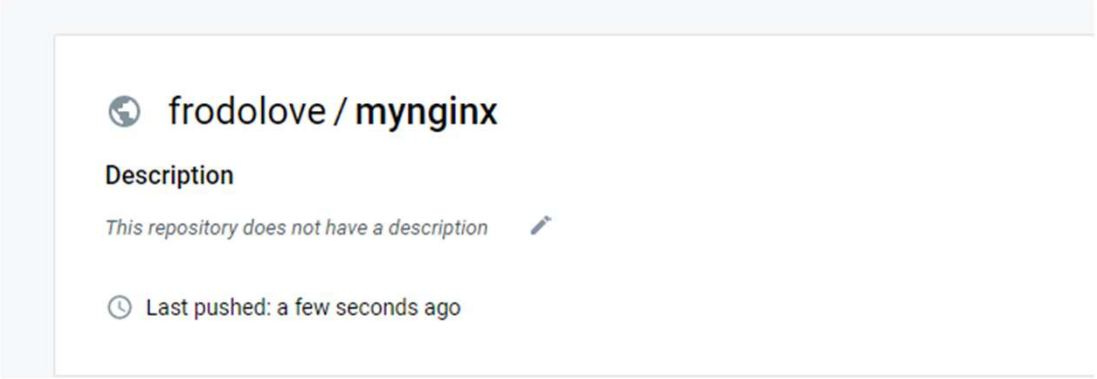


8. Dockerfile 이해하기

1 Docker hub에 Repository 생성하기

새로 생성된 repository 경로는 docker hub계정/Repository명 입니다.

아래 예에서 frodolove는 계정이고 mynginx는 repository 명이 됩니다.



8. Dockerfile 이해하기

2 nginx 소스 코드 생성하기

작업을 위한 work 경로를 만들고 해당 경로로 이동 합니다.

mkdir ~/work; cd $_

echo “This is mynginx Version1 " > ~/work/index.html 로 index.html 파일을 생성 합니다.

8. Dockerfile 이해하기

3 Dockerfile 생성하기

아래 명령어를 실행하여 /work 경로에 Dockerfile을 생성합니다.

```
cat <<EOF | sudo tee ~/work/Dockerfile
FROM nginx
COPY ./index.html /usr/share/nginx/html/index.html
EOF
```

8. Dockerfile 이해하기

④ Dockerfile로 부터 image build하기

Dockerfile과 index.html 파일이 있는 위치에서

docker image build -t dockerhub계정/mynginx(repositor명):1.0 . (반드시 **.** 으로 현재 경로인 것을 표시해야 합니다.)으로 image를 build 합니다.

(예 : **docker image build -t frodolove/mynginx:1.0 .)**

```
[ec2-user@ip-172-31-33-52 work]$ docker image build -t frodolove/mynginx:1.0 .
[+] Building 2.1s (7/7) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 160B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                                                               2.0s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 2B                                                                                               0.0s
 => [internal] load build context                                                                                             0.0s
 => => transferring context: 90B                                                                                              0.0s
 => [1/2] FROM docker.io/library/nginx:latest@sha256:c15da6c91de8d2f436196f3a768483ad32c258ed4e1beb3d367a27ed67253e6          0.0s
 => => resolve docker.io/library/nginx:latest@sha256:c15da6c91de8d2f436196f3a768483ad32c258ed4e1beb3d367a27ed67253e6          0.0s
 => CACHED [2/2] COPY ./index.html /usr/share/nginx/html/index.html                                                           0.0s
 => exporting to image                                                                                                        0.0s
 => => exporting layers                                                                                                       0.0s
 => => writing image sha256:0a435f75c9626fdbe597d35b3cda7dfd5c3ed63b9775aa96dcf09c01a3903c7d                                  0.0s
 => => naming to docker.io/frodolove/mynginx:1.0                                                                              0.0s
[ec2-user@ip-172-31-33-52 work]$
```

8. Dockerfile 이해하기

5 Docker hub에 이미지 push 하기

docker image ls 명령어로 새로운 image가 빌드 된 것을 확인 합니다.

```
[ec2-user@ip-172-31-33-52 work]$ docker image ls
REPOSITORY          TAG       IMAGE ID       CREATED              SIZE
frodolove/mynginx   1.0       0a435f75c962   About a minute ago   192MB
[ec2-user@ip-172-31-33-52 work]$
```

8. Dockerfile 이해하기

5 Docker hub에 이미지 push 하기

docker image push docker계정/mynginx:1.0 를 이용하여 docker hub에 push 합니다.

(예: docker image push frodolove/mynginx:1.0)

그러면 아래와 같이 access denied 메시지가 나옵니다

```
[ec2-user@ip-172-31-33-52 work]$ docker image push frodolove/mynginx:1.0
The push refers to repository [docker.io/frodolove/mynginx]
7ff8be69d478: Preparing
8030dd26ec5d: Preparing
d84233433437: Preparing
f8455d4eb3ff: Preparing
286733b13b0f: Preparing
46a24b5c31d8: Waiting
84accda66bf0: Waiting
6c4c763d22d0: Waiting
denied: requested access to the resource is denied
[ec2-user@ip-172-31-33-52 work]$
```

8. Dockerfile 이해하기

5 Docker hub에 이미지 push 하기

docker login –u 도커허브_ID 를 통해서 docker hub에 login 합니다.

Login Succeeded가 나오면 앞에 과정을 다시 시도 합니다.

```
[ec2-user@ip-172-31-33-52 work]$ docker login -u frodolove

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

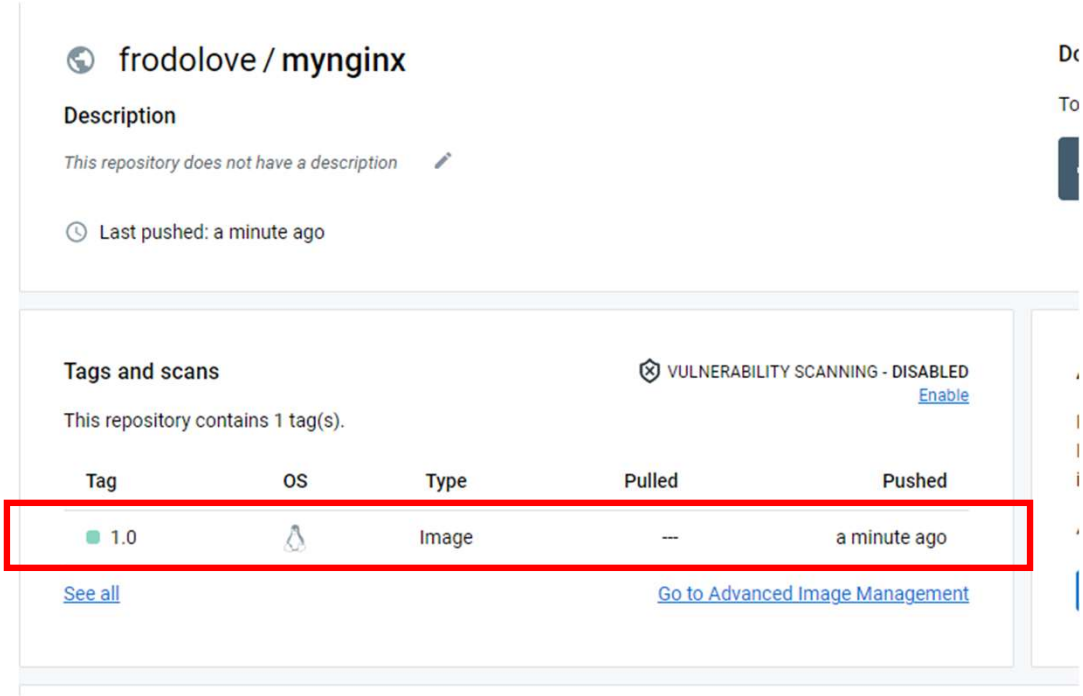
WARNING! Your credentials are stored unencrypted in '/home/ec2-user/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
[ec2-user@ip-172-31-33-52 work]$ docker image push frodolove/mynginx:1.0
The push refers to repository [docker.io/frodolove/mynginx]
7ff8be69d478: Pushed
8030dd26ec5d: Mounted from library/nginx
d84233433437: Mounted from library/nginx
f8455d4eb3ff: Mounted from library/nginx
286733b13b0f: Mounted from library/nginx
46a24b5c31d8: Mounted from library/nginx
84accda66bf0: Mounted from library/nginx
6c4c763d22d0: Mounted from library/nginx
1.0: digest: sha256:d079e4ee69cc6cd5361675266fb27a2a16a9dfd31f2e8fd1f5618979ad0f5d8f size: 1985
[ec2-user@ip-172-31-33-52 work]$
```

8. Dockerfile 이해하기

5 Docker hub에 이미지 push 하기

Docker hub에 접속해서 mynginx repository에 push된 image가 있는 지 확인 합니다.



8. Dockerfile 이해하기

6 Local에 있는 이미지 삭제 하기

docker image rm $(docker image ls -q) 로 local에 있는 모든 image를 삭제 합니다.

```
[ec2-user@ip-172-31-33-52 work]$ docker image rm $(docker image ls -q)
Untagged: frodolove/mynginx:1.0
Untagged: frodolove/mynginx@sha256:d079e4ee69cc6cd5361675266fb27a2a16a9dfd31f2e8fd1f5618979ad0f5d8f
Deleted: sha256:0a435f75c9626fdbe597d35b3cda7dfd5c3ed63b9775aa96dcf09c01a3903c7d
[ec2-user@ip-172-31-33-52 work]$ docker image ls
REPOSITORY   TAG       IMAGE ID   CREATED   SIZE
[ec2-user@ip-172-31-33-52 work]$
```

8. Dockerfile 이해하기

7 Docker hub에 push 한 이미지로 container 생성하기

docker container run -d -p 80:80 --name myweb 도커계정/mynginx:1.0

(예) docker container run -d -p 80:80 --name myweb frodolove/mynginx:1.0

```
[ec2-user@ip-172-31-33-52 work]$ docker container run -d -p 80:80 --name myweb frodolove/mynginx:1.0
Unable to find image 'frodolove/mynginx:1.0' locally
1.0: Pulling from frodolove/mynginx
254e724d7786: Already exists
913115292750: Already exists
3e544d53ce49: Already exists
4f21ed9ac0c0: Already exists
d38f2ef2d6f2: Already exists
40a6e9f4e456: Already exists
d3dc5ec71e9d: Already exists
c57144ff2396: Already exists
Digest: sha256:d079e4ee69cc6cd5361675266fb27a2a16a9dfd31f2e8fd1f5618979ad0f5d8f
Status: Downloaded newer image for frodolove/mynginx:1.0
471f1b786f73985cfcb627028b9aa0d4a44649a2f1c2a5103d5ba7e8844e15c7
[ec2-user@ip-172-31-33-52 work]$ docker image ls
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
frodolove/mynginx   1.0       0a435f75c962   17 minutes ago   192MB
[ec2-user@ip-172-31-33-52 work]$ docker container ls
CONTAINER ID   IMAGE                   COMMAND                  CREATED          STATUS          PORTS
          NAMES
471f1b786f73   frodolove/mynginx:1.0   "/docker-entrypoint.…"   16 seconds ago   Up 15 seconds   0.0.0.0:80->80/tcp, [::]:8
0->80/tcp   myweb
[ec2-user@ip-172-31-33-52 work]$
```

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

도커 이미지의 인프라 구성을 기술한 파일

Dockerfile reference : https://docs.docker.com/engine/reference/builder/

docker image build로 Dockerfile에 기술된 내용을 기반으로 이미지 생성

예시

```
FROM python:3.9-slim
WORKDIR /app
COPY print_args.py .
ENTRYPOINT ["python", "print_args.py"]
CMD ["Hello", "from", "CMD!"]
```

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

기본 서식은 명령 인수 형식으로 기재 합니다.

명령은 대/소문자 상관없지만 일반적으로 대문자를 사용합니다.

이미지 레이어 수와 이미지 사이즈를 최소화 합니다.

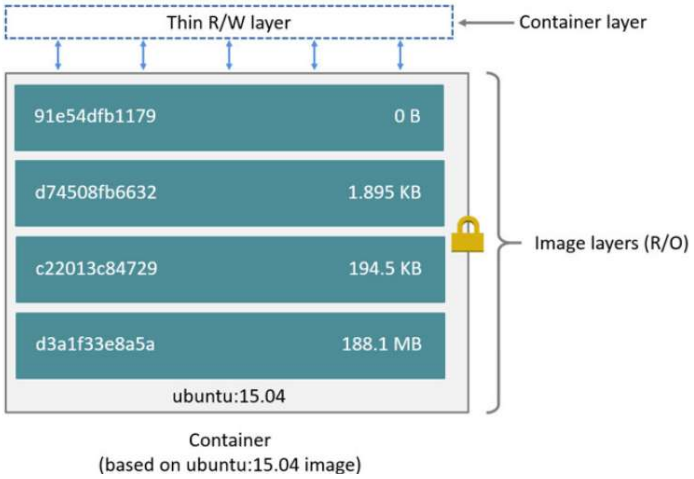
8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

기본 서식은 명령 인수 형식으로 기재 합니다.

명령은 대/소문자 상관없지만 일반적으로 대문자를 사용합니다.

이미지 레이어 수와 이미지 사이즈를 최소화 합니다.



8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

FROM

베이스 이미지를 명시하는 필수 항목

```
mkdir -p ~/work/build/from && cd $_
cat <<EOF > Dockerfile
FROM openjdk:8
EOF

docker image build -t from .
```

```
docker image ls
docker container run -it from
docker container ls
```

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

RUN

1. Shell From

```
RUN apt-get update && apt-get install -y curl
```

2. Exec From

```
RUN ["apt-get", "install", "-y", "curl"]
```

항목	Shell Form	Exec Form
문법	쉘 명령어 한 줄 (/bin/sh -c)	명령 배열 (JSON 배열)
변수 확장	가능 ($HOME, $PATH)	제한적 (쉘이 없으므로 직접 해석 안 됨)
변수 확장	가능 ($HOME, $PATH)	제한적 (쉘이 없으므로 직접 해석 안 됨)

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

RUN

```
mkdir -p ~/work/build/run && cd $_
cat <<EOF > Dockerfile
FROM openjdk:8
# exec form
RUN ["/usr/bin/apt-get", "update"]
# shell from
RUN apt-get install -y net-tools; ₩ apt-get clean; ₩
rm -rf /var/lib/apt/lists/*
EOF

docker image build -t run .
```

```
docker image ls
docker container run -it from
docker container ls
```

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

RUN

```
mkdir -p ~/work/build/run && cd $_
cat <<EOF > Dockerfile
FROM openjdk:8
# exec form
RUN ["/usr/bin/apt-get", "update"]
# shell from
RUN apt-get install -y net-tools; ₩ apt-get clean; ₩
rm -rf /var/lib/apt/lists/*
EOF

docker image build -t run .
```

```
docker image ls
docker image history run
docker container run -it from
docker container ls
```

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

COPY

파일 또는 디렉토리를 복사하는 명령

ADD

COPY 명령과 기본적으로 동일한 기능 수행

네트워크 (URL)를 통해 파일 복사 가능

압축 자동 해제

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

ENV

환경변수를 설정하는 명령

ENV 명령으로 지정한 환경변수는 컨테이너 실행시에 --env 옵션으로 변경 가능

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

CMD

환경변수를 설정하는 명령

ENV 명령으로 지정한 환경변수는 컨테이너 실행시에 --env 옵션으로 변경 가능

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

CMD

> Exec Type

쉘을 거치지 않고 프로세스를 직접 실행 (exec 형태)

CMD ["executable", "param1", "param2"]

예) CMD ["nginx", "-g", "daemon off;"]

> Shell Type

/bin/sh -c를 통해 실행됨

CMD ["nginx", "-g", "daemon off;"]

CMD java -jar app.jar > output.log

> CMD with ENTRYPOINT

CMD는 ENTRYPOINT와 **함께 쓰일 때 인자(argument)**로 동작할 수 있음

ENTRYPOINT ["java", "-jar"]

CMD ["app.jar"]

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

ENTRYPOINT

컨테이너 시작 시 자동으로 실행되는 명령을 지정할 때 사용

CMD 명령과 함께 사용되면 CMD 명령의 값이 ENTRYPOINT 명령의 파라미터로 전달 됨

CMD 명령과 다르게 컨테이너 실행 시 덮어쓰기 불가, 인자로 추가 됨

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

ENTRYPOINT

> Exec Type

쉘을 거치지 않고 명령을 직접 실행 (pid 1로 시작됨)

ENTRYPOINT ["executable", "param1", "param2"]

예) ENTRYPOINT ["nginx", "-g", "daemon off;"]

> Shell Type

/bin/sh -c를 통해 실행됨

ENTRYPOINT command param1 param2

ENTRYPOINT echo Hello World

> CMD with ENTRYPOINT

CMD는 ENTRYPOINT와 **함께 쓰일 때 인자(argument)**로 동작할 수 있음

ENTRYPOINT ["java", "-jar"]

CMD ["app.jar"]

8. Dockerfile 이해하기

9 Dockerfile에 대한 이해

CMD vs ENTRYPOINT

항목	CMD	ENTRYPOINT
목적	**기본 실행 명령 또는 인자 제공**	**항상 실행할 명령 지정**
덮어쓰기	docker run의 인자로 **덮어씀**	덮어쓰기 **불가**, 인자로 **추가됨**
위치	Dockerfile에 한 번만 사용 가능	CMD와 함께 쓸 수 있음
조합 사용	CMD 단독 또는 ENTRYPOINT와 조합 사용 가능	주로 ENTRYPOINT + CMD 조합 사용

9. Dockerfile 실습하기

1 실습1

```
mkdir -p ~/work/build/docker1 && cd $_


cat <<EOF > Dockerfile
FROM openjdk:11
ADD https://docker-k8s-edu.s3.ap-northeast-2.amazonaws.com/java/frodo.jar frodo.jar
ENTRYPOINT ["java", "-jar"]
CMD ["/frodo.jar"]
EOF


docker image build -t sysinfo:1.0 .
docker container run -d -p 80:8080 --name sysinfo sysinfo:1.0
```

```
FROM openjdk:11

ADD https://docker-k8s-edu.s3.ap-northeast-2.amazonaws.com/java/frodo.jar frodo.jar

ENTRYPOINT ["java", "-jar"]
CMD ["/frodo.jar"]
```

ENTRYPOINT ["java", "-jar", "/frodo.jar"]

또는

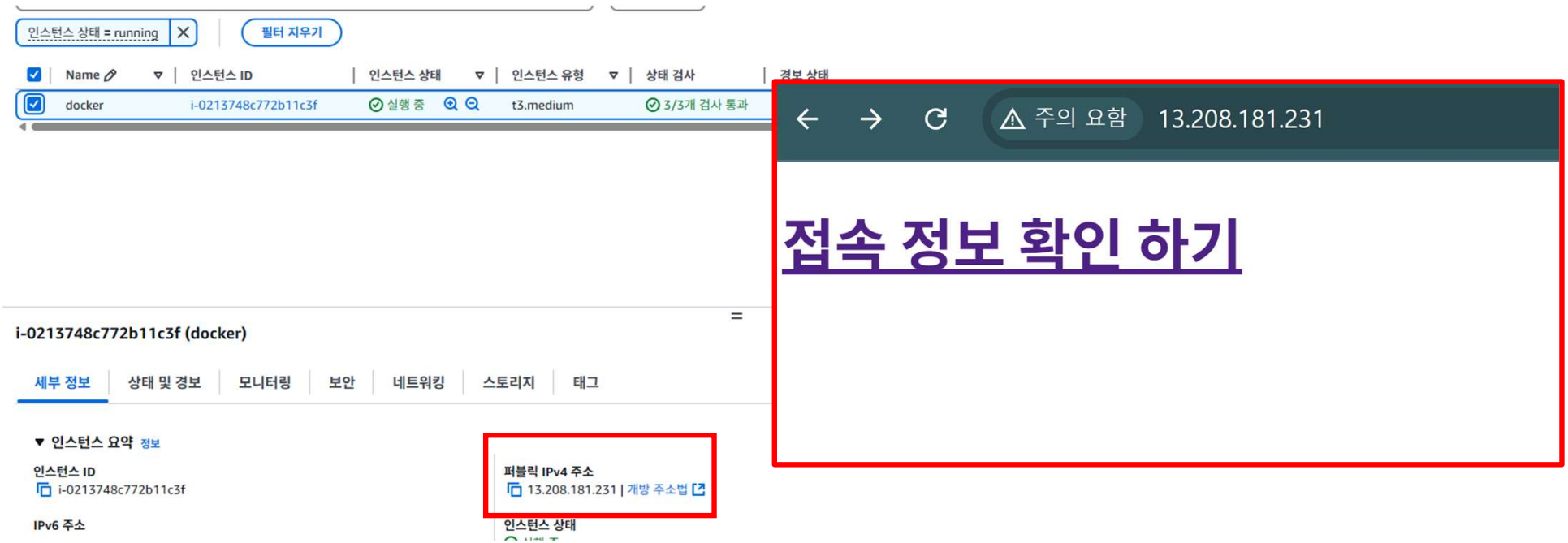
CMD ["java", "-jar", "/frodo.jar"]

로 변경 가능

9. Dockerfile 실습하기

실습1

EC2 Public IP를 이용하여 브라우저에서 접속해 봅니다.



9. Dockerfile 실습하기

1️⃣ 실습1

다음 실습을 위해서 모든 컨테이너와 이미지를 삭제 합니다.

docker container rm -f $(docker container ls -aq) ➔ 만들어진 모든 컨테이너 삭제

docker image rm -f $(docker image ls -q) ➔ 모든 local 이미지 삭제

9. Dockerfile 실습하기

실습1

```
mkdir -p ~/work/build/docker2 && cd $_

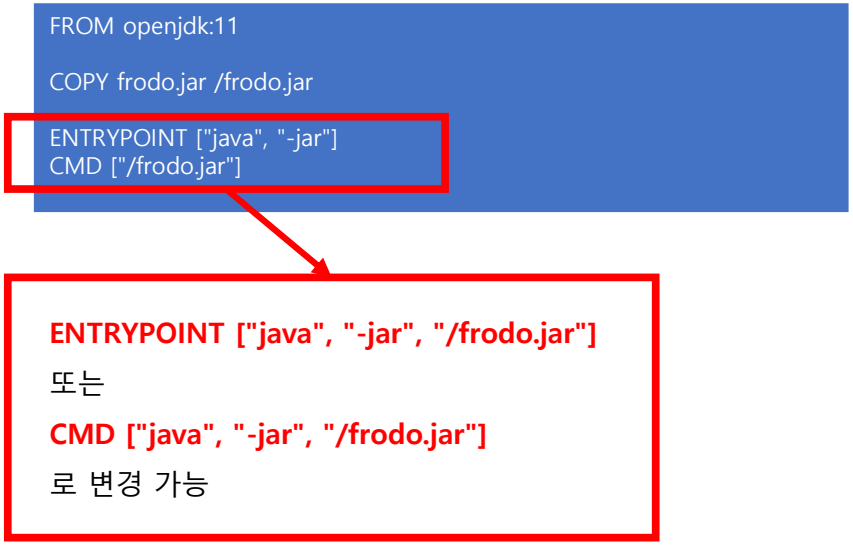
curl -o frodo.jar https://docker-k8s-edu.s3.ap-northeast-2.amazonaws.com/java/frodo.jar

ls ➔ 파일이 다운 받아 졌는지 확인 합니다.


cat <<EOF > Dockerfile
FROM openjdk:11
COPY frodo.jar /frodo.jar
ENTRYPOINT ["java", "-jar"]
CMD ["/frodo.jar"]
EOF


docker image build -t sysinfo:2.0 .

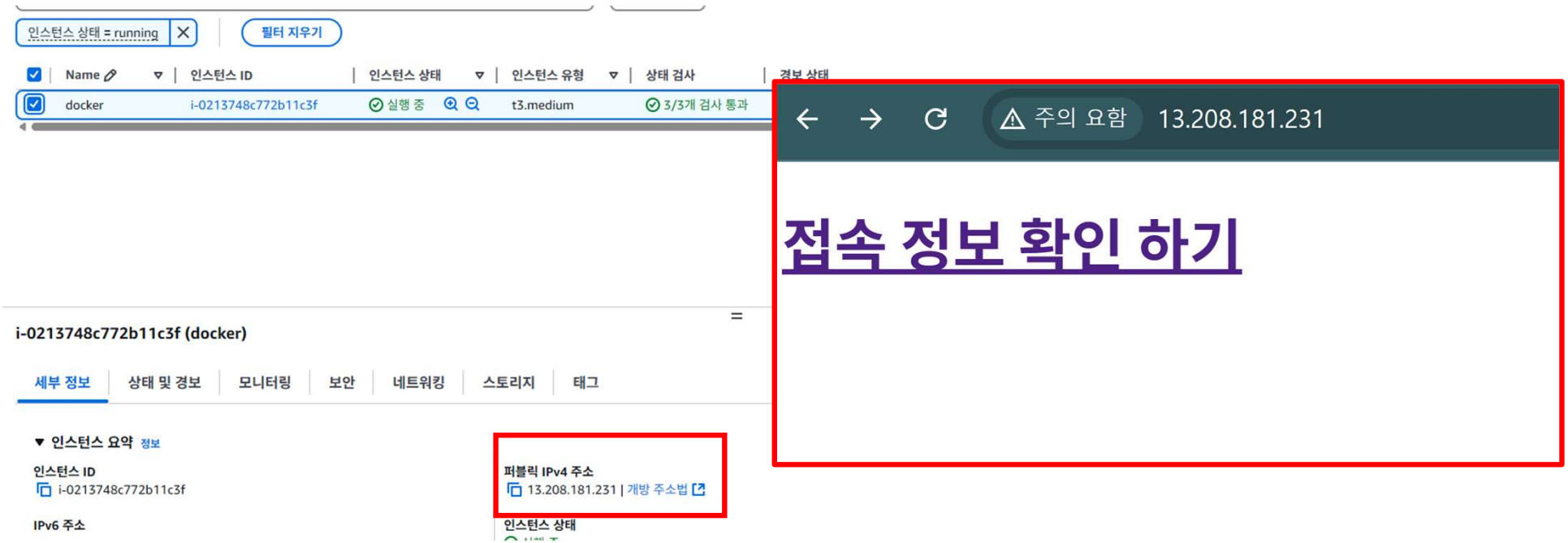
docker container run -d -p 80:8080 --name sysinfo sysinfo:2.0
```



9. Dockerfile 실습하기

실습2

EC2 Public IP를 이용하여 브라우저에서 접속해 봅니다.



9. Dockerfile 실습하기

2 실습2

다음 실습을 위해서 모든 컨테이너와 이미지를 삭제 합니다.

docker container rm -f $(docker container ls -aq) ➔ 만들어진 모든 컨테이너 삭제

docker image rm -f $(docker image ls -q) ➔ 모든 local 이미지 삭제

9. Dockerfile 실습하기

3 실습3

```
mkdir -p ~/work/build/docker3 && cd $_
echo "Web Test!!" > index.html

cat <<EOF > Dockerfile
FROM rockylinux:9
# Nginx 설치
RUN dnf install -y epel-release && ₩
    dnf install -y nginx && ₩
    dnf install -y iputils  && ₩
    dnf install -y net-tools && ₩
    dnf clean all

# 환경변수
ENV APP_NAME="frodo"
ENV APP_PORT=80
# index.html 복사
COPY index.html /usr/share/nginx/html/index.html
# 포트 오픈
EXPOSE 80
# Nginx 실행
CMD ["nginx", "-g", "daemon off;"]
EOF

docker image build -t 도커허브계정/rockynginx:1.0 .
docker image push 도커허브계정/rockynginx:1.0
docker container run -d -p 80:80 --name rocky-nginx 도커허브계정/rockynginx:1.0
```

```
FROM rockylinux:9

# Nginx 설치
RUN dnf install -y epel-release && ₩
    dnf install -y nginx && ₩
    dnf install -y iputils  && ₩
    dnf install -y net-tools && ₩
    dnf clean all

# 환경변수
ENV APP_NAME="frodo"
ENV APP_PORT=80

# index.html 복사
COPY index.html /usr/share/nginx/html/index.html

# 포트 오픈
EXPOSE 80

# Nginx 실행
CMD ["nginx", "-g", "daemon off;"]
```

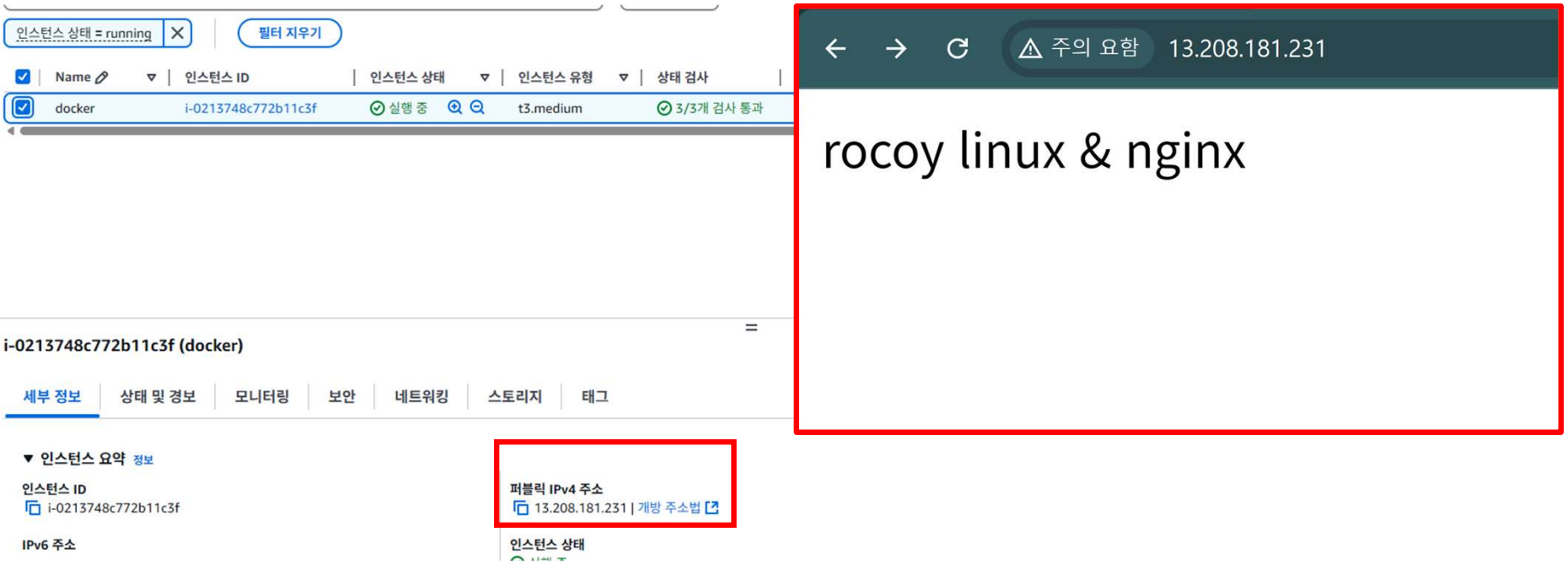
해당 이미지는 추후 사용할 수 있도록 Docker hub에 push 합니다.

```
docker image build -t frodolove/rockynginx:1.0 .
docker image push frodolove/rockynginx:1.0
docker container run -d -p 80:80 --name rocky-nginx frodolove/rockynginx:1.0
```

9. Dockerfile 실습하기

실습2

EC2 Public IP를 이용하여 브라우저에서 접속해 봅니다.



9. Dockerfile 실습하기

실습3

생성된 컨테이너에서 환경 변수를 확인 합니다.

docker container ls

docker container exec rocky-nginx env

```
[ec2-user@ip-172-31-33-52 docker3]$ docker container ls
CONTAINER ID   IMAGE                     COMMAND                  CREATED         STATUS         PORTS                                     NAMES
39c2514d453f   frodolove/rockynginx:1.0   "nginx -g 'daemon of…"   4 minutes ago   Up 4 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp   rocky-nginx
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec rocky-nginx env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=39c2514d453f
APP_NAME=frodo
APP_PORT=80
HOME=/root
[ec2-user@ip-172-31-33-52 docker3]$
```

9. Dockerfile 실습하기

3 실습3

생성된 이미지의 history를 확인 합니다.

docker image history 도커허브_계정/rockynginx:1.0

예) docker image history frodolove/rockynginx:1.0

```
[ec2-user@ip-172-31-33-52 docker3]$ docker image ls
REPOSITORY            TAG       IMAGE ID       CREATED         SIZE
frodolove/rockynginx  1.0       ced92264781a   4 minutes ago   208MB
[ec2-user@ip-172-31-33-52 docker3]$ docker image history frodolove/rockynginx:1.0
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
ced92264781a   5 minutes ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   COPY index.html /usr/share/nginx/html/index.…   20B       buildkit.dockerfile.v0
<missing>      5 minutes ago   ENV APP_PORT=80                                 0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   ENV APP_NAME=frodo                              0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   RUN /bin/sh -c dnf install -y epel-release &…   32.6MB    buildkit.dockerfile.v0
<missing>      18 months ago   CMD ["/bin/bash"]                               0B        buildkit.dockerfile.v0
<missing>      18 months ago   ADD layer.tar.xz / # buildkit                   176MB     buildkit.dockerfile.v0
```

9. Dockerfile 실습하기

3 실습3

다음 실습을 위해서 모든 컨테이너와 이미지를 삭제 합니다.

docker container rm -f $(docker container ls -aq) ➔ 만들어진 모든 컨테이너 삭제

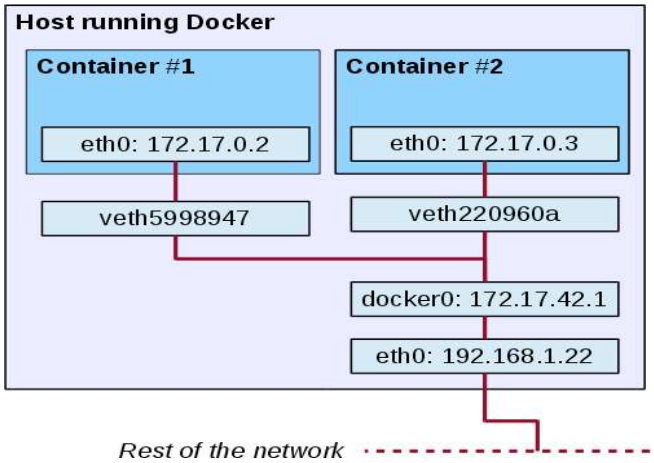
docker image rm -f $(docker image ls -q) ➔ 모든 local 이미지 삭제

10. Docker Network

1 Docker Network 개요

veth: virtual eth

docker0: Docker 엔진에 의해 기본 생성되는 Bridge 네트워크로 veth와 eth 간 다리 역할



10. Docker Network

2 Docker Network 조회

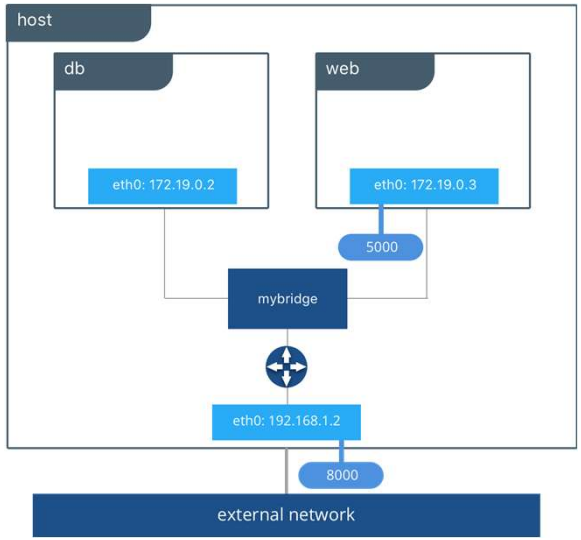
docker network ls 커맨드를 사용하면 현재 생성되어 있는 Docker 네트워크 목록을 조회할 수 있습니다.

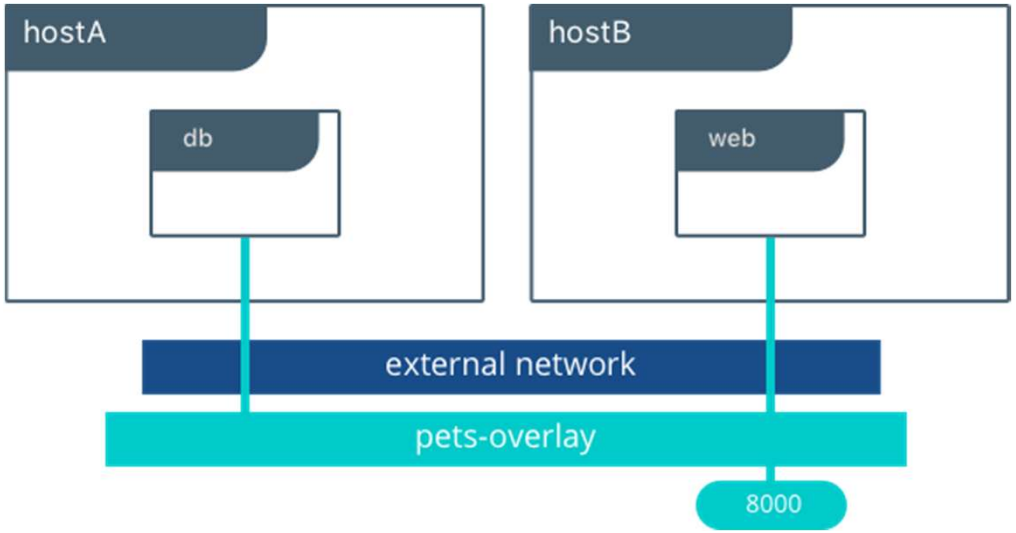
```
[root@console work]# docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
276d178b67b9   bridge    bridge    local
a2a96eb38ba7   host      host      local
76aad383f00a   none      null      local
[root@console work]#
```

10. Docker Network

3 Docker Network 종류

Docker 네트워크는 bridge, host, overlay 등 목적에 따라 다양한 종류의 네트워크 드라이버(driver)를 지원 합니다.





10. Docker Network

4 사용자 Network 만들기

docker network create --driver bridge mynet

docker network ls

```
[ec2-user@ip-172-31-33-52 docker3]$ docker network create --driver bridge mynet
docker network ls
5f034c661db3ce634579f19951bfa7e608f953c8d72deae946473fc8fdebcb54
NETWORK ID     NAME      DRIVER    SCOPE
8fa004e460e5   bridge    bridge    local
5fdbf92091ca   host      host      local
5f034c661db3   mynet     bridge    local
a725b1dcfed6   none      null      local
[ec2-user@ip-172-31-33-52 docker3]$
```

10. Docker Network

5 컨테이너 간 통신 테스트

실습을 위해서 다음과 같이 컨테이너를 만듭니다.

docker run -d --name web1 도커허브_계정/rockynginx:1.0

docker run -d --name web2 도커허브_계정/rockynginx:1.0

예)

docker run -d --name web1 frodolove/rockynginx:1.0

docker run -d --name web2 frodolove/rockynginx:1.0

```
[ec2-user@ip-172-31-33-52 docker3]$ docker run -d --name web1 frodolove/rockynginx:1.0
docker run -d --name web2 frodolove/rockynginx:1.0
Unable to find image 'frodolove/rockynginx:1.0' locally
1.0: Pulling from frodolove/rockynginx
446f83f14b23: Already exists
190e2745cfe1: Already exists
5241dcf4d52e: Already exists
Digest: sha256:a94119df8a99c7e79ef35fa8afbb46fef86869db6433cc3b310f056ceb95e5ef
Status: Downloaded newer image for frodolove/rockynginx:1.0
8a06662e699379f53c1f626869d5ede242529e60a1f8332e3b028615331229d2
b4397bade8054ee7839b730e0ee8b470bf0a51bb8ad649c9bb75ec3fbc0c4d0b
[ec2-user@ip-172-31-33-52 docker3]$ docker container ls
CONTAINER ID   IMAGE                       COMMAND                  CREATED          STATUS          PORTS     NAMES
b4397bade805   frodolove/rockynginx:1.0   "nginx -g 'daemon of…"   10 seconds ago   Up 9 seconds    80/tcp    web2
8a06662e6993   frodolove/rockynginx:1.0   "nginx -g 'daemon of…"   11 seconds ago   Up 10 seconds   80/tcp    web1
[ec2-user@ip-172-31-33-52 docker3]$
```

10. Docker Network

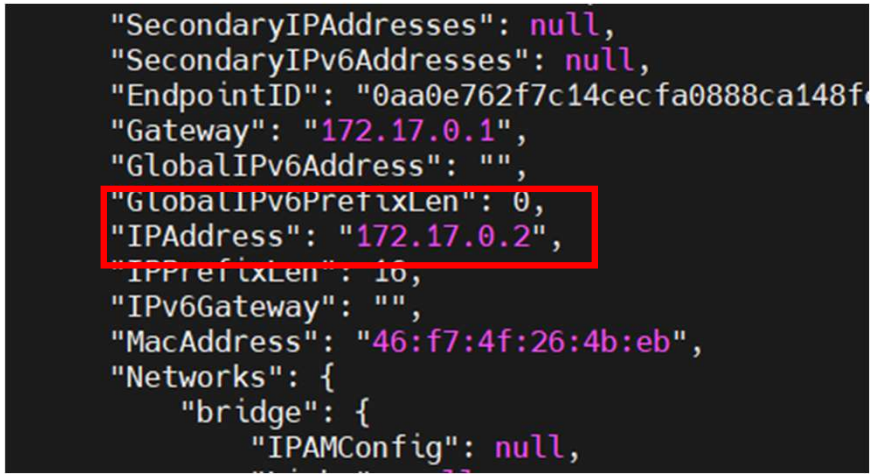
5 컨테이너 간 통신 테스트

docker container inspect 컨테이너이름 명령어로 컨테이너 IP를 확인합니다.

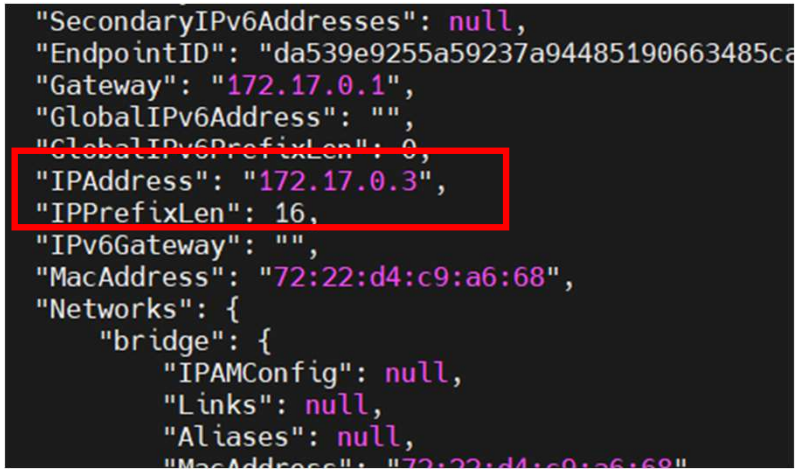
docker container inspect web1

docker container inspect web2

Web 1



Web 2



10. Docker Network

5 컨테이너 간 통신 테스트

web1에서 web2로 ping 테스트를, web2에서 web1으로 ping 테스트를 합니다.

docker container exec web1 ping 172.17.0.3 → web2 IP

docker container exec web2 ping 172.17.0.2 → web1 IP

Ping 출력이 보이면 ctrl + c로 출력을 중단 합니다.

```
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web1 ping 172.17.0.3
PING 172.17.0.3 (172.17.0.3) 56(84) bytes of data.
64 bytes from 172.17.0.3: icmp_seq=1 ttl=64 time=0.033 ms
64 bytes from 172.17.0.3: icmp_seq=2 ttl=64 time=0.057 ms
64 bytes from 172.17.0.3: icmp_seq=3 ttl=64 time=0.576 ms
64 bytes from 172.17.0.3: icmp_seq=4 ttl=64 time=0.061 ms
64 bytes from 172.17.0.3: icmp_seq=5 ttl=64 time=0.062 ms
64 bytes from 172.17.0.3: icmp_seq=6 ttl=64 time=0.060 ms
64 bytes from 172.17.0.3: icmp_seq=7 ttl=64 time=0.058 ms
```

```
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web2 ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.037 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=64 time=0.059 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=64 time=0.062 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=64 time=0.061 ms
64 bytes from 172.17.0.2: icmp_seq=5 ttl=64 time=0.068 ms
```

10. Docker Network

5 컨테이너 간 통신 테스트

정리

docker 실행 시 별도의 옵션이 없으면 기본 Bridge에 컨테이너가 생성 되고 같은 Bridge Network에서의 컨테이너 간에는 통신이 가능합니다.

10. Docker Network

5 컨테이너 간 통신 테스트

실습을 위해서 3번째 4번째 컨테이너를 앞에서 만든 mynet이라는 Bridge Network에 생성합니다.

docker run -d --name web3 --network mynet 도커허브_계정/rockynginx:1.0

docker run -d --name web4 --network mynet 도커허브_계정 /rockynginx:1.0

예)

docker run -d --name web3 --network mynet frodolove/rockynginx:1.0

docker run -d --name web4 --network mynet frodolove/rockynginx:1.0

10. Docker Network

5 컨테이너 간 통신 테스트

docker container inspect 컨테이너이름 명령어로 컨테이너 IP를 확인합니다.

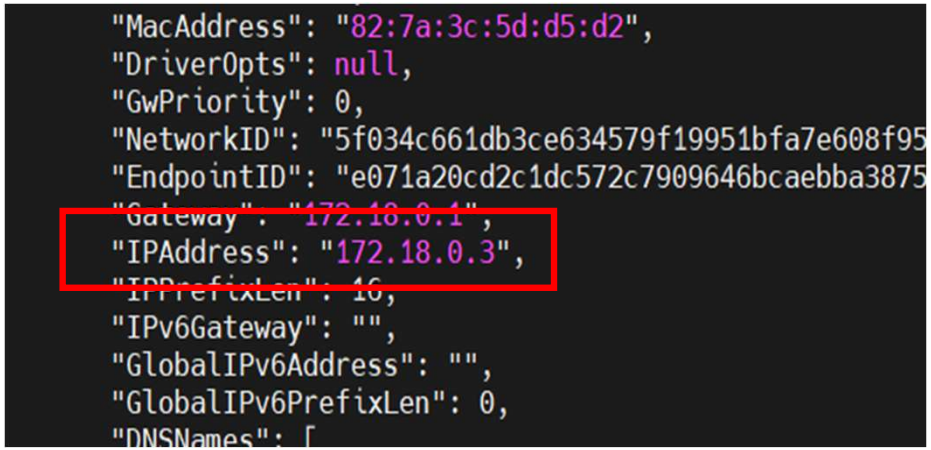
docker container inspect web3

docker container inspect web4

Web 3



Web 4



10. Docker Network

5 컨테이너 간 통신 테스트

web1에서 web3과 web4 ping 테스트를 합니다.

docker container exec web1 ping 172.18.0.2 → web3 IP

docker container exec web1 ping 172.18.0.3 → web3 IP

Ping 출력이 보이면 ctrl + c로 출력을 중단 합니다.

```
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web1 ping 172.18.0.2
^Ccontext canceled
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web1 ping 172.18.0.3
^Ccontext canceled
[ec2-user@ip-172-31-33-52 docker3]$
```

10. Docker Network

5 컨테이너 간 통신 테스트

web3에서 web4로 ping 테스트를, web4에서 web3으로 ping 테스트를 합니다.

docker container exec web3 ping 172.18.0.3 → web3 IP

docker container exec web4 ping 172.18.0.2 → web3 IP

Ping 출력이 보이면 ctrl + c로 출력을 중단 합니다.

```
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web3 ping 172.18.0.3
PING 172.18.0.3 (172.18.0.3) 56(84) bytes of data.
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.068 ms
64 bytes from 172.18.0.3: icmp_seq=2 ttl=64 time=0.059 ms
64 bytes from 172.18.0.3: icmp_seq=3 ttl=64 time=0.050 ms
^Ccontext canceled
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web4 ping 172.18.0.2
PING 172.18.0.2 (172.18.0.2) 56(84) bytes of data.
64 bytes from 172.18.0.2: icmp_seq=1 ttl=64 time=0.053 ms
64 bytes from 172.18.0.2: icmp_seq=2 ttl=64 time=0.067 ms
64 bytes from 172.18.0.2: icmp_seq=3 ttl=64 time=0.071 ms
64 bytes from 172.18.0.2: icmp_seq=4 ttl=64 time=0.069 ms
```

10. Docker Network

5 컨테이너 간 통신 테스트

정리2

컨테이너 생성 시 Network를 정의 할 수 있습니다.

같은 Bridge Network에 있는 컨테이너 간 통신은 가능하지만

다른 Bridge Network에 있는 컨테이너 간 통신은 기본적으로 불가능 합니다.

10. Docker Network

5 컨테이너 간 통신 테스트 추가 실습

web1을 mynet에 연결 하여 web3과 web3와 통신 할 수 있게 합니다.

docker network connect myweb web1

docker container exec web1 ping 172.18.0.2 → web3의 IP

docker container exec web1 ping 172.18.0.3 → web4의 IP

```
[ec2-user@ip-172-31-33-52 docker3]$ docker network connect mynet web1
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web1 ping 172.18.0.2
PING 172.18.0.2 (172.18.0.2) 56(84) bytes of data.
64 bytes from 172.18.0.2: icmp_seq=1 ttl=64 time=0.044 ms
64 bytes from 172.18.0.2: icmp_seq=2 ttl=64 time=0.068 ms
^Ccontext canceled
[ec2-user@ip-172-31-33-52 docker3]$ ^C
[ec2-user@ip-172-31-33-52 docker3]$ docker container exec web1 ping 172.18.0.3
PING 172.18.0.3 (172.18.0.3) 56(84) bytes of data.
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.053 ms
64 bytes from 172.18.0.3: icmp_seq=2 ttl=64 time=0.075 ms
```

10. Docker Network

5 컨테이너 간 통신 테스트 추가 실습

다음 실습을 위해서 모든 컨테이너와 이미지를 삭제 합니다.

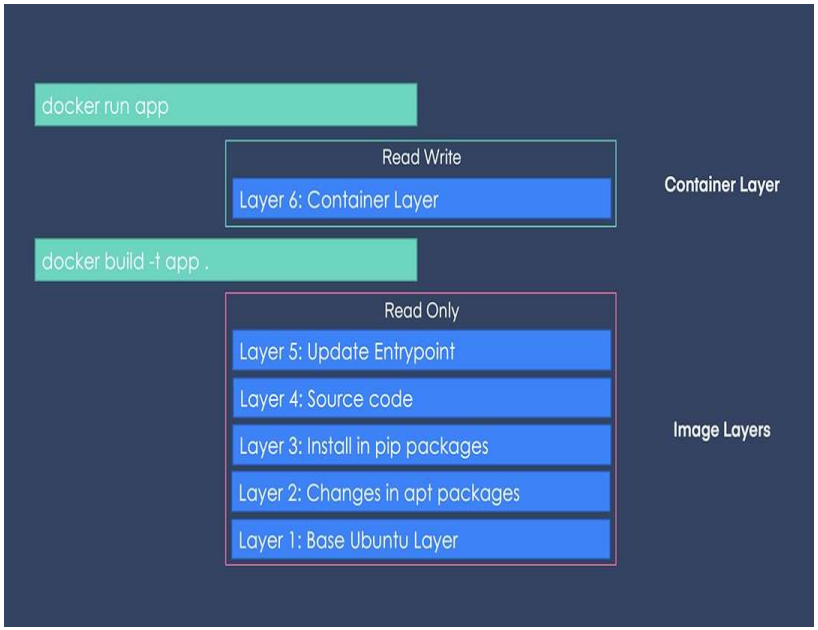
docker container rm -f $(docker container ls -aq) ➔ 만들어진 모든 컨테이너 삭제

docker image rm -f $(docker image ls -q) ➔ 모든 local 이미지 삭제

11. Docker Volume

1 개요

Docker는 컨테이너 내부의 파일 변경 사항을 **UnionFS(Union File System)**를 통해 관리합니다.
UnionFS는 이미지 레이어와 쓰기 레이어(write layer)를 결합해 컨테이너의 파일 시스템을 구성합니다.
컨테이너가 삭제되면 이와 함께 **쓰기 레이어도 삭제**되므로, 쓰기 레이어에 저장된 데이터는 **함께 사라집니다**.
쓰기 레이어는 이미지 레이어에서 변경된 내용을 저장하는 영역으로, 이로 인해 **데이터가 휘발성**이라는 특징이 있습니다.
따라서 데이터를 **컨테이너 외부에 안전하게 저장하거나**, **여러 컨테이너 간에 공유**하고자 할 때는 **Docker Volume을 사용하는 것이 권장됩니다.**



11. Docker Volume

2 Volume 실습

Docker Host에 ~/work/web 이라는 디렉토리를 만듭니다.

```
mkdir -p ~/work/web && cd $_
```

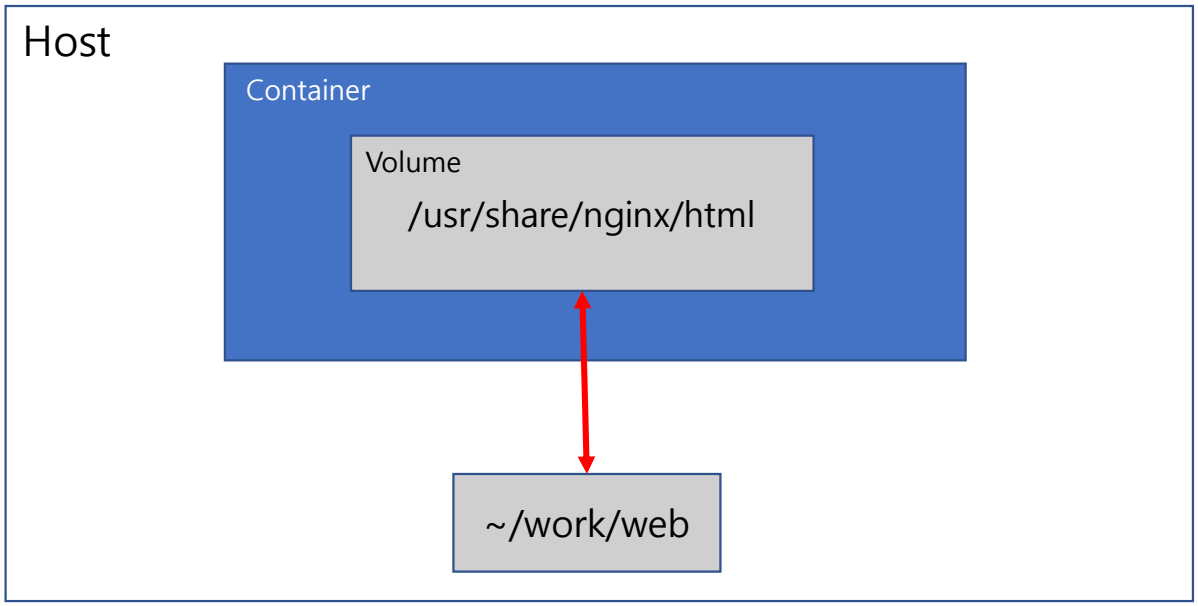
다음을 실행하여 ~/work/web/index.html 파일을 만듭니다.

```
echo "This is my web site Version 1" > index.html
```

11. Docker Volume

2 Volume 실습

docker container run --name mynginx -d -p 80:80 -v ~/work/web/:/usr/share/nginx/html nginx



11. Docker Volume

2 Volume 실습

docker container ls 로 Container가 정상적으로 실행 되었는지 확인 한 후 접속 해 봅니다.

```
[ec2-user@ip-172-31-33-52 web]$ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                   NAMES
ae3ccc4200c6   nginx     "/docker-entrypoint.…"   6 seconds ago   Up 6 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp   mynginx
```

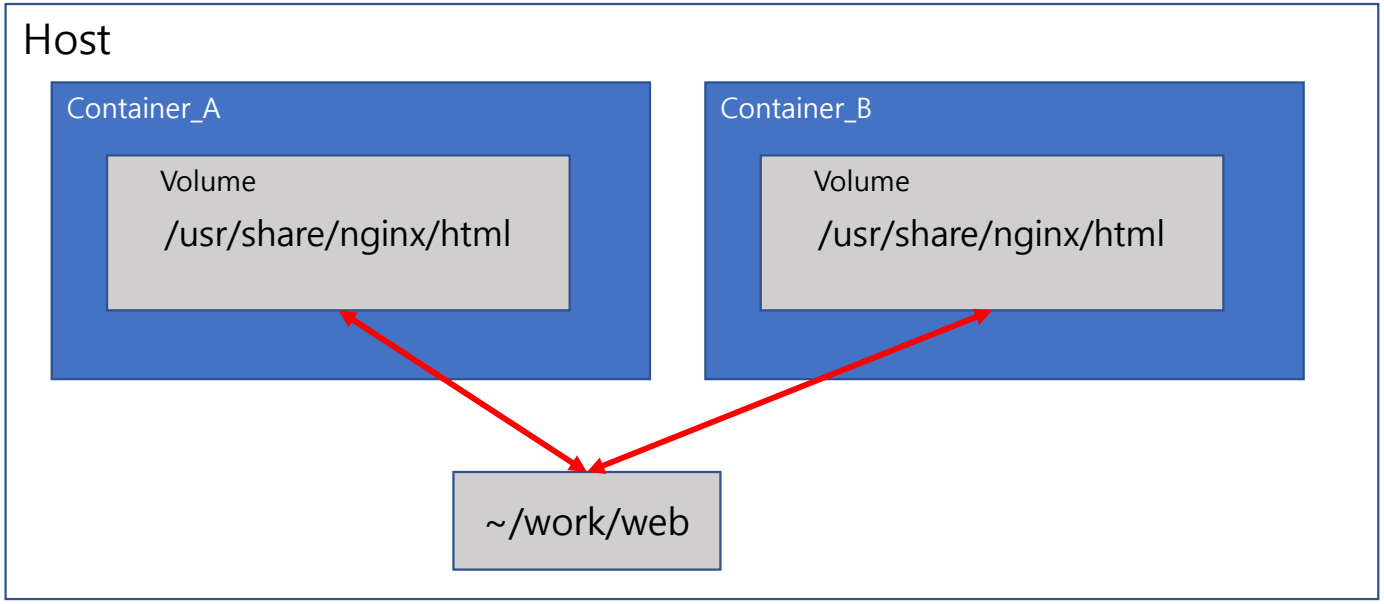


This is my web site Version 1

11. Docker Volume

2 Volume 실습

docker container run --name mynginx2 -d -p 8080:80 -v ~/work/web/:/usr/share/nginx/html nginx



11. Docker Volume

2 Volume 실습

두번째 컨테이너가 생성 된 것을 확인 하고 접속을 해봅니다.

```
[ec2-user@ip-172-31-33-52 web]$ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              PORTS                                                  NAMES
2eb463bbcb73   nginx     "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   mynginx2
ae3ccc4200c6   nginx     "/docker-entrypoint.…"   5 minutes ago        Up 5 minutes        0.0.0.0:80->80/tcp, [::]:80->80/tcp       mynginx
[ec2-user@ip-172-31-33-52 web]$
```



This is my web site Version 1

11. Docker Volume

2 Volume 실습

다음 실습을 위해서 모든 컨테이너와 이미지를 삭제 합니다.

docker container rm -f $(docker container ls -aq) ➔ 만들어진 모든 컨테이너 삭제

docker image rm -f $(docker image ls -q) ➔ 모든 local 이미지 삭제

12. Container 연동 실습

① Java로 개발한 application container와 MYSQL 연동하기

DB 컨테이너 만들기 – 다음을 실행하여 DB 컨테이너를 실행합니다.

docker network create --driver bridge mynet ➔ 앞선 실습에서 진행 했으므로 입력하지 않아도 됩니다.

```
docker container run --name mysql-svc -d --net mynet ₩
-e MYSQL_ROOT_PASSWORD=frodo ₩
-e MYSQL_DATABASE=frodo ₩
-e MYSQL_USER=frodo ₩
-e MYSQL_PASSWORD=frodo ₩
-e MYSQL_ROOT_HOST= ' % '  ₩
mysql:8
```

12. Container 연동 실습

1 Java로 개발한 application container와 MYSQL 연동하기

DB 컨테이너 생성 확인 – docker container ls

DB 컨테이너에 접속하기 – docker container exec -it mysql-svc bash

DB에 접속하기 – mysql -u frodo -p

Database 확인 하기 – show databases;

12. Container 연동 실습

1 Java로 개발한 application container와 MYSQL 연동하기

```
[ec2-user@ip-172-31-33-52 web]$ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                  NAMES
7dd4a5d708a4   mysql:8   "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes   3306/tcp, 33060/tcp   mysql-svc
[ec2-user@ip-172-31-33-52 web]$ docker container exec -it mysql-svc bash
bash-5.1# mysql -u frodo -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.4.5 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| frodo              |
| information_schema |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)

mysql>
```

12. Container 연동 실습

1 Java로 개발한 application container와 MYSQL 연동하기

Application 연동하기

docker container run --name myapp -d -p 80:8080 --net mynet frodolove/java:1.0

docker container ls로 두개 container가 정상적으로 running 중인지 확인 합니다.

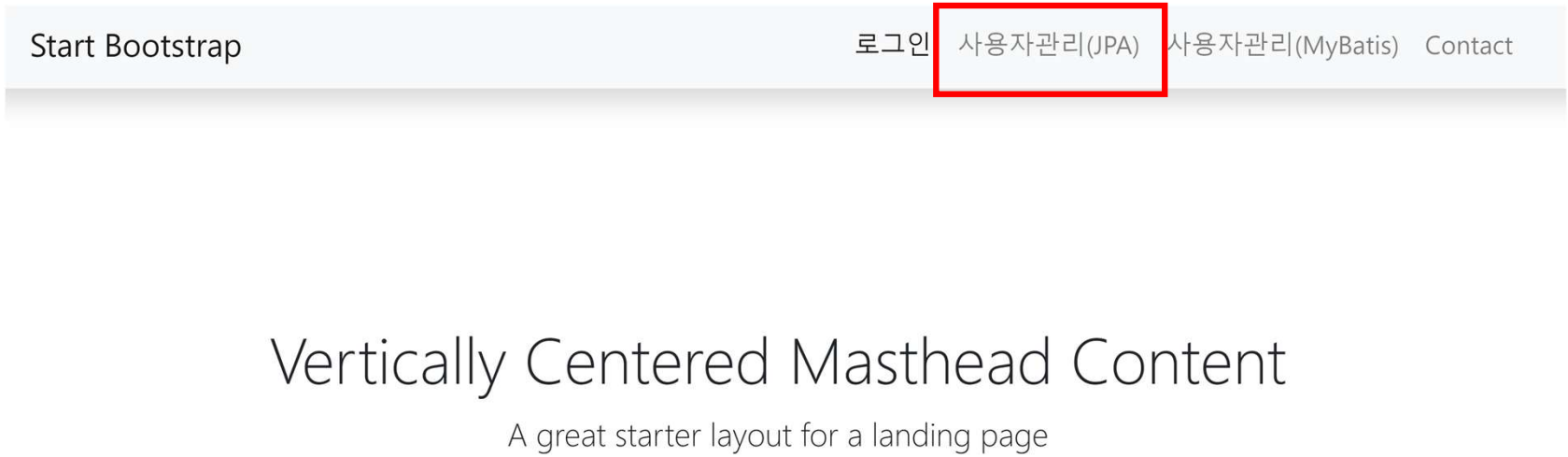
```
[ec2-user@ip-172-31-33-52 web]$ docker container ls
CONTAINER ID   IMAGE               COMMAND                  CREATED          STATUS          PORTS                                          NAMES
27e566ceeb01   frodolove/java:1.0  "java -jar /myspring…"   10 seconds ago   Up 9 seconds    0.0.0.0:80->8080/tcp, [::]:80->8080/tcp        myapp
7dd4a5d708a4   mysql:8             "docker-entrypoint.s…"   5 minutes ago    Up 5 minutes    3306/tcp, 33060/tcp                            mysql-svc
[ec2-user@ip-172-31-33-52 web]$
```

12. Container 연동 실습

2 접속 테스트 하기

아래와 같이 페이지가 오픈 되면 정상적으로 연동 되었음을 알 수 있습니다.

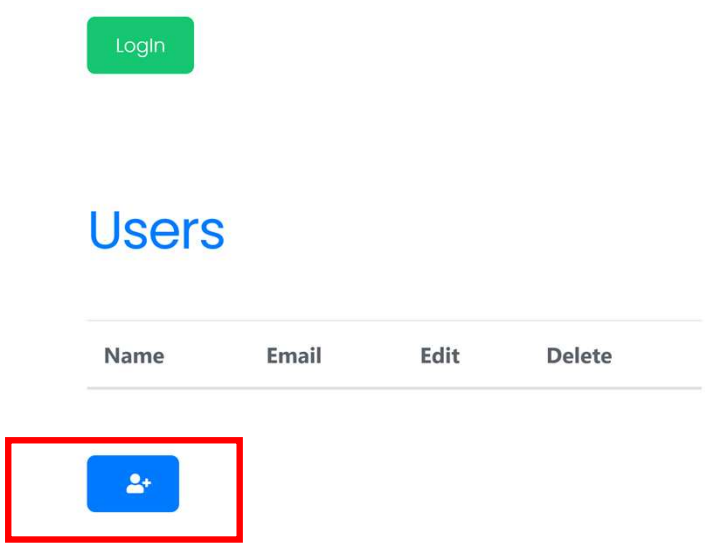
사용자관리(JPA) 부분을 클릭 합니다.

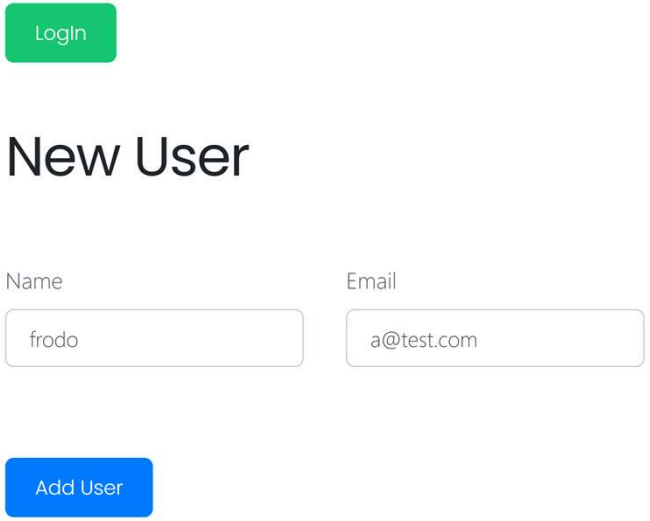


12. Container 연동 실습

2 접속 테스트 하기

아래 +아이콘을 클릭 하고 간단한 사용자를 입력 해 봅니다.





12. Container 연동 실습

2 접속 테스트 하기

DB에 직접 접속해서 table 레코드를 조회 합니다.

```
[ec2-user@ip-172-31-33-52 web]$ docker container exec -it  mysql-svc bash
bash-5.1# mysql -u frodo -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 8.4.5 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use frodo
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from user;
+----+------------------+-------+
| id | email            | name  |
+----+------------------+-------+
|  1 | frodo@frodo.co.kr | frodo |
+----+------------------+-------+
1 row in set (0.00 sec)
```

12. Container 연동 실습

2 접속 테스트 하기

아래 명령어로 DB에 직접 레코드를 입력 합니다.

INSERT INTO user (email, name)

VALUES ('neo@frodo.co.kr', 'neo');

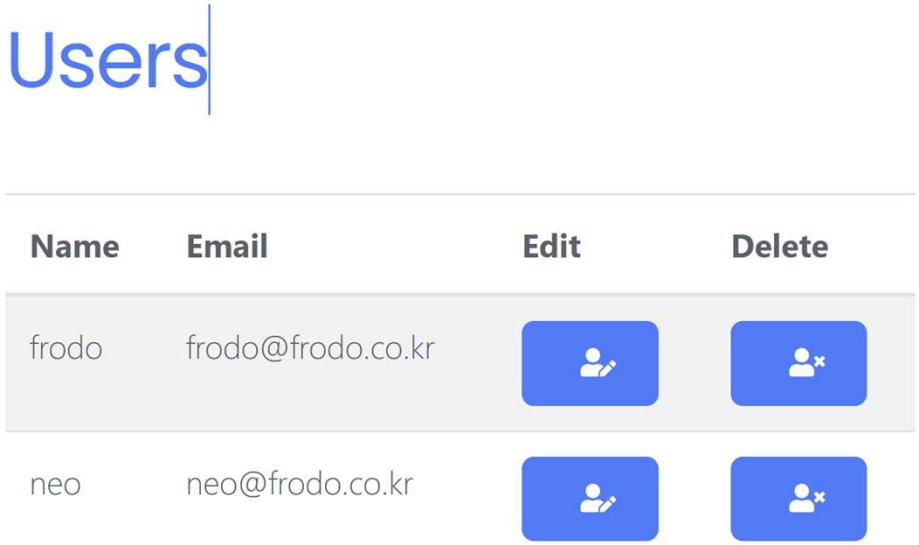
```
mysql> INSERT INTO user (email, name)
    -> VALUES ('neo@frodo.co.kr', 'neo');
Query OK, 1 row affected (0.00 sec)

mysql> select * from user;
+----+-------------------+-------+
| id | email             | name  |
+----+-------------------+-------+
|  1 | frodo@frodo.co.kr | frodo |
|  2 | neo@frodo.co.kr   | neo   |
+----+-------------------+-------+
2 rows in set (0.00 sec)
```

12. Container 연동 실습

2 접속 테스트 하기

브라우저에서 생성된 user가 보이는 지 확인 합니다.



12. Container 연동 실습

2 접속 테스트 하기

다음 실습을 위해서 모든 컨테이너와 이미지를 삭제 합니다.

docker container rm -f $(docker container ls -aq) ➔ 만들어진 모든 컨테이너 삭제

docker image rm -f $(docker image ls -q) ➔ 모든 local 이미지 삭제

13. Docker 통합 Lab

1 접속 테스트 하기

Chapter 2

Docker compose

1. Docker Compose

1 Docker compose란

- ✔ 단일 서버에서 여러 컨테이너를 프로젝트 단위로 묶어서 관리
- ✔ docker-compose.yaml 파일을 통해 명시적 관리
- ✔ 프로젝트 단위로 도커 네트워크와 볼륨 관리 프로젝트 내 서비스 간 의존성 정의 가능
- ✔ 프로젝트 내 서비스 디스커버리 자동화
- ✔ 쉬운 컨터이너 수평확장



1. Docker Compose

2 Docker compose 구성요소

프로젝트 (Project)

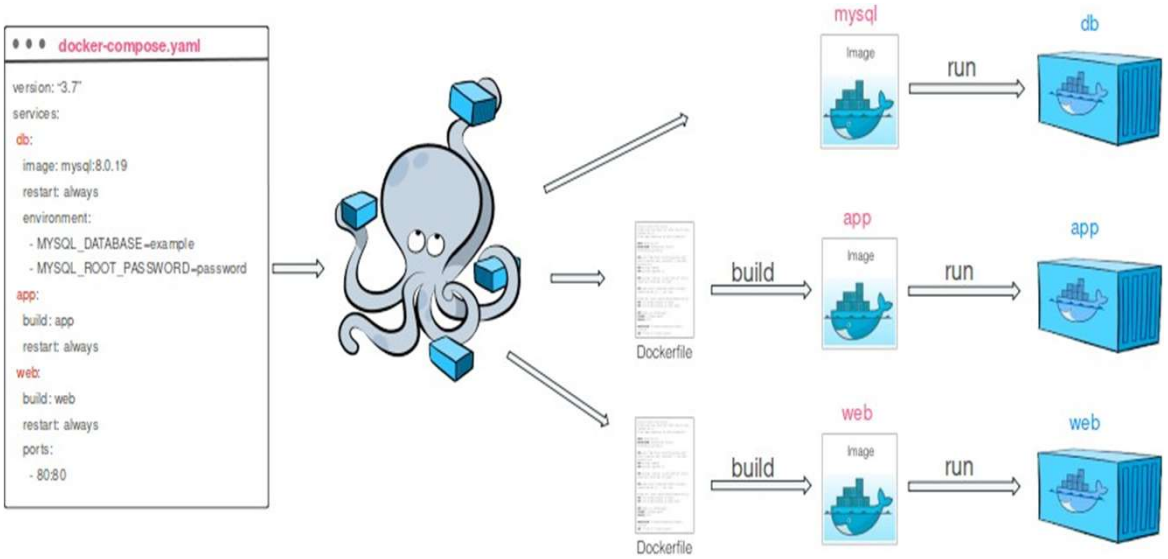
도커 컴포즈에서 다루는 워크스페이스 단위.
함께 관리하는 서비스 컨테이너의 묶음. 프로젝트 단위로 기본 도커 네트워크가 생성 됨

서비스 (Service)

도커 컴포즈에서 컨테이너를 관리하기 위한 단위.
scale을 통해 서비스 컨테이너의 수 확장 가능.

컨테이너 (Container)

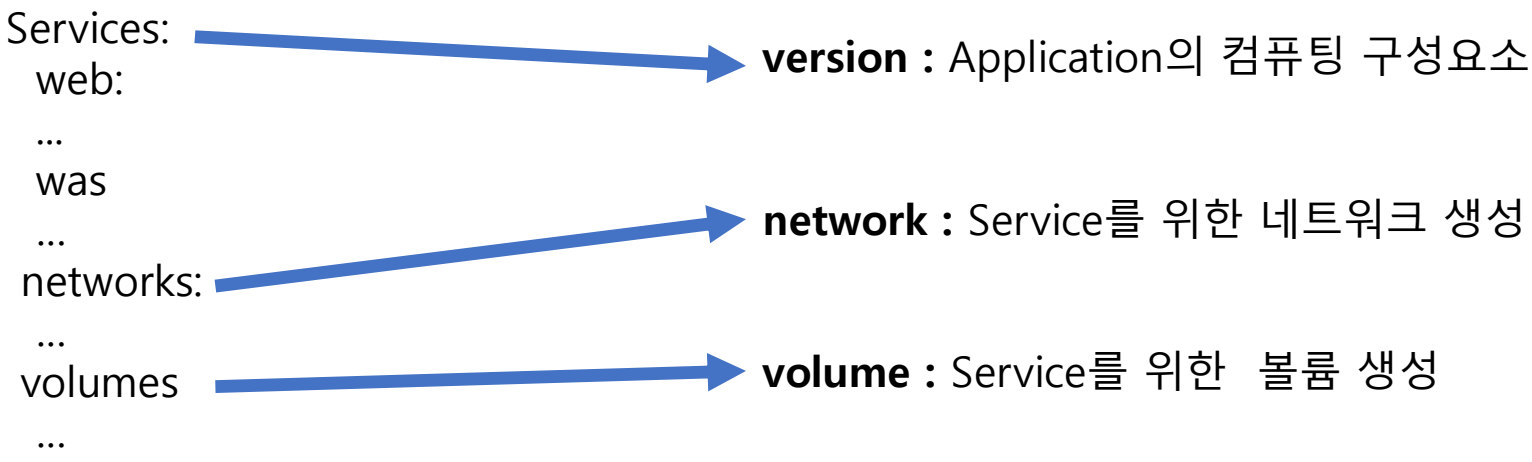
서비스를 통해 컨테이너 관리.



1. Docker Compose

3 Docker compose yaml

Compose 파일은 Docker 애플리케이션의 서비스, 네트워크 및 볼륨을 정의 하는 YAML 파일입니다. YAML 파일 예시는 아래와 같습니다.



1. Docker Compose

4 Docker-compose 주요 명령어

docker compose version
버전 확인

docker compose ls
실행중인 프로젝트 목록 확인

docker compose ls -a
전체 프로젝트 목록 확인

docker compose up -d --scale *서비스명=확장할 숫자*
전체 프로젝트 목록 확인

2. Docker Compose 실습하기

1 Docker compose 실행하기

Docker compose yaml 파일이 있는 경로로 이동

docker compose up -d

docker compose ls

2. Docker Compose 실습하기

4 Docker compose 중단 및 삭제 하기

docker-compose stop 중단하기

docker-compose ls - 명령어로 확인 가능

docker-compose down 삭제하기

Chapter 3

쿠버네티스

1. Kubernetes 기본

1 배경

컨테이너를 도입했지만 대규모로 컨테이너를 관리하기란 쉽지 않습니다.

어떻게 하면 컨테이너 인프라를 더 효과적으로 관리할 수 있을까요?

Kubernetes를 사용해 보세요

1. Kubernetes 기본

2 개요

구글 내부에서 사용하던 배포시스템 borg를 기반으로 재 작성하여 2014년에 오픈소스로 공개 되었습니다.

가장 대표적인 컨테이너 오케스트레이션 시스템으로 사실상 표준 (De Facto Standard)입니다.



1. Kubernetes 기본

3 기능

- ✓ Stateless 및 Stateful 애플리케이션 지원
- ✓ 자동 확장
- ✓ 리소스 한도
- ✓ 확장성
- ✓ 이동성

1. Kubernetes 기본

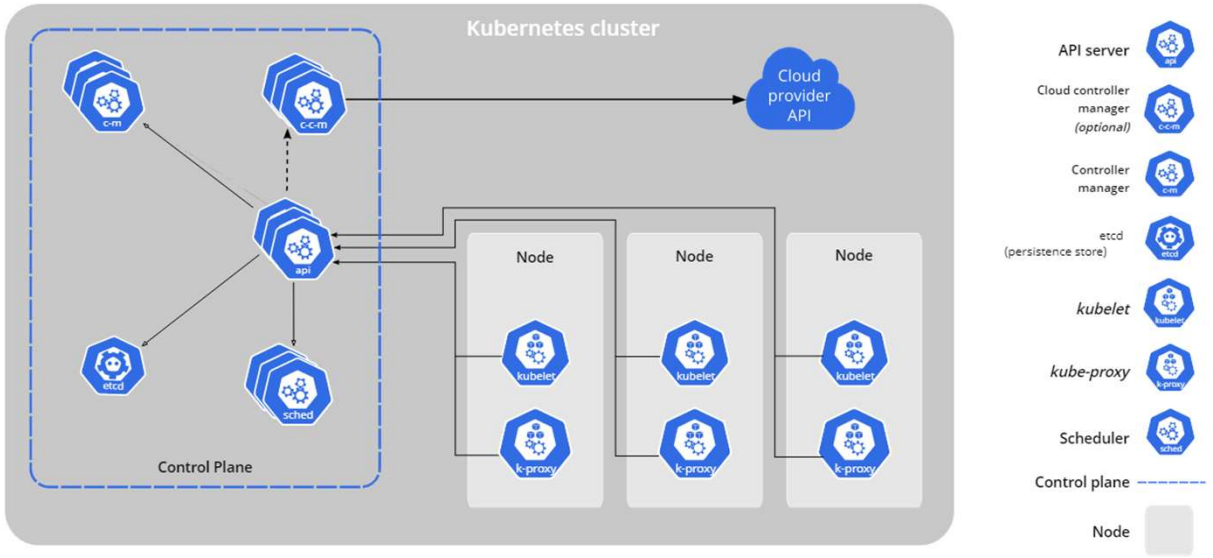
Kubernetes Components

Control Plane

- ✓ 클러스터를 관리하는 역할
- ✓ 상태 관리 및 명령어 처리

Node

- ✓ 어플리케이션 컨테이너 실행



1. Kubernetes 기본

5 Control Plane

API Server

- ✓ 클러스터를 관리하는 역할
- ✓ 상태 관리 및 명령어 처리

Scheduler

- ✓ 노드의 자원 사용 상태를 관리하여, 새로운 워크로드 배포 관리

Controller Manager

- ✓ 여러 컨트롤러 프로세스를 관리
- ✓ 각 컨트롤러는 클러스터로부터 특정 리소스 상태의 변화를 감지하여 클러스터에 반영

ETCD

- ✓ 분산 Key-Value 저장소
- ✓ 클러스터 상태 저장

1. Kubernetes 기본

Node

Kubelet

- ✓ 컨테이너 런타임과 통신하여 컨테이너 라이프 사이클 관리
- ✓ API 서버와 통신하며 노드의 리소스 관리

CRI

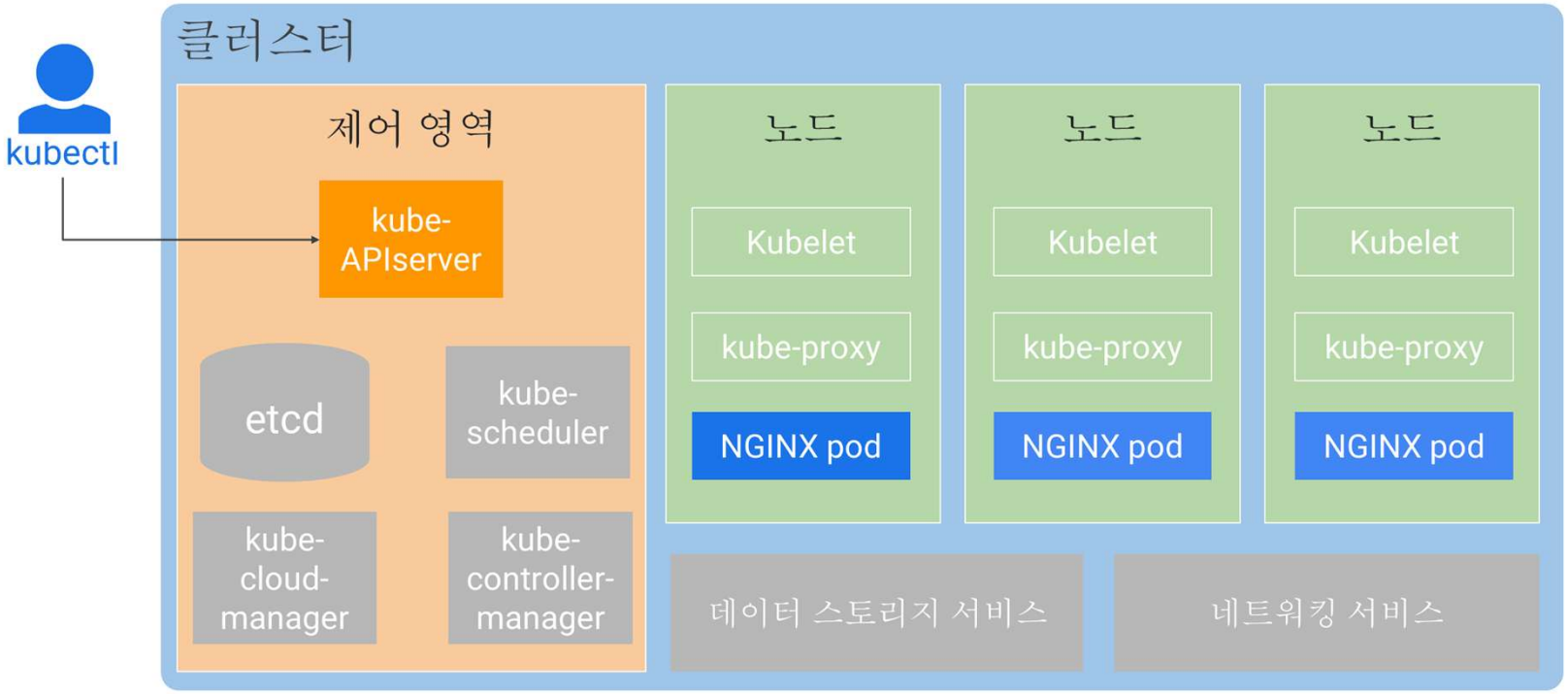
- ✓ Kubelet이 컨테이너 런타임과 통신할 때 사용되는 인터페이스

Kube-proxy

- ✓ 오버레이 네트워크 구성
- ✓ 네트워크 프록시 및 내부 로드밸런스 역할 수행

1. Kubernetes 기본

Kubectl



1. Kubernetes 기본

Kubectl

- ✓ $HOME/.kube/config에 있는 구성(config) 파일 사용
- ✓ 구성 파일에 포함되는 내용:
  - 대상 클러스터 이름
  - 클러스터의 사용자 인증 정보
- ✓ 현재 구성 보기: kubectl config view

1. Kubernetes 기본

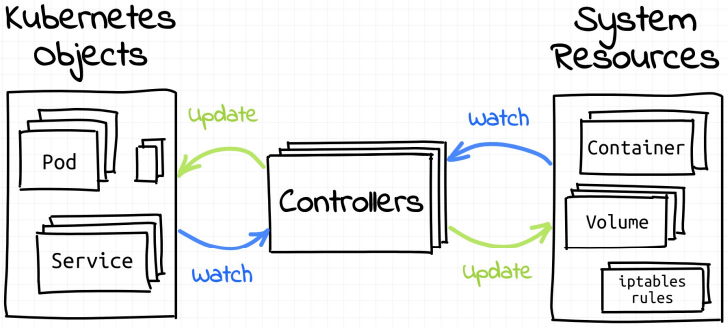
API와 리소스 오브젝트

API 리소스

- ✓ 쿠버네티스가 관리할 수 있는 오브젝트 종류
- ✓ Pod, service, configmap, secret
- ✓ Node, ServiceAccount

오브젝트

- ✓ API리소스를 인스턴스화 한 것



1. Kubernetes 기본

API와 리소스 오브젝트

현재 쿠버네티스 클러스터가 지원하는 API 목록 출력

Kubectl api-resources

```
[frodo@console ~]$  kubectl api-resources
W1127 21:01:05.060401    3647 gcp.go:119] WARNING: the gcp auth plugin is deprecated in v1.22+, unavailable in v1.26+; use gcloud instead.
To learn more, consult https://cloud.google.com/blog/products/containers-kubernetes/kubectl-auth-changes-in-gke
NAME                              SHORTNAMES   APIVERSION                             NAMESPACED   KIND
bindings                                       v1                                     true         Binding
componentstatuses                 cs           v1                                     false        ComponentStatus
configmaps                        cm           v1                                     true         ConfigMap
endpoints                         ep           v1                                     true         Endpoints
events                            ev           v1                                     true         Event
limitranges                       limits       v1                                     true         LimitRange
namespaces                        ns           v1                                     false        Namespace
nodes                             no           v1                                     false        Node
persistentvolumeclaims            pvc          v1                                     true         PersistentVolumeClaim
persistentvolumes                 pv           v1                                     false        PersistentVolume
pods                              po           v1                                     true         Pod
podtemplates                                   v1                                     true         PodTemplate
replicationcontrollers            rc           v1                                     true         ReplicationController
resourcequotas                    quota        v1                                     true         ResourceQuota
secrets                                        v1                                     true         Secret
serviceaccounts                   sa           v1                                     true         ServiceAccount
services                          svc          v1                                     true         Service
mutatingwebhookconfigurations                  admissionregistration.k8s.io/v1        false        MutatingWebhookConfiguration
validatingwebhookconfigurations                admissionregistration.k8s.io/v1        false        ValidatingWebhookConfiguration
customresourcedefinitions         crd,crds     apiextensions.k8s.io/v1                false        CustomResourceDefinition
apiservices                                    apiregistration.k8s.io/v1              false        APIService
controllerrevisions                            apps/v1                                true         ControllerRevision
```

1. Kubernetes 기본

API와 리소스 오브젝트

특정 API 리소스에 대해 간단한 설명 확인

kubectl explain API resource

```
[frodc@console ~]$ kubectl explain deployment
W1127 21:02:23.892265    3683 gcp.go:119] WARNING: the gcp auth plugin is deprecated in v1.22+, unavailable in v1.26+; use gcloud instead.
To learn more, consult https://cloud.google.com/blog/products/containers-kubernetes/kubectl-auth-changes-in-gke
KIND:     Deployment
VERSION:  apps/v1

DESCRIPTION:
     Deployment enables declarative updates for Pods and ReplicaSets.

FIELDS:
   apiVersion   <string>
     APIVersion defines the versioned schema of this representation of an
     object. Servers should convert recognized schemas to the latest internal
     value, and may reject unrecognized values. More info:
     https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources

   kind <string>
     Kind is a string value representing the REST resource this object
     represents. Servers may infer this from the endpoint the client submits
     requests to. Cannot be updated. In CamelCase. More info:
     https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds

   metadata     <Object>
     Standard object's metadata. More info:
     https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata

   spec <Object>
     Specification of the desired behavior of the Deployment.

   status       <Object>
     Most recently observed status of the Deployment.
```

2. Kubernetes 실습 환경 구성

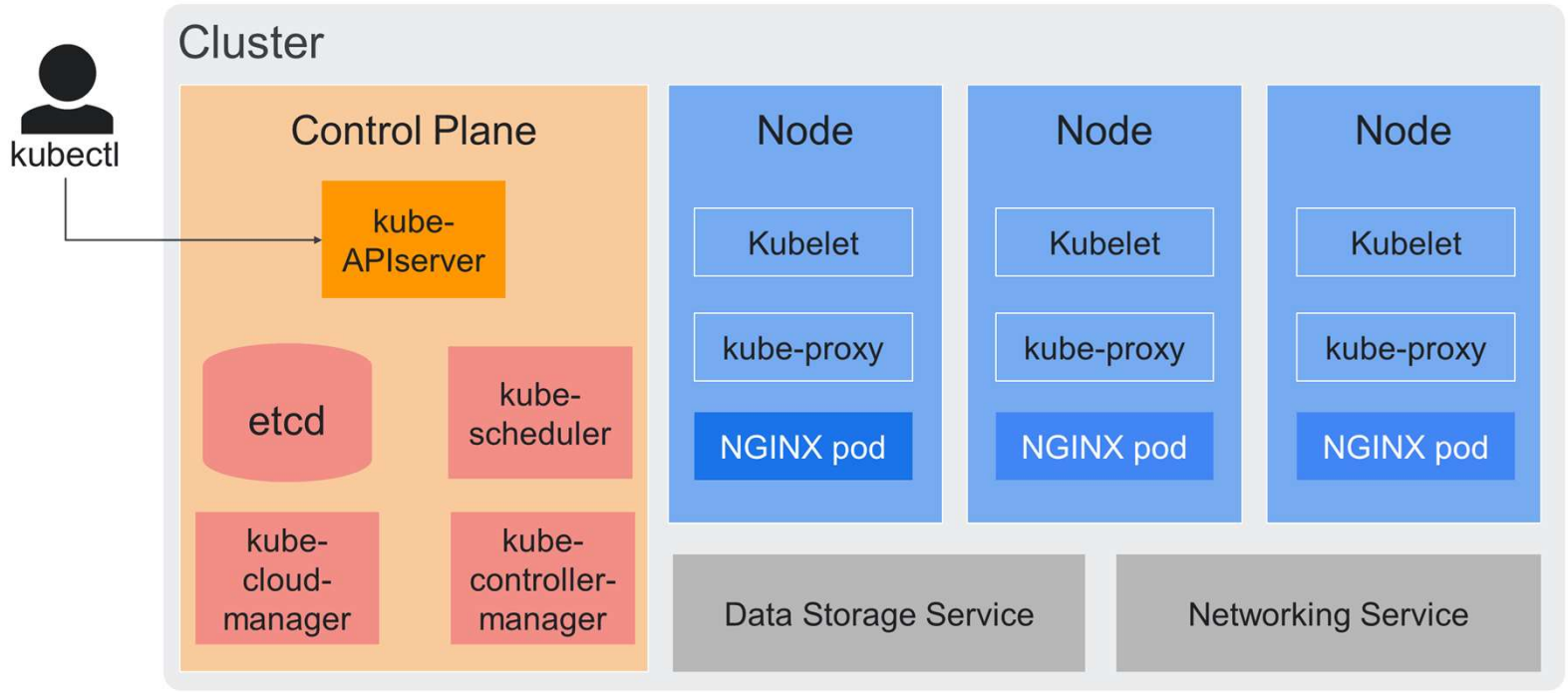
Chapter 4

쿠버네티스 기초 및 활용

실습으로 이해하는 Kubernetes

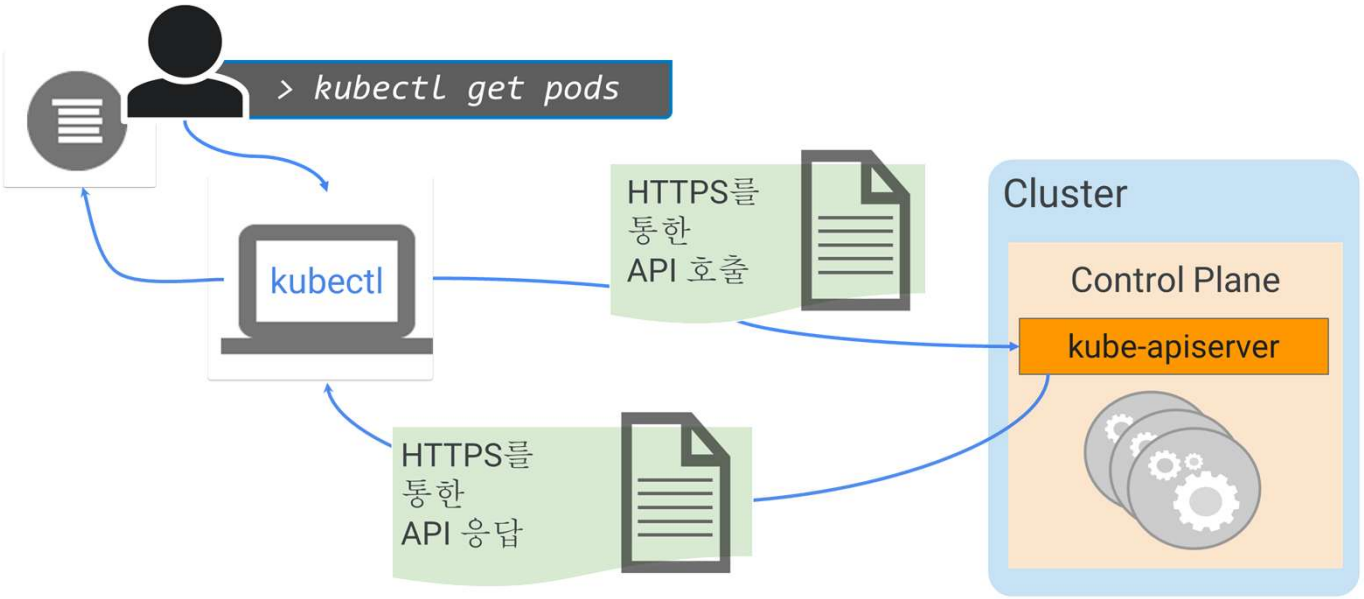
1. Kubernetes 운영

1 개요



1. Kubernetes 운영

2 Kubectl을 이용하여 클러스터 내 pod 목록 보기



1. Kubernetes 운영

3 Kubectl 구성하기

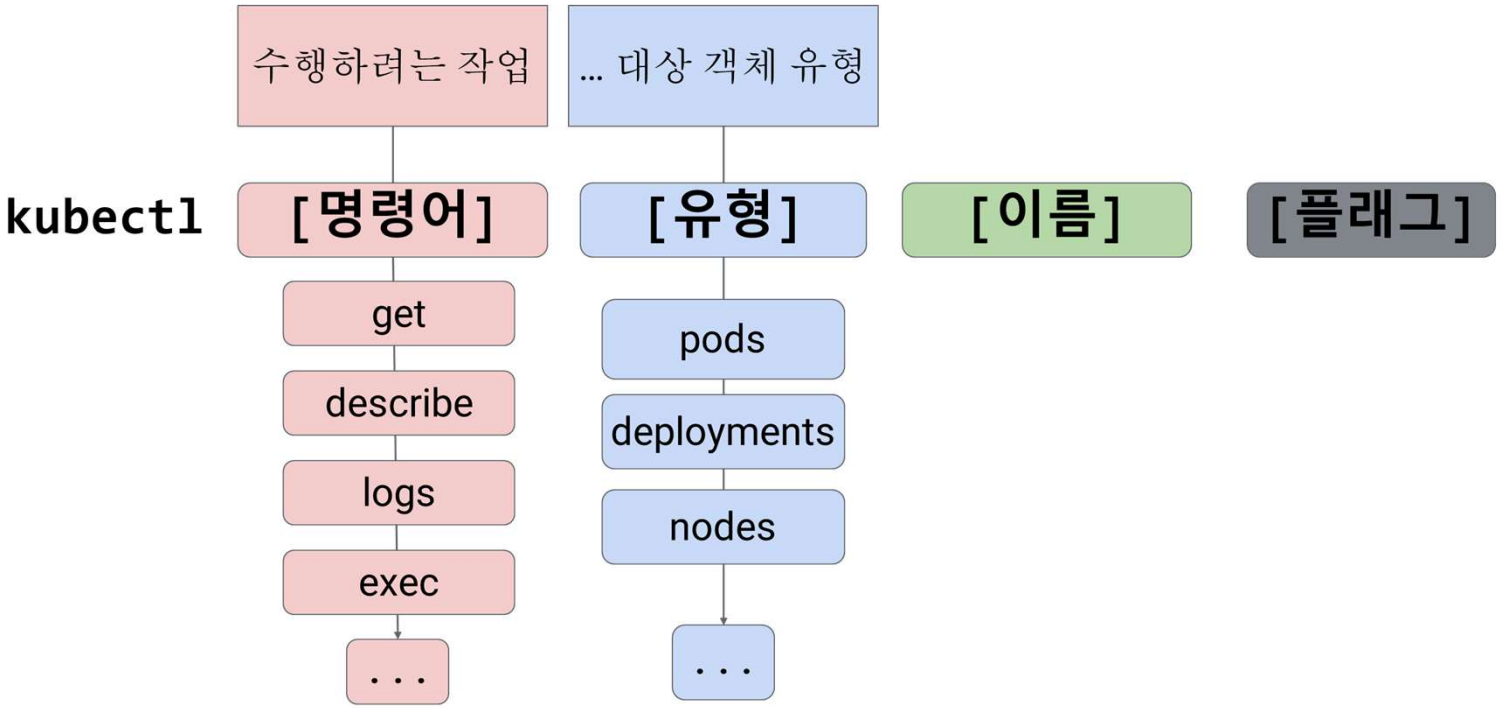
$HOME/.kube/config에 있는 conig 파일 사용

Config 파일에 포함되는 내용:

- ✓ 대상 클러스터 이름
- ✓ 클러스터의 사용자 인증 정보

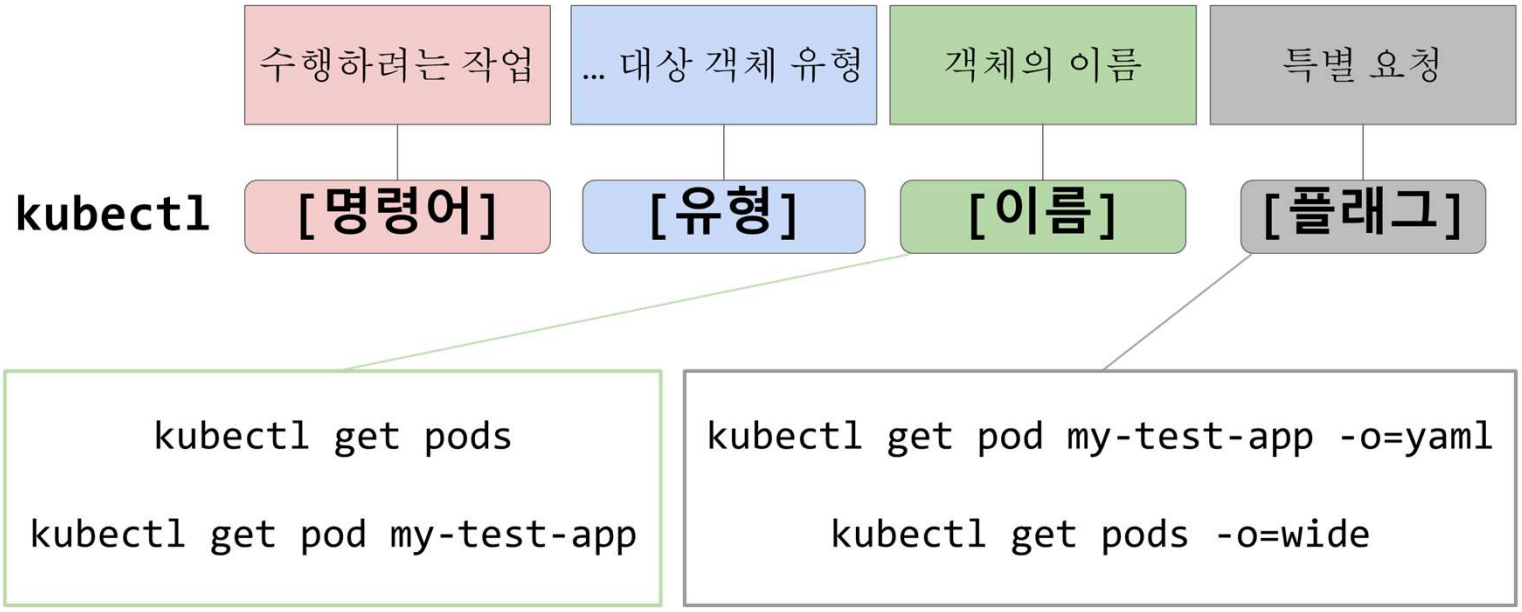
1. Kubernetes 운영

3 Kubectl 구성하기



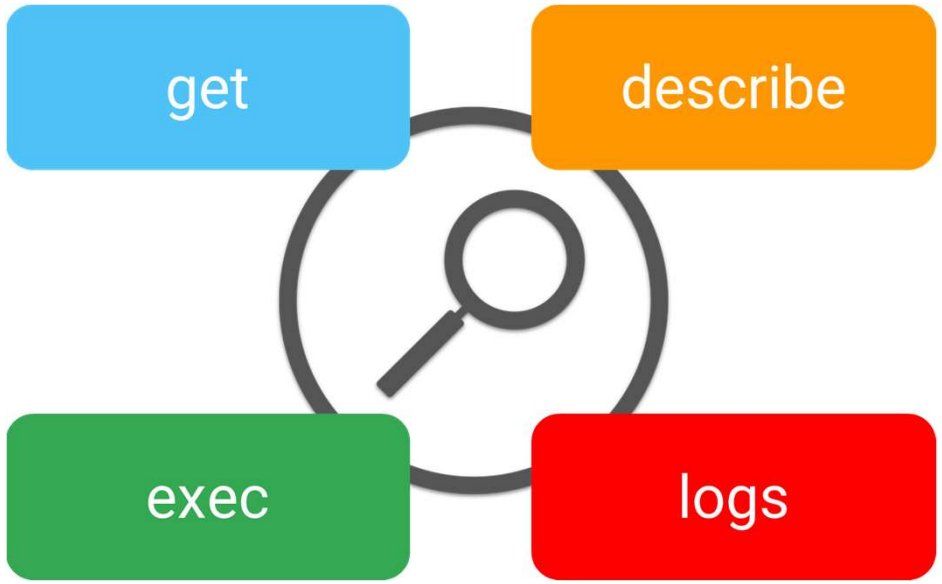
1. Kubernetes 운영

3 Kubectl 구성하기



1. Kubernetes 운영

④ Workload에 관한 정보 수집



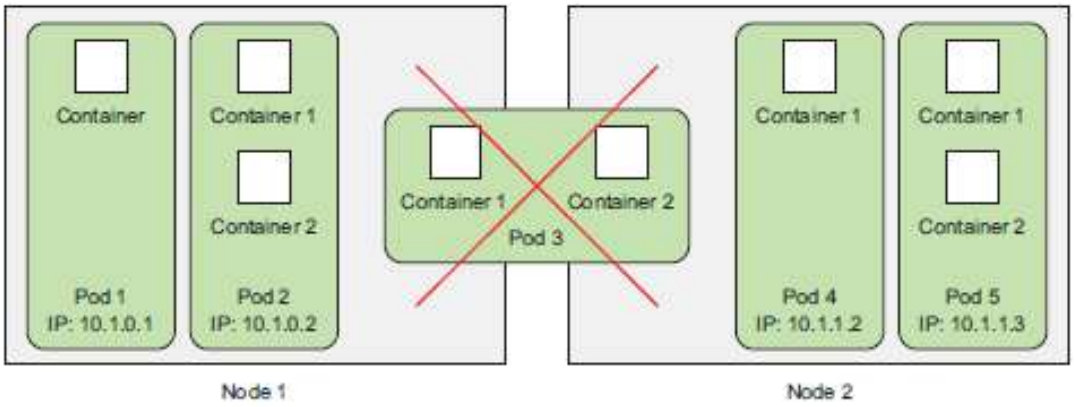
2. pod

1 개요

pod는 컨테이너의 그룹이며 쿠버네티스의 기본 빌딩 블록

쿠버네티스에서는 pod로 배포하고 운영

pod는 안에 모든 컨테이너는 한 개의 워크 노드에서 실행

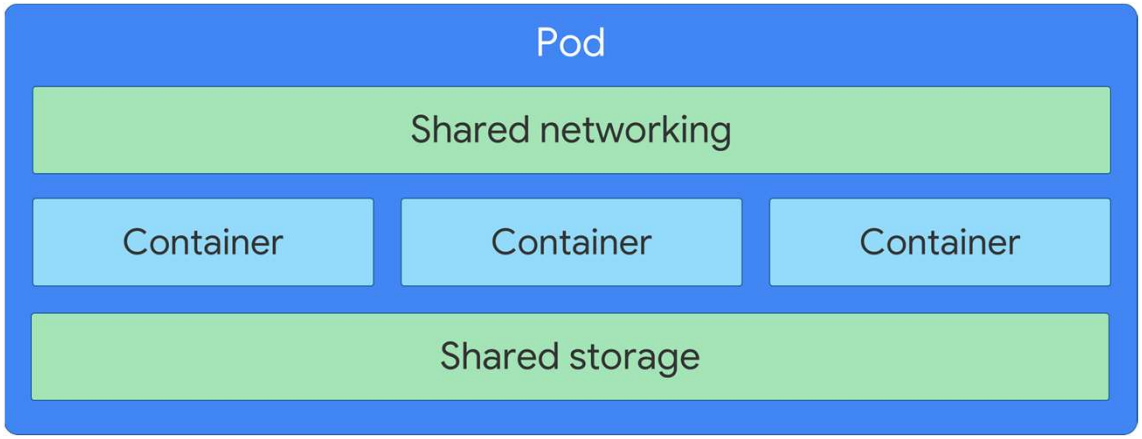


2. pod

② pod 간 네트워크

한 pod 안에서 컨테이너는 리소스를 공유.

같은 pod 안에 컨테이너는 같은 pod의 IP를 사용하기 때문에 서로 다른 포트 사용

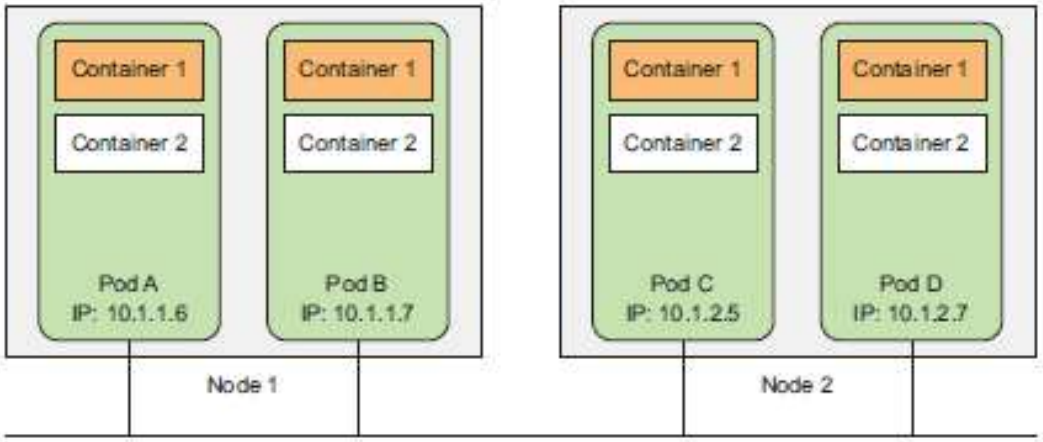


2. pod

➋ pod 간 네트워크

같은 클러스터 안에서 pod는 다른 pod로의 IP접근 가능.

서로 다른 워커 노드에 있는 pod간이라도 IP통신 가능



2. pod

3. pod 생성하기

kubectl 명령어를 이용하여 생성하기

kubectl run nginx --image=nginx

kubectl 명령어를 이용하여 yaml 만들기 예

kubectl run nginx --image=nginx --labels=app=nginx --dry-run=client -o yaml > nginx-pod.yaml

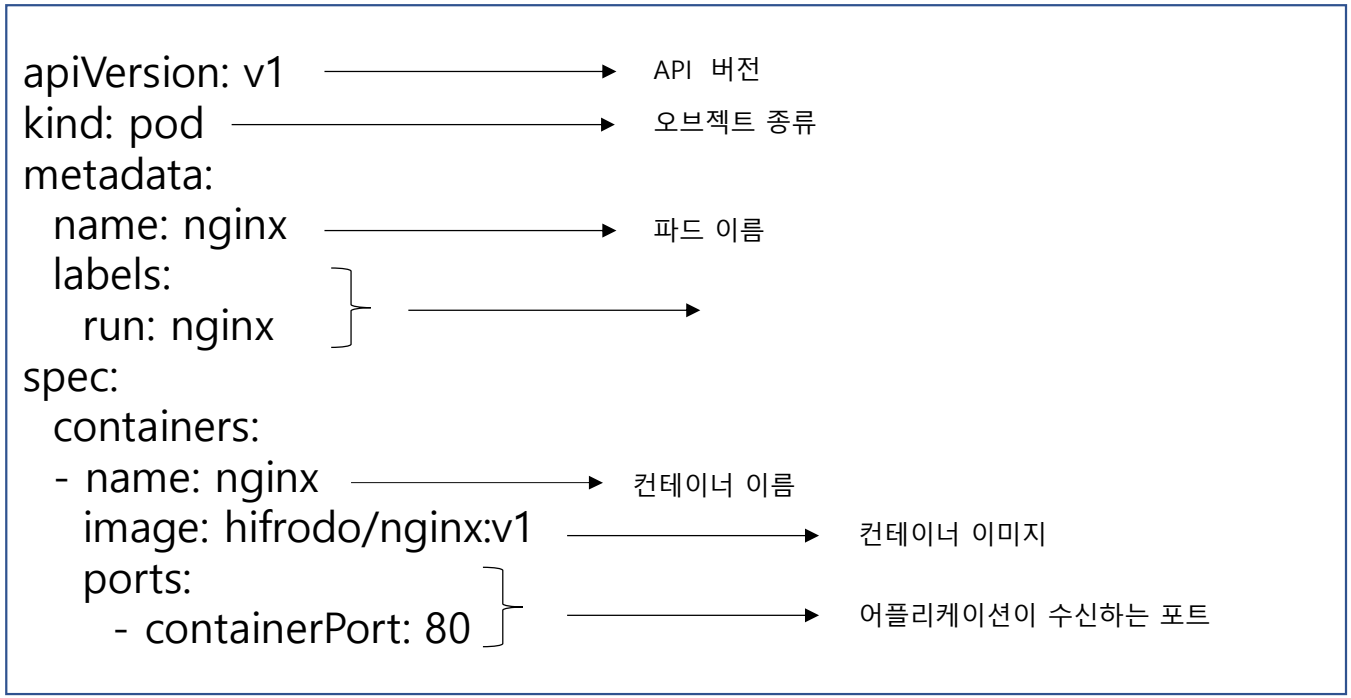
실행하기

kubectl apply -f nginx-pod.yaml

2. pod

3 pod 생성하기

pod YAML 파일



2. pod

3 pod 생성하기

YAML 파일 실행하기

kubectl apply –f 파일 명

생성된 pod 확인 하기

kubectl get pods

2. pod

3. pod 생성하기

생성된 POD 정보를 YMAL 파일로 출력하기

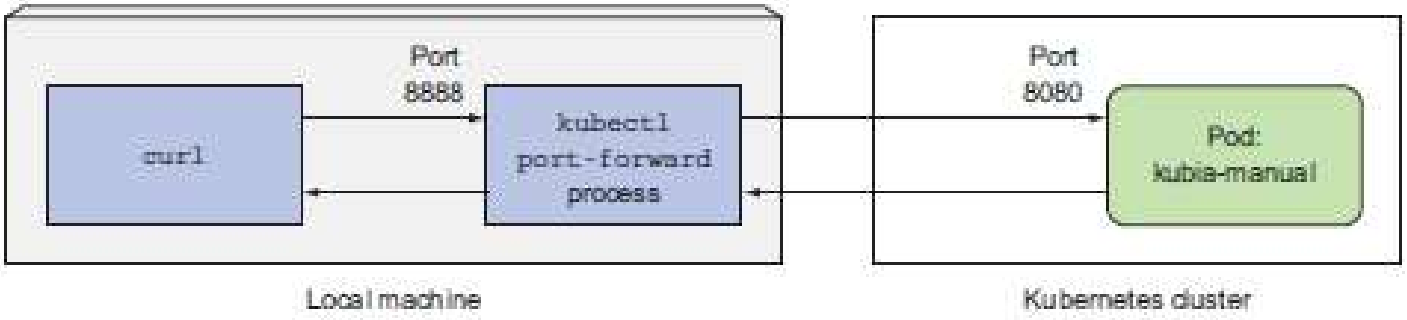
kubectl get pod 파드명 -o yaml

```
C:₩Users₩hifro₩AppData₩Local₩Google₩Cloud SDK₩yaml₩study>kubectl get pod nginx -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubernetes.io/limit-ranger: 'LimitRanger plugin set: cpu request for container
      nginx'
  creationTimestamp: "2020-05-16T22:27:10Z"
  labels:
    run: nginx
  name: nginx
  namespace: default
  resourceVersion: "113981"
  selfLink: /api/v1/namespaces/default/pods/nginx
  uid: 624f8e76-97c4-11ea-ae3e-42010a800022
spec:
  containers:
  - image: hifrodo/nginx:v1
    imagePullPolicy: IfNotPresent
    name: nginx
    ports:
    - containerPort: 80
      protocol: TCP
    resources:
      requests:
        cpu: 100m
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: default-token-4zpgl
      readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
```

2. pod

④ pod 접속 하기

kubectl port-forward pod명 port번호



2. pod

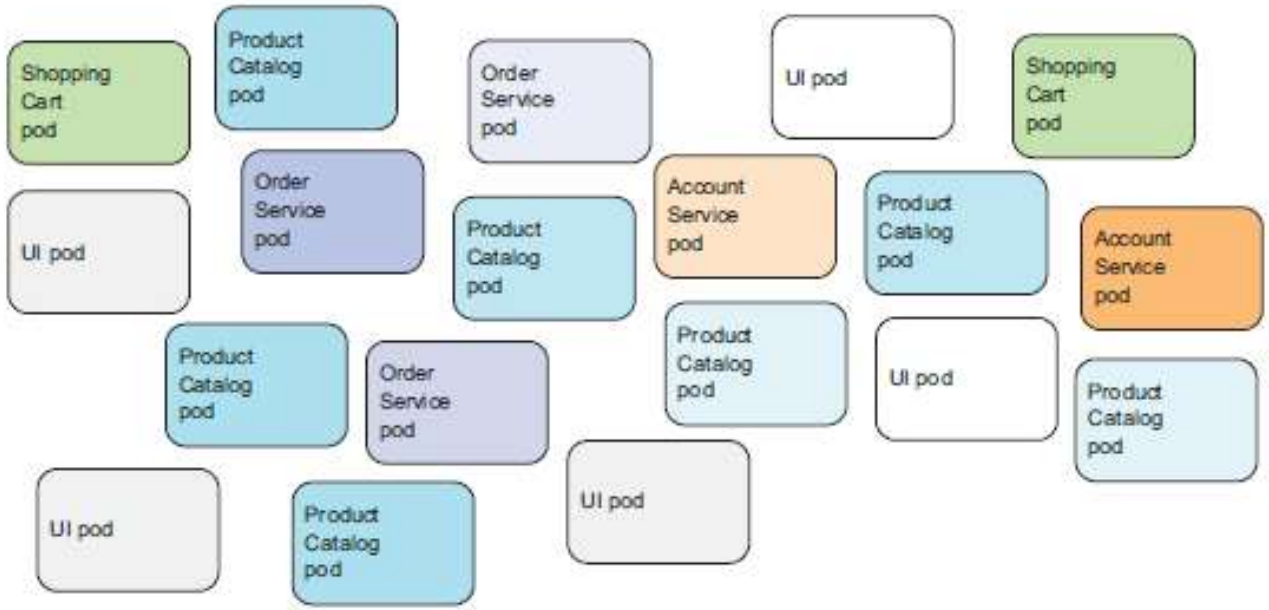
5 pod 접속 실습하기

kubectl run nginx --image=nginx

kubectl port-forward nginx 8080:80 --address=0.0.0.0

2. pod

레이블을 이용한 pod 구성



2. Pod

6 레이블을 이용한 Pod 구성

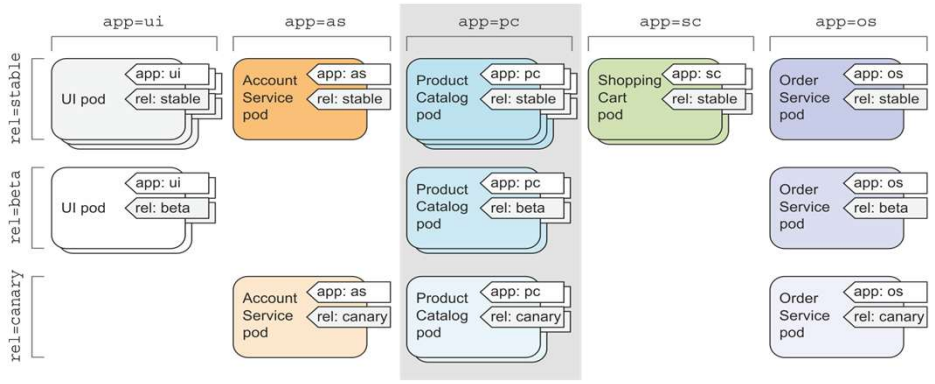
쿠버네티스 모든 리소스를 조직화 할 수 있는 강력한 쿠버네티스 기능

리소스에 첨부하는 키-값의 쌍

레이블 셀렉터를 사용해 리소스를 선택할 때 사용

리소스는 하나 이상의 레이블을 가질 수 있습니다.

레이블은 리소스를 만들 때 혹은 후에 추가가 가능



2. pod

레이블을 이용한 pod 구성

pod 생성시 레이블 지정

```yaml
apiVersion: v1
kind: pod
metadata:
  name: nginx
  labels:
    app: nginx
    env: prod
spec:
  containers:
  - name: nginx
    image: hifrodo/nginx:v1
    ports:
      - containerPort: 80
```

두개의 레이블 설정

2. pod

5 레이블을 이용한 pod 구성

파드에 레이블 확인하기

kubectl get pod --show-labels

```
C:₩Users₩hifro₩AppData₩Local₩Google₩Cloud SDK₩yaml₩study>kubectl get pod --show-labels
NAME    READY   STATUS    RESTARTS   AGE   LABELS
nginx   1/1     Running   0          56s   app=nginx,env=prod
```

레이블에 app=nginx와 env=prod 이 포함된 파드 생성 확인

특정 레이블 정보만 보기

kubectl get pod –L env

```
C:₩Users₩hifro₩AppData₩Local₩Google₩Cloud SDK₩yaml₩study>kubectl get pod -L env
NAME    READY   STATUS    RESTARTS   AGE     ENV
nginx   1/1     Running   0          3m33s   prod
```

2. pod

실습1

Kubernetes Cluster에 pod를 배포하고 확인 해 봅니다.

명령어를 통한 배포

kubectl run nginx --image=nginx 를 실행하여 nginx pod를 배포해 봅니다.

pod 확인 하기

kubectl get pod

kubectl get pod -o yaml

Kubectl get pod -o wide

2. pod

실습2

Yaml파일을 이용한 배포

```yaml
apiVersion: v1
kind: pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
      - containerPort: 80
```

kubetctl apply –f 파일명

2. pod

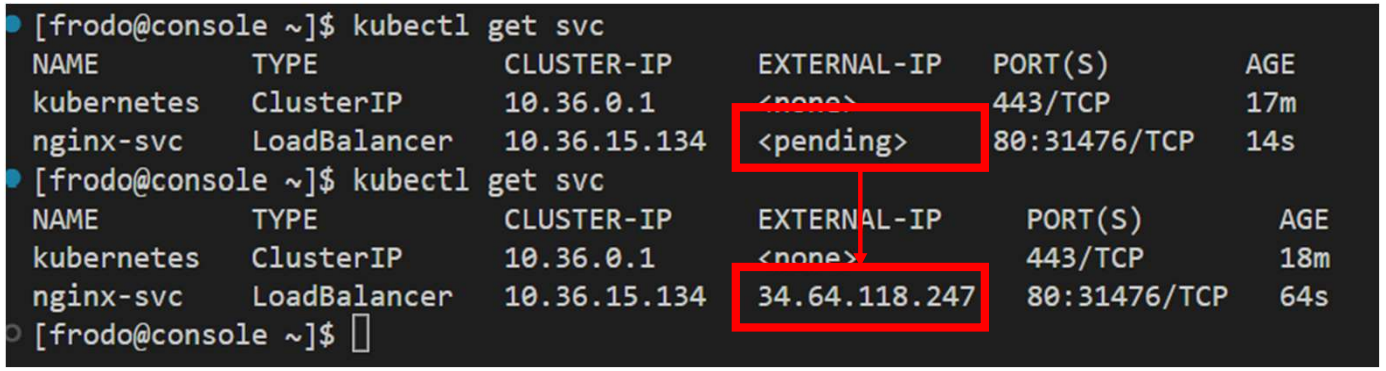
실습3

pod를 외부로 노출 해 보기

kubectl expose pod nginx --name nginx-svc --port=80 --type=LoadBalancer

서비스 IP 확인 하기

kubectl get service (=kubectl get svc)



2. pod

pod 확장하기

서비스의 안정을 위해서 여러 개의 pod가 동작 하도록 만들고 싶습니다.

그럴 때는 어떻게 해야 할까요?

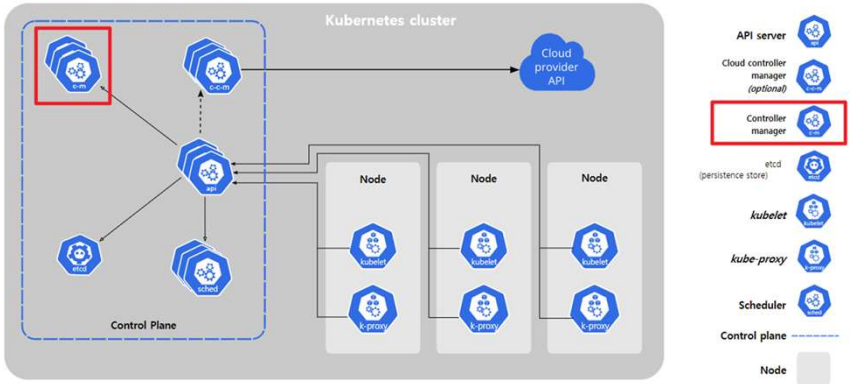
예를 들어 nginx가 동작 하는 pod를 3개 실행하고 싶다면 어떻게 해야 할까요?

3. Controller

1 개요

앞서서 같은 종류의 pod를 3개 만들기 위해서 3번의 명령어를 실행 하거나 3개의 매니페스트 파일이 필요한 걸까?





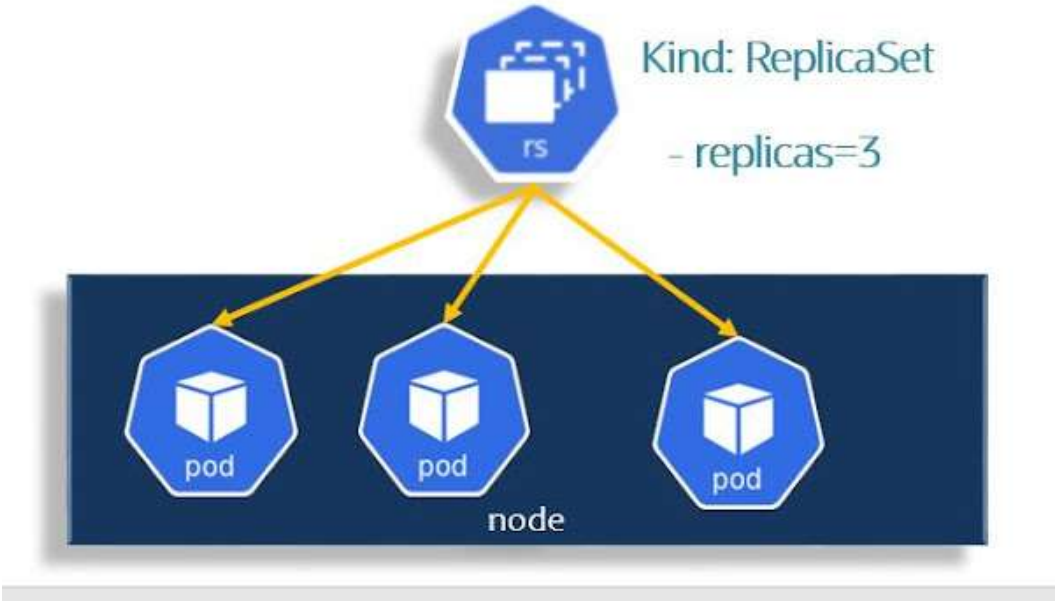
< Kubernetes Architecture >

4. Replicaset

1 개요

ReplicaSet의 목적은 pod를 생성하고 유지 하는 것입니다.

이제 더 이상 원하는 수의 pod를 만들기 위해서 여러 번의 명령어를 실행하거나 여러 개의



4. Replicaset

실습1 RS생성하기

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs1
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: frodo5020/nginx:1.0
```

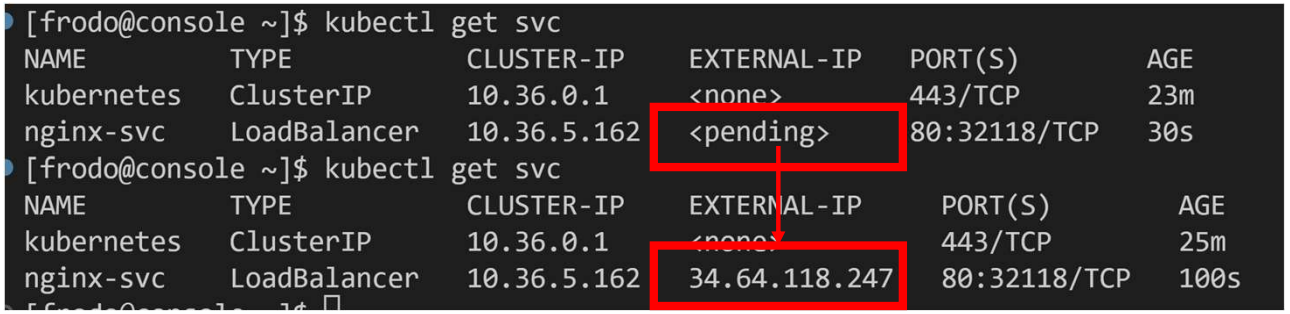
```
[frodo@console ~]$ kubectl get pod
NAME              READY   STATUS    RESTARTS   AGE
nginx-rs1-c48w6   1/1     Running   0          20s
nginx-rs1-hr2c5   1/1     Running   0          20s
nginx-rs1-pgnft   1/1     Running   0          20s
```

4. Replicaset

실습1 RS생성하기

아래와 같이 서비스 오브젝트 매니페스트 파일을 만든 뒤 실행합니다.

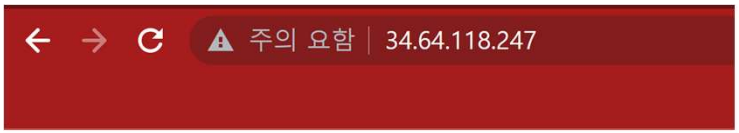
```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
  type: LoadBalancer
```



4. Replicaset

실습1 RS생성하기

정상적으로 오픈 되는 것을 확인 할 수 있습니다.



This is v1

```
NAME         TYPE           CLUSTER-IP     EXTERNAL-IP     PORT(S)        AGE
kubernetes   ClusterIP      10.36.0.1      <none>          443/TCP        27m
                            10.36.5.162    34.64.118.247   80:32118/TCP   4m34s
[frodo@console ~]$ curl 34.64.118.247
This is v1[frodo@console ~]$
```

4. Replicaset

3 실습2 복제 본 조정하기

아래 명령어로 복제 본(replicas)의 수를 조정 합니다.

kubectl scale --replicas="복제 본 수 " rs "Replicaset이름"

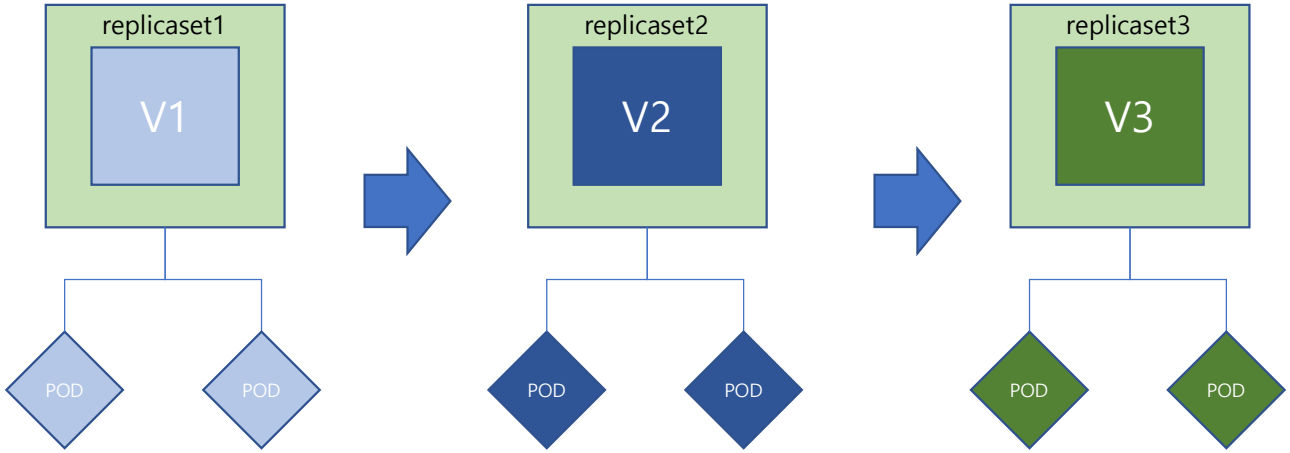
예) kubectl scale --replicas=5 rs nginx-rs1

```
[frodo@console ~]$ kubectl get pod
NAME              READY   STATUS    RESTARTS   AGE
nginx-rs1-c48w6   1/1     Running   0          20m
nginx-rs1-cd2db   1/1     Running   0          11s
nginx-rs1-fgc87   1/1     Running   0          11s
nginx-rs1-hr2c5   1/1     Running   0          20m
nginx-rs1-pgnft   1/1     Running   0          20m
```

4. Replicaset

4 운영 중인 이미지 업데이트 하기

운영중인 workload의 이미지를 v1에서 v2로 변경하고 싶습니다. 어떻게 해야 할까요?



4. Replicaset

5 새로운 Replicaset 생성하고 복제 본 조정하기

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs2
  labels:
    app: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: frodo5020/nginx:2.0
```

```
[frodo@console ~]$ kubectl get pod
NAME              READY   STATUS    RESTARTS   AGE
nginx-rs1-c48w6   1/1     Running   0          51m
nginx-rs1-cd2db   1/1     Running   0          30m
nginx-rs1-fgc87   1/1     Running   0          30m
nginx-rs1-hr2c5   1/1     Running   0          51m
nginx-rs1-pgnft   1/1     Running   0          51m
nginx-rs2-55njd   1/1     Running   0          18s
nginx-rs2-5p7rb   1/1     Running   0          18s
nginx-rs2-5snn5   1/1     Running   0          17s
nginx-rs2-6d2t7   1/1     Running   0          17s
nginx-rs2-rlqp8   1/1     Running   0          18s
[frodo@console ~]$
```

4. Replicaset

5 새로운 Replicaset 생성하고 복제 본 조정하기

```
[frodo@console ~]$ kubectl get svc
NAME         TYPE           CLUSTER-IP    EXTERNAL-IP     PORT(S)        AGE
kubernetes   ClusterIP      10.36.0.1     <none>          443/TCP        76m
nginx-svc    LoadBalancer   10.36.5.162   34.64.118.247   80:32118/TCP   53m
[frodo@console ~]$ while true; do sleep 1; curl http://34.64.118.247;done
This is v1This is v2This is v1This is v2This is v2This is v2This is v1This is v1This is
 v2This is v2This is v2This is v2This is v1This is v1This is v1This is v1This is v2This
 is v1This is v1This is v2This is v1This is v1
```

4. Replicaset

5 새로운 Replicaset 생성하고 복제 본 조정하기

아래 명령어로 v1 Replicaset의 복제 본 수를 0으로 조정 합니다.

kubectl scale --replicas= 0 rs "v1Replicaset 이름"

예) kubectl scale --replicas=0 rs nginx-rs1

```
[frodo@console ~]$ kubectl scale --replicas=0 rs nginx-rs1
replicaset.apps/nginx-rs1 scaled
[frodo@console ~]$ kubectl get pod
NAME              READY   STATUS    RESTARTS   AGE
nginx-rs2-55njd   1/1     Running   0          14m
nginx-rs2-5p7rb   1/1     Running   0          14m
nginx-rs2-5snn5   1/1     Running   0          14m
nginx-rs2-6d2t7   1/1     Running   0          14m
nginx-rs2-rlqp8   1/1     Running   0          14m
```

4. Replicaset

5 새로운 Replicaset 생성하고 복제 본 조정하기

```
[frodo@console ~]$ kubectl get svc
NAME         TYPE           CLUSTER-IP    EXTERNAL-IP     PORT(S)        AGE
kubernetes   ClusterIP      10.36.0.1     <none>          443/TCP        83m
nginx-svc    LoadBalancer   10.36.5.162   34.64.118.247   80:32118/TCP   60m
[frodo@console ~]$ while true; do sleep 1; curl http://34.64.118.247;done
This is v2This is v2This is v2This is v2This is v2This is v2This is v2This is v2This is
 v2This is v2
```

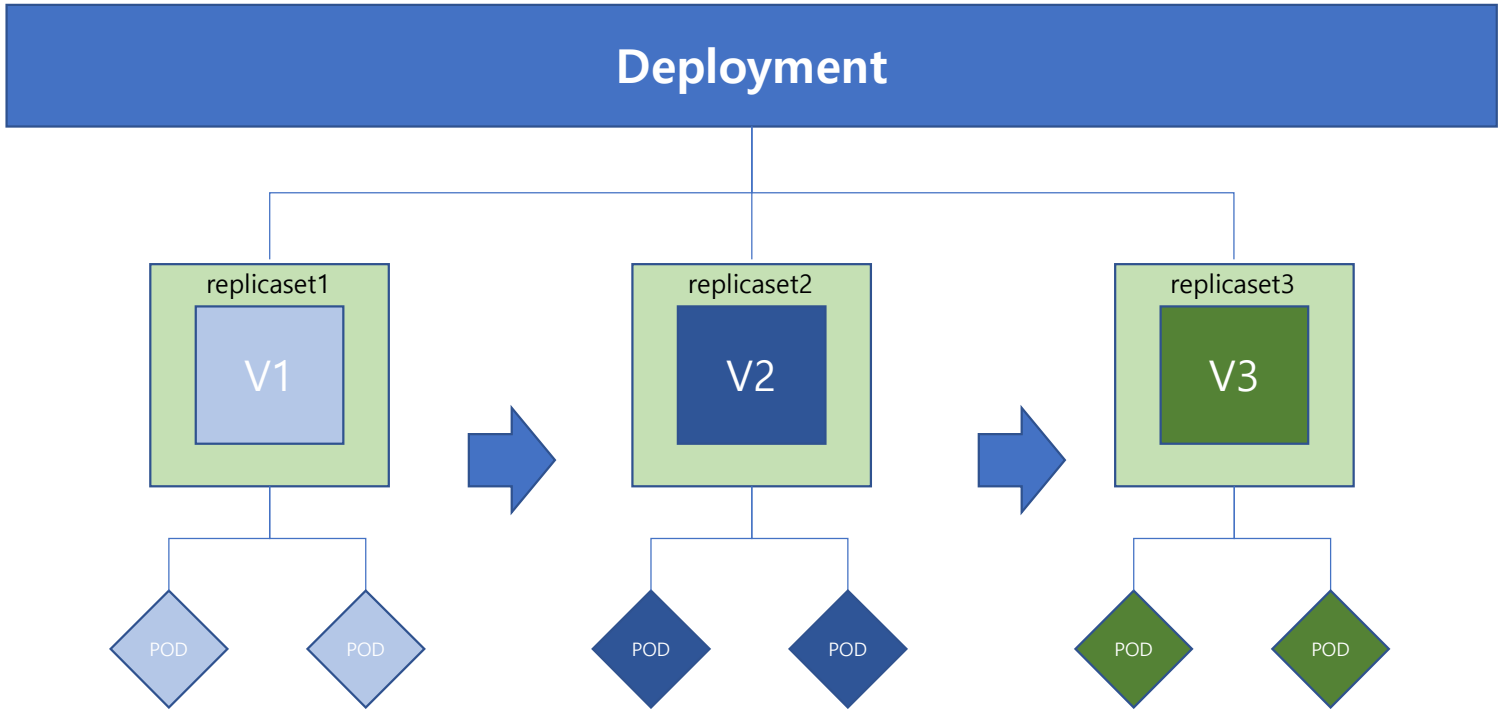
4. Replicaset

5 새로운 Replicaset 생성하고 복제 본 조정하기

```
[frodo@console ~]$ kubectl scale --replicas=5 rs nginx-rs1
cale --replicas=0 rs nginx-rs2
replicaset.apps/nginx-rs1 scaled
[frodo@console ~]$ kubectl scale --replicas=0 rs nginx-rs2
replicaset.apps/nginx-rs2 scaled
[frodo@console ~]$ kubectl get pod
NAME              READY   STATUS    RESTARTS   AGE
nginx-rs1-2nwbx   1/1     Running   0          6s
nginx-rs1-jc4fz   1/1     Running   0          6s
nginx-rs1-jsgf5   1/1     Running   0          6s
nginx-rs1-skm6r   1/1     Running   0          6s
nginx-rs1-wpgkm   1/1     Running   0          6s
[frodo@console ~]$
```

5. Deployment

1 개요



5. Deployment

실습1 Deployment를 이용하여 pod 배포하기

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 5
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: frodo5020/nginx:1.0
        ports:
        - containerPort: 80
```

```
[frodo@console ~]$ kubectl get deploy
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deploy   5/5     5            5           105s
[frodo@console ~]$ kubectl get rs
NAME                      DESIRED   CURRENT   READY   AGE
nginx-deploy-bcc76bcdb    5         5         5       108s
[frodo@console ~]$ kubectl get pod
NAME                            READY   STATUS    RESTARTS   AGE
nginx-deploy-bcc76bcdb-5wmtw    1/1     Running   0          110s
nginx-deploy-bcc76bcdb-c4wxd    1/1     Running   0          111s
nginx-deploy-bcc76bcdb-cfh7v    1/1     Running   0          110s
nginx-deploy-bcc76bcdb-pxpft    1/1     Running   0          111s
nginx-deploy-bcc76bcdb-vtc9z    1/1     Running   0          111s
```

5. Deployment

실습1 Deployment를 이용하여 pod 배포하기

```
[frodo@console ~]$ kubectl get svc
NAME         TYPE           CLUSTER-IP    EXTERNAL-IP     PORT(S)        AGE
kubernetes   ClusterIP      10.36.0.1     <none>          443/TCP        106m
nginx-svc    LoadBalancer   10.36.5.162   34.64.118.247   80:32118/TCP   83m
[frodo@console ~]$ while true; do sleep 1; curl http://34.64.118.247;done
This is v1This is v1This is v1This is v1This is v1This is v1This is v1This is v1This is
 v1This is v1This is v1This is v1This is v1This is v1This is v1This is v1This is v1This
 is v1
```

5. Deployment

3 실습2 Deployment이미지 변경하기

kubectl set image deployment.v1.apps/"Deployment 이름" "Container이름"="변경할 이미지 "

예) kubectl set image deployment.v1.apps/nginx-deploy nginx=frodo5020/nginx:2.0

```
[frodo@console ~]$ kubectl set image deployment.v1.apps/nginx-deploy nginx=frodo5020/ng
inx:2.0
deployment.apps/nginx-deploy image updated
[frodo@console ~]$ kubectl get rs
NAME                      DESIRED   CURRENT   READY   AGE
nginx-deploy-bcc76bcdb    0         0         0       9m42s
nginx-deploy-c5bfb66f5    5         5         5       9s
[frodo@console ~]$ kubectl get pod
NAME                            READY   STATUS    RESTARTS   AGE
nginx-deploy-c5bfb66f5-5725l    1/1     Running   0          11s
nginx-deploy-c5bfb66f5-7wz4p    1/1     Running   0          12s
nginx-deploy-c5bfb66f5-hj5vd    1/1     Running   0          10s
nginx-deploy-c5bfb66f5-lp9kp    1/1     Running   0          12s
nginx-deploy-c5bfb66f5-tcl89    1/1     Running   0          12s
```

5. Deployment

3 실습2 Deployment이미지 변경하기

```
[frodo@console ~]$ kubectl get svc
NAME         TYPE           CLUSTER-IP    EXTERNAL-IP     PORT(S)        AGE
kubernetes   ClusterIP      10.36.0.1     <none>          443/TCP        115m
nginx-svc    LoadBalancer   10.36.5.162   34.64.118.247   80:32118/TCP   91m
[frodo@console ~]$ while true; do sleep 1; curl http://34.64.118.247;done
This is v2This is v2This is v2This is v2This is v2This is v2This is v2This is v2This is
 v2This is v2This is v2This is v2This is v2This is v2This is v2
```

5. Deployment

3 실습2 Deployment이미지 변경하기

kubectl rollout history deployment/Deployment이름

kubectl rollout history deployment/nginx-deploy

```
[frodo@console ~]$ kubectl rollout history deployment/nginx-deploy
deployment.apps/nginx-deploy
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

[frodo@console ~]$
```

5. Deployment

3 실습2 Deployment이미지 변경하기

kubectl rollout history deployment/nginx-deploy --revision=1

```
[frodo@console ~]$ kubectl rollout history deployment/nginx-deploy --revision=1
deployment.apps/nginx-deploy with revision #1
Pod Template:
  Labels:       app=nginx
        pod-template-hash=bcc76bcdb
  Containers:
   nginx:
    Image:      frodo5020/nginx:1.0
    Port:       80/TCP
    Host Port:  0/TCP
    Environment:        <none>
    Mounts:     <none>
  Volumes:      <none>
```

5. Deployment

3 실습2 Deployment이미지 변경하기

kubectl rollout history deployment/nginx-deploy --revision=2

```
[frodo@console ~]$ kubectl rollout history deployment/nginx-deploy --revision=2
deployment.apps/nginx-deploy with revision #2
Pod Template:
  Labels:       app=nginx
        pod-template-hash=c5bfb66f5
  Containers:
   nginx:
    Image:      frodo5020/nginx:2.0
    Port:       80/TCP
    Host Port:  0/TCP
    Environment:        <none>
    Mounts:     <none>
  Volumes:      <none>
```

5. Deployment

4 실습3 롤백하기

kubectl rollout undo deployment/nginx-deploy

```
[frodo@console ~]$ kubectl rollout undo deployment/nginx-deploy
deployment.apps/nginx-deploy rolled back
[frodo@console ~]$ kubectl rollout history deployment/nginx-deploy
deployment.apps/nginx-deploy
REVISION  CHANGE-CAUSE
2         <none>
3         <none>
```

5. Deployment

4 실습3 롤백하기

kubectl rollout history deployment/nginx-deploy --revision=3

```
[frodo@console ~]$ kubectl rollout history deployment/nginx-deploy --revision=3
deployment.apps/nginx-deploy with revision #3
Pod Template:
  Labels:       app=nginx
        pod-template-hash=bcc76bcdb
  Containers:
   nginx:
    Image:      frodo5020/nginx:1.0
    Port:       80/TCP
    Host Port:  0/TCP
    Environment:        <none>
    Mounts:     <none>
  Volumes:      <none>
```

5. Deployment

실습3 롤백하기

kubectl rollout undo deployment/nginx-deploy --to-revision=2

```
[frodo@console ~]$ kubectl rollout undo deployment/nginx-deploy --to-revision=2
deployment.apps/nginx-deploy rolled back
[frodo@console ~]$ kubectl rollout history deployment/nginx-deploy
deployment.apps/nginx-deploy
REVISION  CHANGE-CAUSE
3         <none>
4         kubectl set image deployment.v1.apps/nginx-deploy nginx=frodo5020/nginx:3.0 -
-record=true
5         <none>
```

6. Service

1 개요

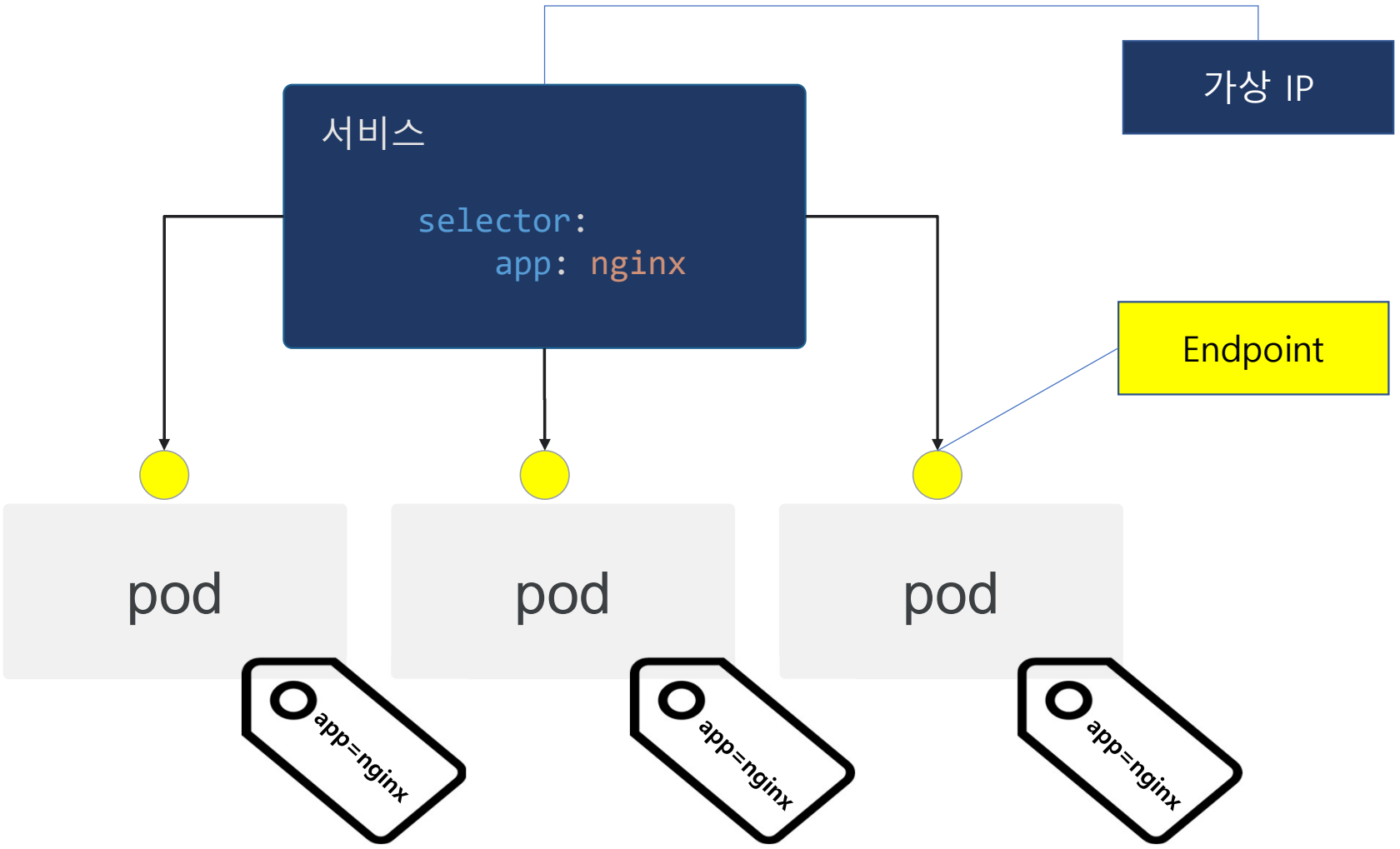
POD는 일시적

POD의 IP를 예측할 수 없습니다.

동일한 서비스를 제공하는 여러 POD에 접근하기 위한 단일 IP가 필요

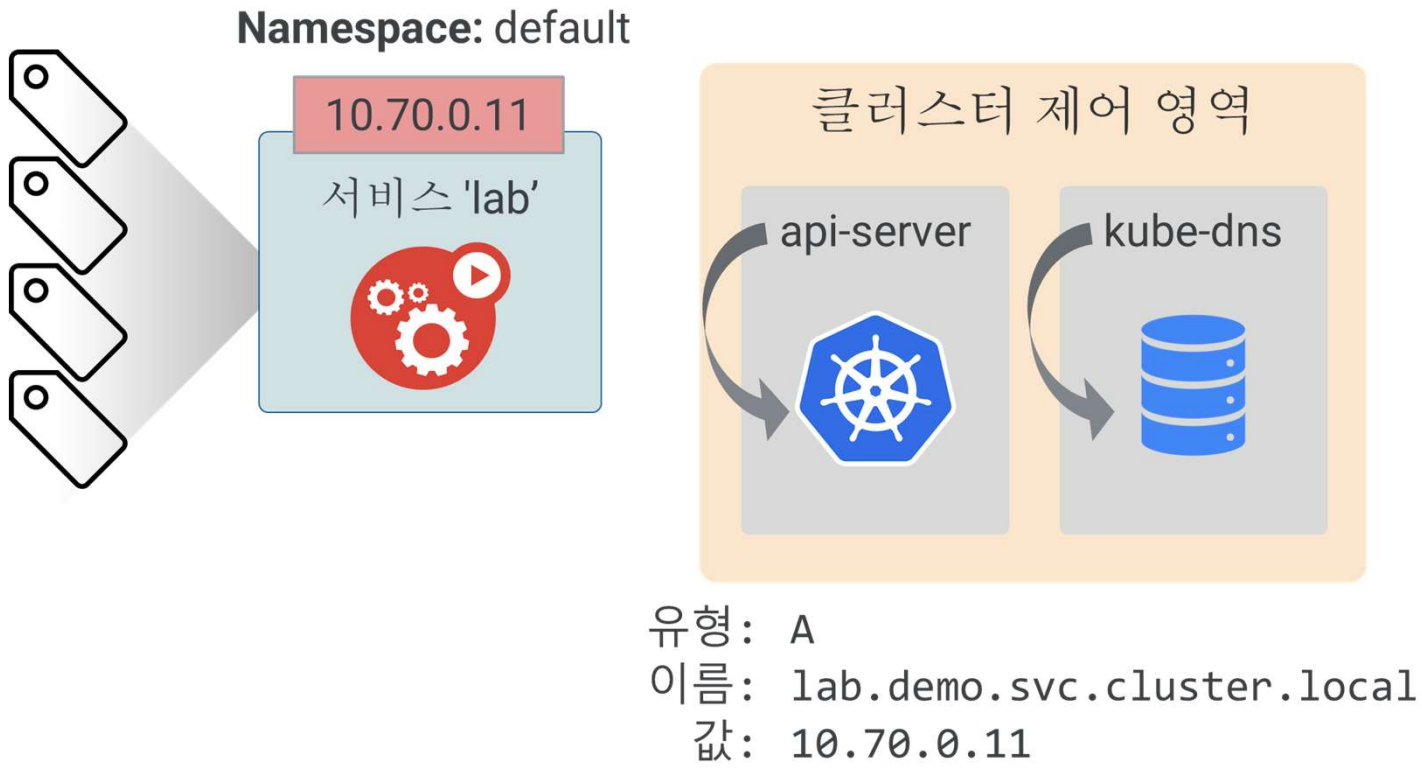
6. Service

1 개요



6. Service

2 서비스 DNS




```
유형: A
이름: lab.demo.svc.cluster.local
  값: 10.70.0.11
```

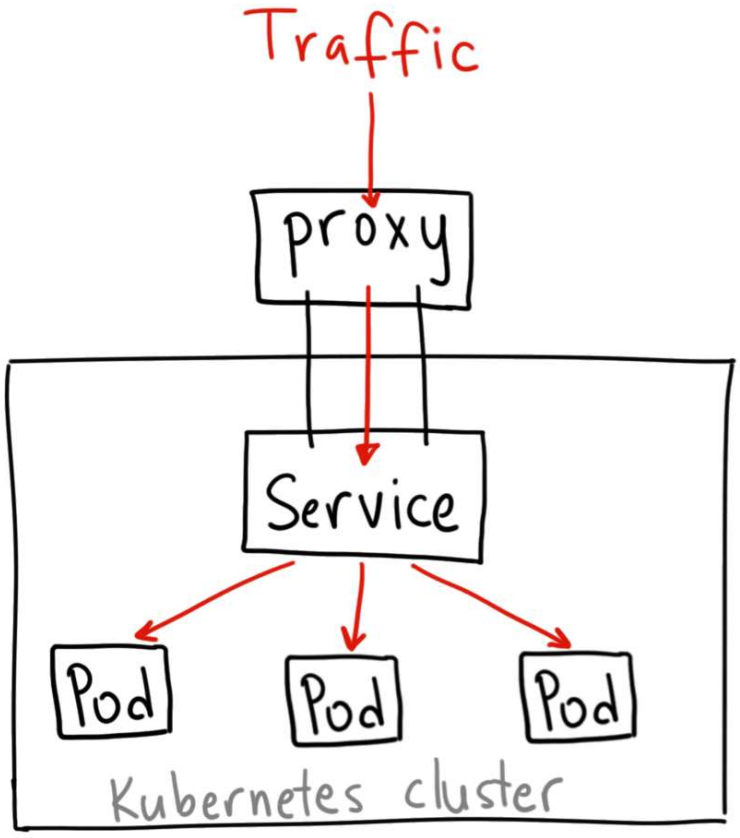
6. Service

서비스 유형



6. Service

ClusterIP



ClusterIP 유형의 서비스 구조 (이미지 출처 : Google Cloud)

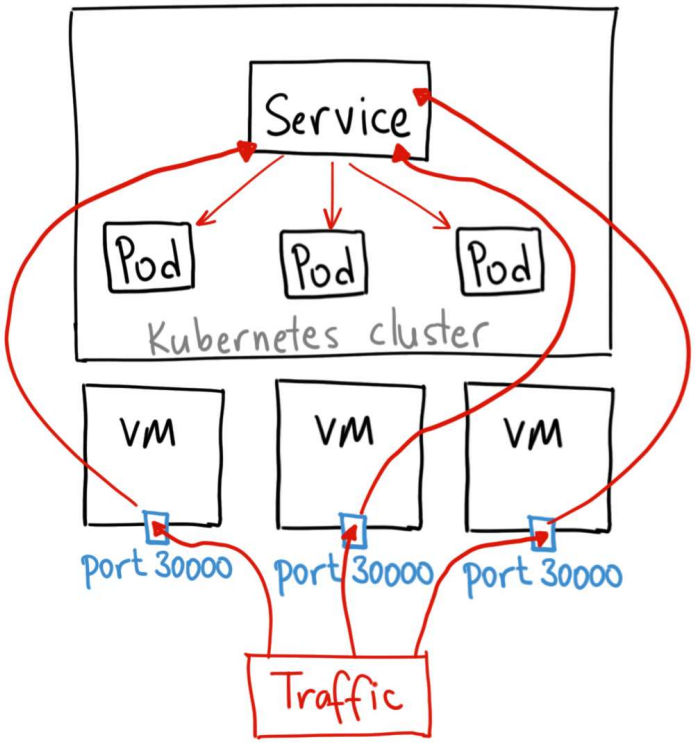
6. Service

ClusterIP

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
  type: ClusterIP
```

6. Service

4 NodePort



ClusterIP 유형의 서비스 구조 (이미지 출처 : Google Cloud)

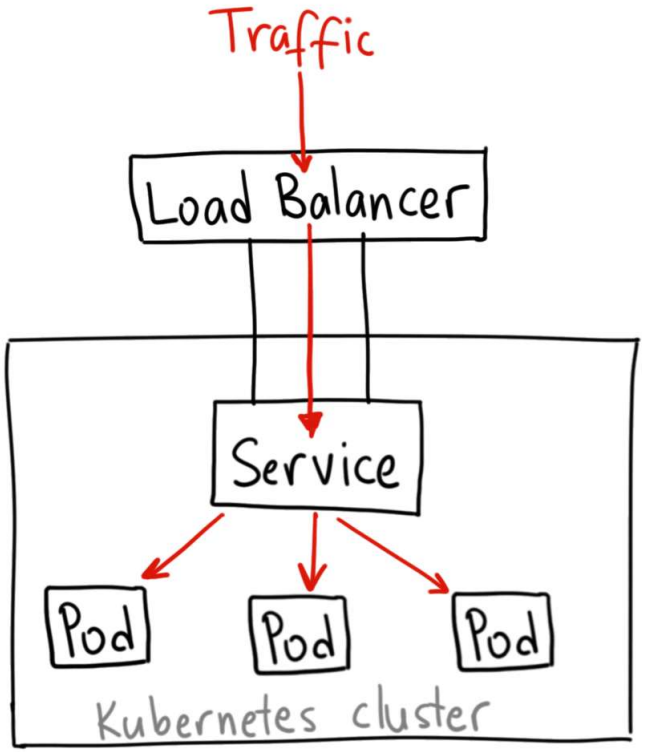
6. Service

NodePort

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30000
  type: NodePort
```

6. Service

LoadBalancer



ClusterIP 유형의 서비스 구조 (이미지 출처 : Google Cloud)

6. Service

LoadBalancer

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30000
  type: LoadBalancer
```

6. Service

실습 준비하기

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 5
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: frodo5020/nginx:1.0
        ports:
        - containerPort: 80
```

6. Service

실습1 ClusterIP 유형의 서비스 생성하기

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
```

6. Service

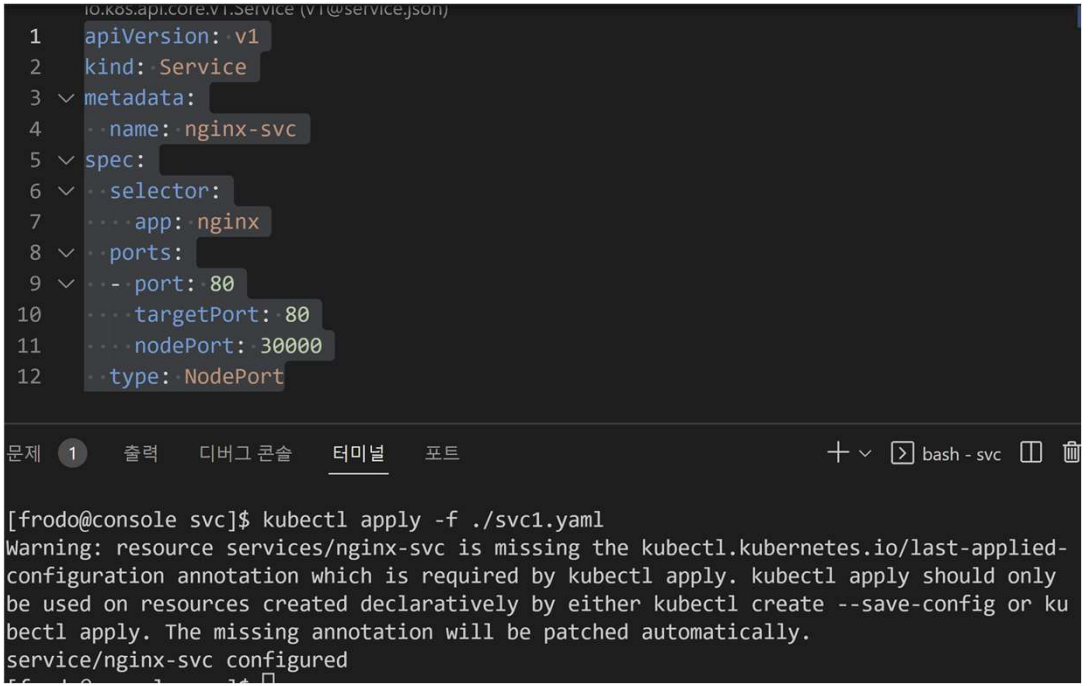
실습1 ClusterIP 유형의 서비스 생성하기

```
[frodo@console ~]$ kubectl get svc
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.36.0.1      <none>        443/TCP   9m57s
nginx-svc    ClusterIP   10.36.12.212   <none>        80/TCP    8m23s
[frodo@console ~]$ kubectl run -it --rm centosshell --image=centos /bin/bash
If you don't see a command prompt, try pressing enter.
[root@centosshell /]# curl 10.36.12.212
This is v1[root@centosshell /]# curl nginx-svc
This is v1[root@centosshell /]# curl nginx-svc.default.svc.cluster.local
This is v1[root@centosshell /]#
```

6. Service

실습2 NodePort 유형의 서비스 생성하기

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30000
  type: NodePort
```



6. Service

실습2 NodePort 유형의 서비스 생성하기

```
be used on resources created declaratively by either kubectl create --save-config or ku
bectl apply. The missing annotation will be patched automatically.
service/nginx-svc configured
[frodo@console svc]$ kubectl get svc
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.36.0.1      <none>        443/TCP        20m
nginx-svc    NodePort    10.36.12.212   <none>        80:30000/TCP   18m
[frodo@console svc]$
```

6. Service

실습2 NodePort 유형의 서비스 생성하기

```
[frodo@console svc]$ kubectl get node -o wide
NAME                                STATUS   ROLES    AGE     VERSION           INTERNAL-IP   EXTERNAL-IP    OS-IMAGE
         KERNEL-VERSION   CONTAINER-RUNTIME
gke-frodo-default-pool-f448d205-56v3   Ready    <none>   5h24m   v1.23.12-gke.100   10.178.0.13   34.64.70.21    Container-Optimized OS from
Google   5.10.133+        containerd://1.5.13
gke-frodo-default-pool-f448d205-sf1w   Ready    <none>   5h24m   v1.23.12-gke.100   10.178.0.14   34.64.222.44   Container-Optimized OS from
Google   5.10.133+        containerd://1.5.13
gke-frodo-default-pool-f448d205-vsdm   Ready    <none>   5h24m   v1.23.12-gke.100   10.178.0.12   34.64.74.206   Container-Optimized OS from
[frodo@console svc]$ curl 34.64.70.21:30000
This is v1[frodo@console svc]$ curl 34.64.222.44:30000
This is v1[frodo@console svc]$ curl 34.64.74.206:30000
This is v1[frodo@console svc]$
```



This is v1

6. Service

실습3 LoadBalancer 유형의 서비스 생성하기

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30000
  type: LoadBalancer
```



6. Service

실습3 LoadBalancer 유형의 서비스 생성하기

```
[frodo@console svc]$ kubectl get svc
NAME         TYPE           CLUSTER-IP     EXTERNAL-IP     PORT(S)        AGE
kubernetes   ClusterIP      10.36.0.1      <none>          443/TCP        35m
nginx-svc    LoadBalancer   10.36.12.212   34.64.118.247   80:30000/TCP   34m
[frodo@console svc]$ curl 34.64.118.247
This is v1[frodo@console svc]$
```

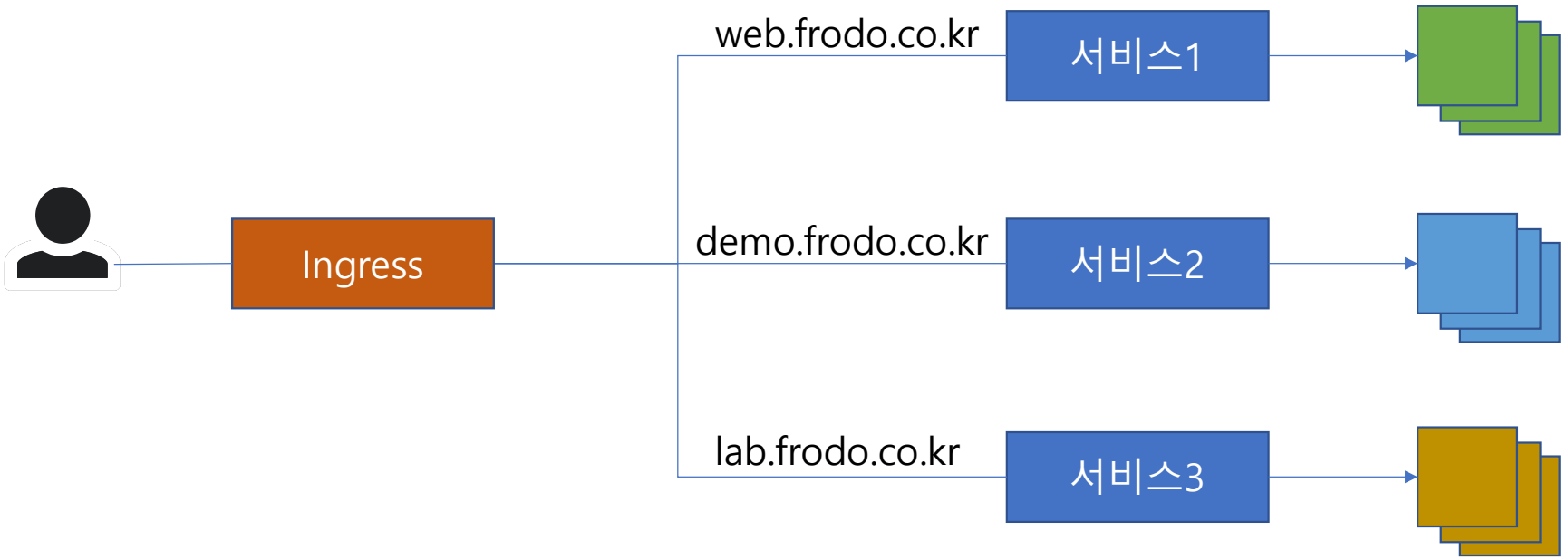
6. Service

10 여러 개의 서비스를 연결하기



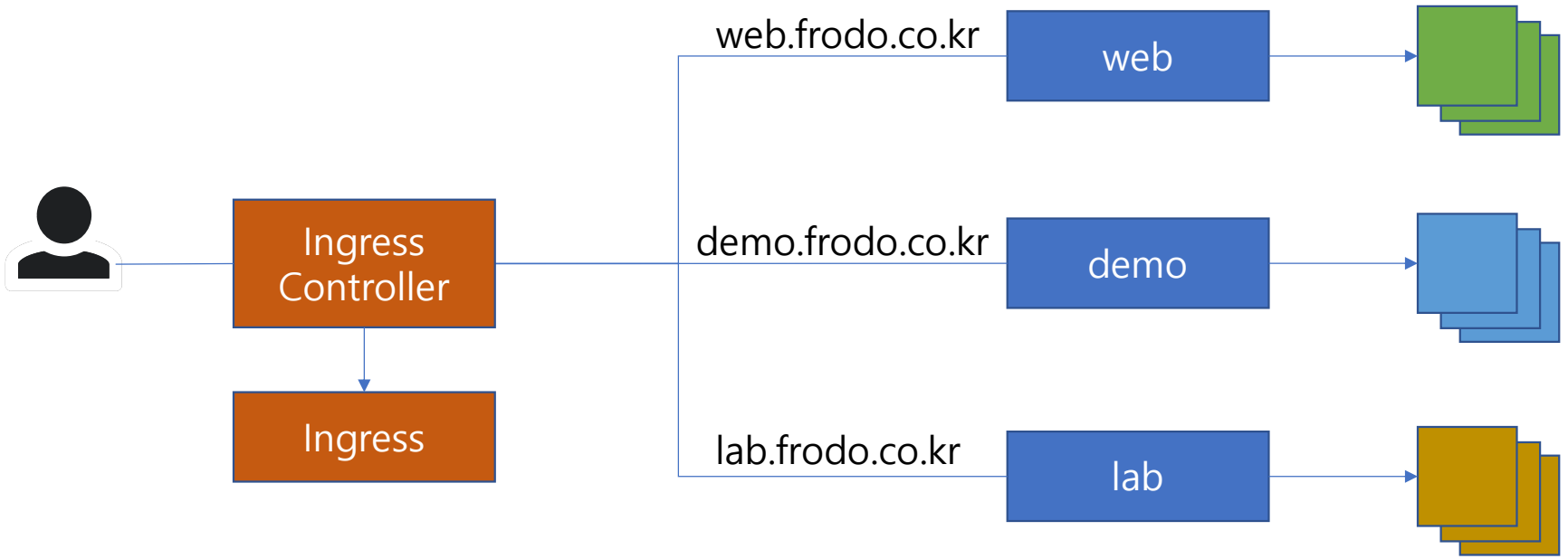
7. Ingress

1 개요



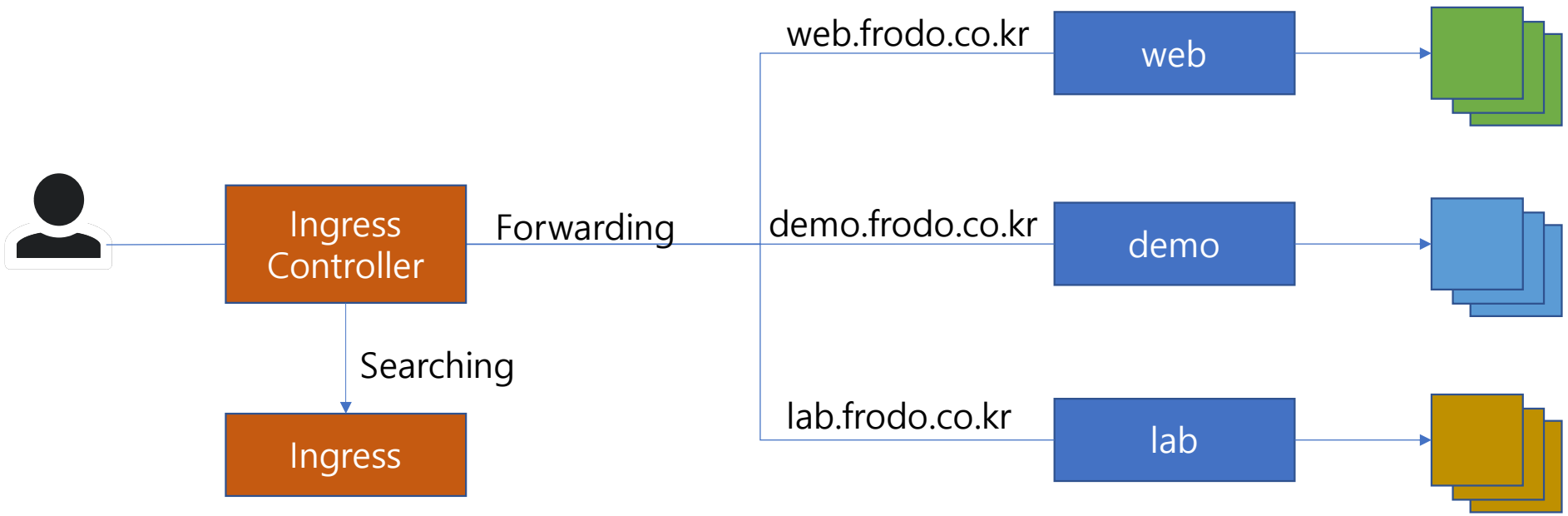
7. Ingress

2 Ingress Controller



7. Ingress

Ingress Controller



7. Ingress

3 Ingress 매니페스트 파일

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myingress
spec:
  rules:
  - host: web.forod.co.kr          # Host 헤더가 일치하는 요청만 매칭
    http:
      paths:
      - pathType: Prefix
        path: "/"                  # /로 시작하는 모든 경로에 매칭
        backend:
          service:
            name: web
            port:                  # web 서비스의 80포트로 포워딩
              number: 80
```

7. Ingress

4 Ingress 특징

